

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Capstone Project (CO4337)

**ALICE – AN INTELLIGENT PERSONAL ASSISTANT
FOR TIME OPTIMIZATION, HEALTH MANAGEMENT,
AND VOICE-BASED NOTE TAKING**

Major: Computer Science

Thesis Committee:	Computer Science - 04 CC
Supervisor(s):	Assoc. Prof. Quan Thanh Tho, Ph.D
Reviewer:	Tran Hong Tai, M.Eng
Student 1:	Dinh Ba Khanh (2252323)
Student 2:	Nguyen Anh Khoa (2252351)
Student 3:	Tran Chi Tai (2252727)

Ho Chi Minh City, May 2026



Instruction's Signature

Date: _____

Assoc. Prof. Quan Thanh Tho, Ph.D (Project Instructor)
Associate Professor
Faculty of Computer Science and Engineering



Acknowledgement

First and foremost, we would like to express our deepest gratitude to Assoc. Prof. Qu  n Th n Th  for his invaluable guidance. His profound knowledge and dedicated support have been instrumental to the success of this specialized project.

Furthermore, we extend our sincere appreciation to the lecturers of the Faculty of Computer Science and Engineering, Vietnam National University - Ho Chi Minh City University of Technology. The fundamental knowledge and solid foundation provided by the faculty have been essential in enabling us to execute and complete this project effectively.

Finally, we would like to thank our families and friends for their unwavering support, encouragement, and for always being by our side throughout this journey.

Ho Chi Minh City, May 2026

The authors,

D nh B  Kh nh

Nguy n Anh Khoa

Tr n Ch  T i



Declaration Of Authenticity

My team consists of Đinh Bá Khánh, Nguyễn Anh Khoa, and Trần Chí Tài. We declare that we solely conducted this specialized project under the supervision of Assoc. Prof. Quản Thành Thơ at the Faculty of Computer Science and Engineering, Vietnam National University, Ho Chi Minh City University of Technology.

Throughout the project, our team ensures that all references and external sources utilized are appropriately cited and thoroughly documented.

Ho Chi Minh City, May 2026

The authors,

Đinh Bá Khánh

Nguyễn Anh Khoa

Trần Chí Tài



Abstract

In the era of rapid technological advancement, managing personal productivity and well-being has become increasingly complex. This thesis presents the development of Alice, a smart personal assistant mobile application designed to optimize time management, track basic health metrics, and streamline note-taking through seamless voice and image interactions.

The core intelligence of Alice is powered by integrating Large Language Model (LLM) APIs, which serve as the primary reasoning engine for processing user inputs. By leveraging the advanced natural language understanding capabilities of state-of-the-art LLMs, the system eliminates the need for traditional, rigid NLP modules. The processing pipeline utilizes OpenAI Whisper for high-accuracy speech-to-text transcription, followed by LLM-based analysis to extract intent and transform unstructured voice commands into actionable calendar events and organized tasks.

A key innovation of this project is the multi-modal scheduling feature, which allows users to generate schedules not only from voice but also from visual data. By processing images of academic timetables or event posters, the system automatically identifies temporal details and event descriptions to populate the user's calendar. Furthermore, Alice provides essential health monitoring by tracking fundamental physical metrics, including height, weight, and Body Mass Index (BMI). Built with Flutter for the cross-platform client, FastAPI for the backend services, and MongoDB for flexible data management, Alice offers a unified and intuitive solution to help users regain control over their daily lives through the power of artificial intelligence.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Goal	1
1.3	Scope	2
1.4	Thesis Structure	2
2	Technology Stacks	4
2.1	Client-side Development	4
2.1.1	Flutter	4
2.1.2	React Native	4
2.1.3	Kotlin Multiplatform	5
2.1.4	Comparison and Mobile Development Technology Selection for the Project	6
2.2	Server Development	7
2.2.1	Django	7
2.2.2	FastAPI	7
2.2.3	Spring Boot	7
2.2.4	Node.js	7
2.2.5	Comparison and Server-side Technology Selection for the Project	9
2.3	Database Management System	10
2.3.1	MongoDB	10
2.3.2	PostgreSQL	10
2.3.3	Comparison and Database Management System Selection for the Project	11
2.3.4	Vector Database for RAG	11
2.4	Generative AI	12
3	System Design and Analysis	14
3.1	Related Works	14
3.1.1	Google Calendar	14
3.1.2	Notion Calendar	14
3.1.3	Reclaim AI	15
3.1.4	Comparative Analysis: Smart Schedule AI	15
3.2	Requirement Analysis	17
3.2.1	Domain Context and Stakeholders	17
3.2.2	Functional Requirements	18
3.2.3	Non-Functional Requirements	21
3.3	Use-case Design	24
3.3.1	Use-case	24



3.3.2	Use-case Descriptions	31
3.4	Architecture Design	43
3.4.1	Architecture Strategy and Pattern Selection	43
3.4.2	Designing Activity Diagrams	52
3.4.3	Component Diagram	60
3.5	Database Design	65
3.5.1	Class Diagram Analysis	65
4	System Implementation	67
4.1	Back-End Implementation	67
4.1.1	Service Structure	67
4.1.2	API Design	68
4.1.3	Generative AI Integration in Alice	69
4.1.3.1	Speech-To-Schedule (s2s) Workflow	69
4.1.3.2	Image-To-Schedule (i2s) Workflow	70
4.1.3.3	RAG Database Generation	71
4.2	Mobile Application Implementation	71
4.2.1	Application Structure	71
4.2.2	API Integration	73
4.2.2.1	Dio Client	73
4.2.2.2	API Integration Architecture	73
4.2.3	Data Management Service	74
4.2.3.1	Hive - Offline Service	74
4.2.3.2	Cache Service	75
4.2.4	Schedule Module	76
4.2.4.1	Calendar View By Week	76
4.2.4.2	Calendar View By Month	78
4.2.4.3	Calendar Drawer	80
4.2.5	Task Module	83
4.2.5.1	Task Home Page	83
4.2.5.2	Task Detail Page	85
5	Deployment and System Evaluation	89
5.1	Deployment	89
5.1.1	Mobile Application Deployment	89
5.1.1.1	Android Deployment	89
5.1.1.2	iOS Deployment	90
5.1.1.3	Additional Considerations	90
5.1.2	Backend Deployment	91
5.1.2.1	Alignment with Cloud Native Computing Foundation (CNCF) Standards	91
5.1.2.2	Containerization Strategy	91
5.1.2.3	Cloud Native Infrastructure	92
5.1.2.4	Stateless Workload Management	95
5.1.2.5	Stateful Workload and Data Persistence	97
5.1.2.6	Networking, Security and External Traffic	98
5.2	Testing and Results	99
5.2.1	Test Data	99
5.2.2	Test Plan	100



5.3	Non-functional Testing	105
5.3.1	Usability Test	105
5.3.2	Performance Test	106
6	Work Evaluation and Future Improvement	107
6.1	Work Evaluation	107
6.2	Future Improvements	108
	References	109
A	Appendix: Task Assignment	112
B	Appendix: Timeline	114

List of Figures

3.1	General Use-case view	25
3.2	Use-case Authentication and Personal Information	26
3.3	Use-case Schedule Management	27
3.4	Use-case Task Management	28
3.5	Use-case Speech To Schedule	29
3.6	Use-case Image To Schedule	30
3.7	Architecture Diagram	45
3.8	Client and Gateway architecture view	46
3.9	Schedule architecture view	47
3.10	Task architecture view	48
3.11	Authentication and Personal Information architecture view	49
3.12	Speech To Schedule architecture view	50
3.13	Image To Schedule architecture view	51
3.14	Add Event Activity Diagram	52
3.15	Add Goal Activity Diagram	53
3.16	Add Task Activity Diagram	54
3.17	Edit Event Activity Diagram	55
3.18	Assign Task To Goal Activity Diagram	56
3.19	ASR Activity Diagram	57
3.20	OCR Activity Diagram	58
3.21	Confirm ASR Action Activity Diagram	59
3.22	Component Diagram	61
3.23	Client and Gateway component view	62
3.24	Schedule component view	62
3.25	Task component view	63
3.26	Authentication and Personal Information component view	63
3.27	Speech To Schedule component view	64
3.28	Image To Schedule component view	64
3.29	Detailed System Class Diagram	66
4.1	RESTful API design and documentation for the Alice Task Module	69
4.2	Architecture and execution flow of the Speech-to-Schedule module using RAG . .	70
4.3	Execution flow of the Image-to-Schedule module using Vision AI	71
4.4	Weekly calendar interface for schedule management	76
4.5	Weekly calendar interface for schedule management	78
4.6	Calendar drawer for view configuration and category filtering	80
4.7	Event creation interface	81



4.8	Task management interface with daily task list	83
4.9	Goal overview and task grouping interface	85
4.10	Goal creation and editing interface	87
5.1	Docker Containerization	91
5.2	Docker Hub	92
5.3	Kubernetes	93
5.4	How kubectl is used in system	93
5.5	Digital Ocean	94
5.6	Minikube	94
5.7	DigitalOcean Dashboard	95
5.8	Services and Deployments in Alice	96
5.9	NGINX Ingress Controller	96
5.10	ConfigMaps and Secrets	97
5.11	Persistent Volume Claims	98
5.12	Ingress Configuration	98
5.13	Cert Manager, Let's Encrypt, and Namecheap	99

List of Tables

2.1	Comparison of Mobile Development Technologies	6
2.2	Comparison of Mobile Backend Frameworks	9
2.3	Comparison of PostgreSQL and MongoDB	11
2.4	Overview of Integrated Generative AI Models	12
2.5	Model Parameters and API Rate Limits	13
3.1	Comparative Analysis of Existing Applications and Smart Schedule AI	16
3.2	Use Case UC-01: Add Personal Event	31
3.3	Use Case UC-02: Add Personal Goal	32
3.4	Use Case UC-03: Add Personal Task	33
3.5	Use Case UC-04: Edit Personal Event(s)	34
3.6	Use Case UC-05: Assign Personal Task to Goal	36
3.7	Use Case UC-06: Process Voice Command (ASR)	37
3.8	Use Case UC-07: Extract Entities from Image (OCR)	39
3.9	Use Case UC-08: Confirm ASR Action	41

Chapter 1

Introduction

In chapter 1, the motivation, goal, and scope of the application are illustrated. The outline of the report is also presented.

1.1 Motivation

In the modern era, the exponential growth of information and the increasing complexity of daily responsibilities have made effective personal organization a mandatory survival skill. While the market is flooded with productivity tools, most remain "static" and "fragmented." Users are often forced to manually input every detail into separate applications for calendars, to-do lists, and health tracking. This manual overhead creates a "productivity paradox," where the tools specifically designed to save time actually consume it through tedious data entry and constant context switching.

Furthermore, the barrier to efficient planning often lies in the source of information. Vital schedules are frequently trapped in non-digital formats, such as printed university timetables or event posters on social media. Traditional applications fail to bridge this gap, requiring users to transcribe data manually. **Alice** is motivated by the vision of a "Minimalist Intelligence"—a system that replaces manual friction with AI-driven intuition. By leveraging Large Language Models (LLMs) and multi-modal processing involving both voice and vision, this project aims to create a centralized "brain" that understands human intent across different formats, allowing users to focus on execution rather than administration.

1.2 Goal

The primary objective of this project is to develop **Alice**, an intelligent personal assistant that serves as a unified hub for time management and personal well-being. A fundamental goal is to achieve intelligent automation, eliminating the burden of manual scheduling by using LLM APIs to transform unstructured voice commands and visual data into structured, actionable calendar events. By consolidating productivity tools, the project aims to provide a single, cross-platform interface that seamlessly integrates task management and scheduling, thereby reducing the cognitive load caused by fragmented workflows.



Additionally, the project seeks to promote essential health awareness. This is achieved by providing a streamlined module for tracking fundamental physical metrics such as BMI, height, and weight, ensuring users maintain a baseline of health monitoring amidst a busy work-intensive schedule. Ultimately, the system strives for a high-efficiency interaction model, emphasizing a "voice-first" and "image-centric" user experience that minimizes the time spent interacting with the screen, making personal management both accessible and effortless.

1.3 Scope

The scope of this project encompasses the design and development of a cross-platform mobile application using the Flutter framework, ensuring full compatibility with both Android and iOS platforms. The functional core of the system is limited to three primary modules: schedule and task management, multi-modal input processing, and basic health tracking. The schedule module supports comprehensive operations for managing events and tasks, while the multi-modal component integrates OpenAI Whisper for voice-to-text transcription and Optical Character Recognition (OCR) technologies to extract scheduling data from images such as academic timetables and event posters.

Regarding the technical implementation, the project focuses on utilizing state-of-the-art LLM APIs to perform intent extraction and generate structured outputs from unstructured user inputs. The backend infrastructure is restricted to a server-side architecture built with FastAPI and a MongoDB database to manage user data and facilitate AI processing pipelines. Furthermore, the scope of health management is specifically narrowed to the recording and calculation of basic physical metrics, including height, weight, and Body Mass Index (BMI). System evaluation is conducted through functional and non-functional testing, specifically focusing on AI processing latency, OCR accuracy, and the overall reliability of the notification system using Black-box testing methodologies.

1.4 Thesis Structure

This thesis is organized into five main chapters, each addressing a specific stage of the system design, development, and evaluation process.

- **Chapter 1 – Introduction:** This chapter introduces the background and motivation of the project. It outlines the problem statement, research objectives, and scope of the work.
- **Chapter 2 – Technology Stacks:** This chapter presents the technology stack selected for system development, including mobile development frameworks, server-side technologies, and the integration of LLM APIs.
- **Chapter 3 – System Design and Analysis:** This chapter focuses on the overall system architecture, requirement analysis, and the design of the multi-modal processing pipeline for voice and image data.
- **Chapter 4 - System Implementation:** This chapter illustrates how we implement each part of our system, how each service organizes its files, how we design the APIs, the workflows of s2s and i2s.



- **Chapter 5 – Deployment and System Evaluation:** This chapter details the system deployment and presents a comprehensive evaluation through functional Black-box testing (including ASR and OCR modules) and non-functional performance testing.
- **Chapter 6 – Work Evaluation and Future Improvement:** The final chapter evaluates the implemented system, discusses achieved results, identifies limitations, and proposes future development directions.

Chapter 2

Technology Stacks

In Chapter 2, the authors present insights into the technologies used in developing the mobile application. These include mobile application development, server-side development, database management systems, and machine learning technologies.

2.1 Client-side Development

2.1.1 Flutter

Flutter, first introduced by Google in 2017, has quickly evolved into one of the most influential frameworks for building cross-platform applications. Its modern architecture and high-performance rendering engine have reshaped how developers create mobile, web, and desktop interfaces.

Flutter applications are constructed using widgets, the fundamental building blocks that define both structure and behavior. Widgets describe their visual configuration and update themselves when their internal state changes. The framework efficiently computes the minimal updates needed to transition between UI states through a diff-based rendering process. This widget-driven approach enables consistent, reactive, and expressive user interfaces across multiple platforms. [1]

2.1.2 React Native

React Native, introduced by Meta in 2015, has become a widely adopted framework for developing cross-platform mobile applications using JavaScript and React principles. Its core idea is to allow developers to write application logic once while rendering platform-specific native UI components, resulting in applications that closely match the performance and behavior of purely native apps.

React Native uses a declarative component model, where UI elements are defined as components that respond to changes in state. Communication between JavaScript logic and native platform APIs is facilitated through a bridge mechanism, enabling access to device features and native modules. This architecture makes React Native well-suited for rapid development cycles and teams already familiar with React and web technologies. [2]



2.1.3 Kotlin Multiplatform

Kotlin Multiplatform, developed by JetBrains, enables the sharing of application logic across multiple platforms while preserving native UI and platform-specific capabilities. Rather than providing a unified rendering system, Kotlin Multiplatform focuses on code reuse at the business-logic level, such as networking, data processing, and state management. This design allows developers to maintain native performance and native interface components on Android, iOS, desktop, and web targets.

A Kotlin Multiplatform project typically separates common code from platform-specific code, with shared modules compiled to different targets such as JVM, JavaScript, or native binaries. Platform-specific modules integrate with the shared logic while implementing UI elements and device APIs. This approach provides flexibility, reduces duplication, and is especially beneficial for large-scale or long-term projects that require consistency across multiple platforms. [3]

2.1.4 Comparison and Mobile Development Technology Selection for the Project

Framework	Pros	Cons
Flutter	Consistent UI Across All Platforms Uses Skia engine for uniform rendering across Android, iOS, web, and desktop. Customizable Widget System Large widget library supporting expressive, custom UIs with minimal boilerplate.	Requires Learning Requires mastering Dart and a widget-centric architecture/mental model. Self-creating UI Manual implementation required for platform-specific navigation and accessibility.
React Native	JavaScript + React Ecosystem Leverages existing JS tools and knowledge for reduced onboarding time. Fast Iteration + Modular Architecture Supports rapid UI testing via fast refresh and flexible native module integration.	Performance Bottlenecks JavaScript-Native bridge may introduce latency in complex, real-time updates. Requires Native Modules for Advanced Features Advanced tasks often require platform-specific expertise (Kotlin, Swift).
Kotlin Multi-platform	Shared Business Logic with Fully Native UI Reduces code duplication while maintaining platform-specific user experiences. Performance and Platform Integration Compiles to native binaries with direct access to all platform-native APIs.	Requires Kotlin and Swift UIs must be written separately for each platform using native frameworks. Limited Learning Resources Smaller ecosystem with fewer ready-made libraries compared to competitors.

Table 2.1: Comparison of Mobile Development Technologies

Based on the comparative analysis of advantages and disadvantages in **Table 2.1**, our team has decided to adopt **Flutter** as the primary framework for mobile development. Flutter's highly customizable widget system enables the creation of rich, interactive user interfaces, making it especially suitable for applications that require enhanced user engagement. In addition, its ability to target multiple platforms through a unified codebase aligns with our long-term objective of expanding to additional platforms in the future. Considering the team's available development time and skill capacity, we conclude that Flutter offers the most balanced solution and is capable of meeting all current and foreseeable project requirements.



2.2 Server Development

2.2.1 Django

Django is a high-level Python web framework created in 2003 by Adrian Holovaty and Simon Willison at the Lawrence Journal-World. It was named after the guitarist Django Reinhardt and officially released to the public in 2005 [4]. The framework encourages rapid development and clean design by managing common web development tasks, freeing developers from "reinventing the wheel." It is also free and open source [5].

Django utilizes the Model-View-Template (MVT) pattern [6], wherein the Model manages the data structure and database queries, the View handles incoming web requests and returns the appropriate response, and the Template controls the layout and presentation of data.

2.2.2 FastAPI

FastAPI is a modern, high-performance web framework for building APIs with Python, originally released in December 2018 by Sebastián Ramírez [14]. It is designed to build APIs using standard Python type hints, ensuring valid data structures and editor support.

According to the official documentation [13], FastAPI distinguishes itself through several key features. First, as its name implies, the framework offers significant speed advantages and is considered one of the fastest Python frameworks available, delivering performance on par with NodeJS and Go. This speed applies not only to execution throughput but also to the rapid pace of development. Second, FastAPI is designed to enhance reliability by reducing developer-induced errors by approximately 40 percent, primarily through its strict typing system. Third, the framework prioritizes robustness and ease of use, featuring an intuitive interface that is easy to learn while minimizing code duplication by allowing multiple parameters per declaration and automatically generating production-ready interactive documentation. Finally, FastAPI is fully compatible with open standards for APIs, specifically OpenAPI (formerly Swagger) and JSON Schema, ensuring standards-based development.

2.2.3 Spring Boot

Spring Boot is an open-source, enterprise-level Java framework built upon the Spring ecosystem. Its primary objective is to streamline the development process by minimizing configuration overhead. Designed to run on the Java Virtual Machine (JVM), Spring Boot facilitates the creation of production-ready, stand-alone applications. The framework is defined by three core features [10]: autoconfiguration, which automatically configures the application based on the dependencies present; opinionated configuration, which provides sensible defaults to reduce the need for manual setup; and stand-alone execution capability, which enables applications to run independently without requiring an external web server.

2.2.4 Node.js

Node.js is an open-source, cross-platform JavaScript runtime environment designed for building scalable network applications. Originally developed by Ryan Dahl in 2009 [11], it allows developers to use JavaScript for server-side scripting. Node.js is supported by a wide range of frameworks, such as Express.js and NestJS, which streamline backend development [12].



The Node.js architecture is characterized by several key features that contribute to its effectiveness in modern web development. Its non-blocking I/O model enables asynchronous execution of operations, preventing the system from stalling during heavy tasks and significantly reducing response latency while improving throughput. The platform offers a rich ecosystem that provides a comprehensive set of built-in modules, such as HTTP and File System, alongside access to a vast library of third-party packages via the Node Package Manager (NPM). Furthermore, Node.js employs an event-driven architecture that utilizes a single-threaded event loop to handle multiple concurrent clients efficiently, making it highly suitable for real-time applications.

2.2.5 Comparison and Server-side Technology Selection for the Project

Framework	Pros	Cons
Django (Python)	Batteries-Included: Built-in ORM, Auth, and Admin panel for rapid prototyping. Mature & Secure: Robust protection against CSRF and SQL injection.	Monolithic: High overhead for simple microservices; rigid built-in behaviors. Synchronous Roots: Less efficient for high-concurrency real-time features.
FastAPI (Python)	High Performance: Asynchronous architecture comparable to Node.js and Go. Auto-Docs: Automatic Swagger UI generation for streamlined API testing.	Microframework: Requires manual configuration of external libraries for Auth/DB. Newer Ecosystem: Smaller community and fewer niche third-party plugins.
Spring Boot (Java/Kotlin)	Enterprise Standard: Robust ecosystem for Cloud/Security and logic sharing with Android. Scalable: Designed for high-availability and complex distributed systems.	Complexity: Steep learning curve due to DI and AOP concepts. Resource Heavy: High memory consumption and slow startup times.
Node.js (JavaScript)	Unified Language: Allows JS/TS sharing between backend and mobile. Real-Time Ready: Event-driven architecture ideal for chat and I/O tasks.	Single-Threaded: CPU-intensive tasks can block the event loop. Unopinionated: Lack of enforced structure can lead to unmaintainable codebases.

Table 2.2: Comparison of Mobile Backend Frameworks

Selection of Backend Technologies Based on the comparative analysis of benefits and drawbacks presented in **Table 2.2**, our team has decided to adopt a Python-centric backend architecture utilizing **FastAPI**.

The primary driver for this architectural decision is the establishment of a unified language ecosystem. By exclusively utilizing Python and the FastAPI framework for both the core application logic and the high-performance AI services, we ensure technological consistency across the entire server-side infrastructure. This consolidation significantly reduces the cognitive load on developers by eliminating the need for context-switching between different programming languages and web frameworks.

Furthermore, Python was selected for its accessibility and machine learning capabilities. As highlighted in the comparison, Python offers a gentle learning curve, allowing the team to rapidly accelerate development. Crucially, given the project's requirement for a "Speech-to-Schedule"



module, Python is the industry standard for Artificial Intelligence and Natural Language Processing (NLP). Using Python allows us to integrate machine learning models directly into our FastAPI services without requiring complex bridges to external languages, ensuring seamless performance for the voice processing features.

2.3 Database Management System

2.3.1 MongoDB

MongoDB is a source-available, cross-platform, document-oriented database classified under the NoSQL paradigm. It utilizes BSON (Binary JSON), a binary representation of JSON-like documents that supports optional schemas. Development began in 2007 by the American software company 10gen as part of a platform-as-a-service product. In 2009, 10gen transitioned to an open-source development model and began offering commercial support, eventually rebranding itself as MongoDB Inc. [7].

Architecturally, MongoDB is designed to store semi-structured data. The database is organized into collections, which function similarly to tables in relational databases. Each collection contains multiple documents. A key feature of MongoDB is its schema flexibility: unlike relational rows, two distinct documents within the same collection are not required to share an identical structure [15].

2.3.2 PostgreSQL

PostgreSQL is a robust, open-source object-relational database management system (ORDBMS) known for its proven reliability and advanced architecture since 1996 [16].

About the main points of PostgreSQL, the system distinguishes itself through a unique combination of capabilities: it supports hybrid data models by natively storing JSON/JSONB alongside relational data, ensures high concurrency without locking conflicts via Multi-Version Concurrency Control (MVCC), and offers exceptional extensibility by allowing developers to write stored procedures in languages like Python. This versatility, combined with strict ACID compliance and Write-Ahead Logging for data integrity, makes it a highly adaptable solution for complex, data-driven applications [17].

2.3.3 Comparison and Database Management System Selection for the Project

Database	Pros	Cons
PostgreSQL (Relational/SQL)	Data Integrity: Fully ACID compliant with strong constraint enforcement (Foreign Keys). Hybrid Capabilities: Native JSONB support for storing unstructured data within relational tables.	Rigid Schema: Structural changes require migrations, which can be complex during growth. Scaling: Primarily relies on vertical scaling; sharding is more complex than in NoSQL systems.
MongoDB (Document/NoSQL)	Flexible Schema: No predefined structure; allows dynamic data format changes during development. Horizontal Scalability: Native sharding support for distributing data across multiple servers.	Weaker Integrity: Lacks default strict relationship enforcement; relies on application-level logic. Resource Usage: Higher RAM and disk requirements due to BSON data denormalization.

Table 2.3: Comparison of PostgreSQL and MongoDB

Based on the comparison in **Table 2.3**, our team decided to choose **MongoDB** as the primary Database Management System for this project. This decision was driven by three key factors aligned with our system's architecture.

First, MongoDB's **document-oriented model** is highly compatible with the data structures used in our application. Since the "Speech-to-Schedule" module utilizes Python and AI libraries that output complex, nested JSON data, storing this information directly in BSON format eliminates the need for complex object-relational mapping (ORM).

Second, the **flexible schema** facilitates rapid prototyping. As the project is in the development phase, the data requirements for user tasks and voice logs are subject to change. MongoDB allows us to iterate on the data model without the overhead of maintaining rigid schema migrations, ensuring that backend development can keep pace with changes in the mobile application.

Finally, the **horizontal scalability** of MongoDB ensures that the system can handle increasing volumes of unstructured voice data and notification logs efficiently as the user base grows.

2.3.4 Vector Database for RAG

Retrieval-Augmented Generation (RAG) is an architecture that connects a machine learning language model to external knowledge resources. It enables the retrieval of related information to be sent alongside the initial input. This approach addresses common limitations such as knowledge cutoffs, AI hallucinations, and a lack of domain specificity. [37]

A vector database stores and retrieves data as numerical representations called vector embeddings, enabling searches based on semantic similarity rather than exact keyword matches. In most RAG implementations, vector databases or vector indexing are utilized to facilitate this semantic search. [37]



ChromaDB

ChromaDB is an open-source vector database explicitly designed to serve as the data infrastructure for AI applications [38]. Unlike traditional operational databases that rely on exact keyword matches, ChromaDB is optimized for storing, indexing, and querying high-dimensional vector embeddings based on semantic similarity.

ChromaDB was selected for this project due to several distinct advantages. First, the availability of a Python SDK facilitates seamless integration into the existing system architecture. Additionally, it provides high-speed performance, which is essential for efficiently retrieving contextual data to help mitigate AI hallucinations.

2.4 Generative AI

Within the Alice system, we extensively leverage Generative AI, primarily utilizing state-of-the-art models hosted by external cloud providers. The architecture integrates services from three principal platforms: **Google AI Studio**, **GroqCloud**, and **ElevenLabs**.

From **Google AI Studio**, we utilize the Gemini Flash 3.1 Lite model. Recognized for its advanced reasoning capabilities, this model is employed to generate the training datasets required for our Retrieval-Augmented Generation (RAG) implementation. [34]

GroqCloud, distinguished by its high-performance Language Processing Units (LPUs), powers multiple core services within our pipeline. Specifically, we utilize OpenAI's Whisper Large V3 for Speech-to-Text (STT) transcription, GPT Oss 120B for natural language entity extraction, Meta's Llama V4 Scout for visual processing within the image service, and Carnopy Labs V1 English for Text-to-Speech (TTS) synthesis. [33]

Finally, **ElevenLabs** is integrated using their V3 model specifically to provide high-fidelity Vietnamese voice generation. [35]

Table 2.4: Overview of Integrated Generative AI Models

Cloud Provider	Model Name	Primary Purpose
Google AI Studio	Gemini Flash 3.1 Lite	RAG Dataset Generation
GroqCloud	OpenAI Whisper Large V3	Speech-to-Text (STT)
GroqCloud	GPT Oss 120B	Entity Extraction
GroqCloud	Meta Llama V4 Scout	Image Processing / Vision
GroqCloud	Carnopy Labs V1 English	Text-to-Speech (English)
ElevenLabs	Eleven V3	Text-to-Speech (Vietnamese)

Table 2.4 illustrates the purposes of the Generative AI models used in Alice BackEnd.

Abbreviations used in Table 2.5:



Table 2.5: Model Parameters and API Rate Limits

Model Name	Parameters	RPM	RPD	TPM	TPD	ASH	ASD
Gemini Flash 3.1 Lite	-	15	250K	500	-	-	-
OpenAI Whisper Large V3	1.55B	20	2K	-	-	7.2K	28.8K
GPT Oss 120B	120B	30	1K	8K	200K	-	-
Meta Llama V4 Scout	17B	30	1K	30K	500K	-	-
Carnopy Labs V1 English	3B	10	100	1.2K	3.6K	-	-

- **RPM:** Requests per Minute
- **RPD:** Requests per Day
- **TPM:** Tokens per Minute
- **TPD:** Tokens per Day
- **ASH:** Audio Seconds per Hour
- **ASD:** Audio Seconds per Day

Table 2.5 details the architectural parameters and operational API rate limits—including request frequencies, token capacities, and audio processing quotas—for each of the Generative AI models integrated into the system.

Chapter 3

System Design and Analysis

In Chapter 3, the authors describe the system design and analysis process. This chapter covers related works that provide motivation for the study, requirement analysis, use-case design, system architecture, database design, the speech-to-schedule module, and the application wireframes.

3.1 Related Works

3.1.1 Google Calendar

Google Calendar is a widely adopted digital scheduling platform offering event creation, reminders, recurring rules, sharing, and cross-device synchronization. Its natural-language event creation [18] is highly convenient but still struggles with ambiguous or culturally dependent expressions, especially in Vietnamese. This limitation motivates the Smart Schedule AI’s use of LLM-based extraction for deeper semantic understanding.

While Google Calendar provides reliable server-driven notifications, it does not adapt to user habits or productivity patterns. Furthermore, it relies heavily on traditional manual text entry, completely lacking multimodal capabilities such as parsing schedule information directly from images (e.g., event posters, school timetables, or screenshots) or voice. Smart Schedule AI addresses this gap by integrating Whisper [20] for speech-to-text, advanced vision models for image-based data extraction, and transformer-based architectures for comprehensive event generation.

3.1.2 Notion Calendar

Notion Calendar [23] integrates scheduling with task and knowledge management, enabling events to be linked with projects, documents, and customizable metadata. This flexible structure informs the Smart Schedule AI’s potential use of enriched event attributes such as tags or priorities to improve learning and personalization.

Its streamlined interface and strong workflow integration emphasize low-friction interaction. However, this interaction is still entirely confined to a graphical user interface (GUI) and manual typing. Smart Schedule AI echoes the principle of low-friction interaction but elevates it through a multimodal approach—allowing users to bypass the UI completely by speaking commands or



uploading images of schedules. Although Notion Calendar does not employ AI-driven optimization, its robust event data model and synchronization mechanisms serve as useful references for designing the backend architecture of the Smart Schedule AI.

3.1.3 Reclaim AI

Reclaim AI [19] is an AI-powered scheduling assistant designed to optimize personal and team productivity through intelligent calendar automation. Unlike traditional calendar systems that primarily store manually created events, Reclaim AI dynamically schedules tasks, habits, meetings, and breaks based on user priorities, deadlines, and availability. Its ability to automatically reorganize schedules when conflicts arise demonstrates the practical application of adaptive scheduling and optimization techniques in modern productivity systems.

However, Reclaim AI primarily focuses on productivity optimization within existing calendar ecosystems and still depends heavily on structured user inputs through text and graphical interfaces. In contrast, Smart Schedule AI extends this idea by introducing true multimodal interaction. By combining voice commands using Whisper [20], image processing for automated event detection from photos, and LLM-based semantic extraction using Gemini [25], users can interact with the system more naturally. Additionally, while Reclaim AI mainly targets complex professional productivity workflows, Smart Schedule AI aims to provide a lightweight, mobile-oriented assistant that combines scheduling efficiency, multimodal inputs, and basic personal metrics (such as BMI tracking) into a single cohesive experience.

3.1.4 Comparative Analysis: Smart Schedule AI

The Smart Schedule AI represents a modern approach to personal time management by shifting the focus from manual data entry to AI-driven multimodal inputs—specifically speech recognition and image processing. While specialized apps like Google Calendar, Notion, and Reclaim AI excel in scheduling mechanics, they operate as traditional text-first systems. This creates friction when users need to quickly input events on the go or extract information from visual materials like flyers, invitations, and screenshots.

A key differentiator is the system’s multimodal AI interface powered by Whisper [20], vision processing modules, and Gemini [25]. Users can create events through natural speech such as “I have a meeting with my team at 3pm tomorrow” or “Lên lịch đi chơi đá banh vào tối ngày mai.” Additionally, users can simply provide an image containing schedule details, transforming the application from a passive storage tool into an active virtual assistant that proactively parses and manages daily routines with near-zero manual typing effort.

To prevent feature bloat and strictly manage development constraints, Smart Schedule AI adopts a highly focused engineering philosophy. Extended health management features have been streamlined to include only essential BMI calculation (Section 3.1.4). This deliberate reduction in scope allows computational and engineering resources to be fully dedicated to perfecting cross-domain AI integration—specifically the accuracy of speech-to-text, image OCR, and adaptive scheduling [24]—which forms the core value proposition of the system.



Table 3.1: Comparative Analysis of Existing Applications and Smart Schedule AI

Features	Google Calendar	Notion Calendar	Reclaim AI	Smart Schedule AI
Calendar Event Management	✓	✓	✓	✓
Task Management Integration	Limited	✓	✓	✓
Voice-based Event Creation	Limited	×	×	✓
Image-based Event Extraction (OCR)	×	×	×	✓
Speech-to-Text Integration	×	×	×	✓
LLM-based Semantic Understanding	×	×	Limited	✓
Adaptive Scheduling Optimization	×	×	✓	✓
Automatic Schedule Reorganization	×	×	✓	✓
Context-aware Scheduling	Limited	Limited	✓	✓
Cross-platform Synchronization	✓	✓	✓	✓
Minimalist Mobile-oriented Design	✓	Limited	Limited	✓
BMI Calculation Support	×	×	×	✓
Unified Multimodal AI Platform	×	Partial	Partial	✓

3.2 Requirement Analysis

3.2.1 Domain Context and Stakeholders

Domain Context

In the modern era, the exponential growth of information and the increasing complexity of daily responsibilities have made effective personal organization a critical survival skill. While the market is flooded with productivity tools, most remain static and fragmented, forcing users to manually input data across separate applications for calendars, to-do lists, and health tracking. This manual overhead creates a productivity paradox in which the tools designed to save time actually consume it through tedious data entry and constant context switching.

Beyond fragmentation, the barrier to efficient planning often lies in the source of information itself. Vital schedules are frequently embedded in non-digital formats, such as printed university timetables or event posters on social media, which traditional applications fail to process automatically. Users are therefore required to transcribe data by hand, further increasing the interaction burden. As workloads intensify and responsibilities expand, effective self-management has emerged as one of the most essential competencies for both personal and professional success.

Advancements in artificial intelligence, particularly Large Language Models (LLMs) and multi-modal processing, offer promising opportunities to transform how individuals manage their daily activities. By enabling users to create schedules and manage tasks through natural voice commands or image-based inputs, these technologies can significantly reduce the friction associated with traditional manual input methods. This aligns with contemporary trends toward intelligent digital assistants, automation, and voice-driven user experiences.

Positioned within this evolving landscape, our project — **Alice** — is motivated by the vision of a Minimalist Intelligence: a unified personal assistant that consolidates schedule management, task tracking, and basic health awareness into a single, cross-platform application. By combining minimalistic design principles with AI-powered multi-modal interaction, the system seeks to simplify self-management processes, reduce cognitive overhead, and support users in maintaining a balanced and productive lifestyle.

Stakeholders

For this project, stakeholders are categorized into two groups: internal stakeholders and external stakeholders.

- **Internal Stakeholders:**

- **Business Owners / Development Team:** This group includes all project members responsible for planning, designing, and implementing the application. They define requirements, make technical decisions, and ensure the system meets its objectives.
- **Project Instructor / Supervisor:** The instructor acts as both advisor and evaluator. They provide guidance during development, ensure academic standards are met, and assess the final project outcome.

- **External Stakeholders:**

- **Customers / End Users:** These stakeholders include students, productivity enthusiasts, and professionals who will use the application to manage schedules, tasks, and basic personal health metrics. Their experience and feedback determine system effectiveness.
- **Technology Providers:** These include third-party tools, platforms, and frameworks such as Flutter, FastAPI, MongoDB, OpenAI Whisper, and LLM APIs. Their stability and performance directly impact product quality.

3.2.2 Functional Requirements

Authentication and User Information – Functional Requirements

- **FR-Auth-1: User Registration**

The system shall allow a new user to create an account using an email address and password. The system shall validate that the email address is in a correct format and is not already registered.

- **FR-Auth-2: User Login (Email and Password)**

The system shall allow a registered user to log in using their email address and password. The system shall display an appropriate error message if the credentials are invalid.

- **FR-Auth-3: Password Reset (Forgot Password)**

The system shall provide a “Forgot Password” option on the login screen.
The system shall send a password reset email to the user’s registered email address upon request.
The system shall allow the user to set a new password via a secure link contained in the password reset email.

- **FR-Auth-4: Update Avatar**

The system shall allow an authenticated user to upload or change their avatar image. The system shall display the updated avatar in the user profile and other relevant screens.

- **FR-Auth-5: Initial Personal Information Input**

The system shall require a user, upon first login, to provide personal information including full name, date of birth, weight, height, and sex.
The system shall store this personal information in the user’s profile and use it as the basis for health metric calculations.

- **FR-Auth-6: Update Personal Information**

The system shall allow an authenticated user to view and update their personal information from a profile screen.

- **FR-Auth-7: Change Password**

The system shall allow an authenticated user to change their password from within the application.
The system shall require the user to enter their current password before setting a new password.

Schedule Management – Functional Requirements

- **FR-Schedule-1: Add Schedule**

The system shall allow a user to create a new schedule by providing a title, selecting a

date, choosing a repeat option, and assigning a category.
The system shall validate all required fields before saving a new schedule.

- **FR-Schedule-2: Manual Schedule Input**
The system shall provide a manual input form that allows users to create schedules without using AI-based features.
- **FR-Schedule-3: AI Voice Input for Schedule Creation**
The system shall allow users to create schedules using voice input.
The system shall transcribe voice input using OpenAI Whisper and process the resulting text through an LLM API to extract relevant schedule information.
The system shall automatically generate a structured schedule entry based on the interpreted voice command.
- **FR-Schedule-4: User-Created Categories During Schedule Creation**
The system shall allow users to create custom categories while creating a schedule.
- **FR-Schedule-5: Modify Schedule**
The system shall allow a user to edit an existing schedule.
The system shall display a confirmation prompt before saving any modifications.
- **FR-Schedule-6: Delete Schedule**
The system shall allow a user to delete an existing schedule.
The system shall display a confirmation prompt before deleting a schedule.
- **FR-Schedule-7: Category Assignment**
The system shall allow users to assign a category to each schedule.
The system shall allow users to define a color for each category.
- **FR-Schedule-8: User-Created Categories from Category View**
The system shall allow users to create custom categories while viewing the category list.
- **FR-Schedule-9: Category List Viewing**
The system shall allow users to view all available schedule categories.
- **FR-Schedule-10: Category Visibility Control**
The system shall allow users to select which categories are visible or hidden in the schedule view.
- **FR-Schedule-11: Category-Based Reminders**
The system shall allow users to configure reminders based on the category assigned to a schedule.
The system shall trigger reminders according to predefined reminder rules associated with each category.
- **FR-Schedule-12: Schedule Views**
The system shall provide multiple schedule views, including a Today View and a Week View.
- **FR-Schedule-13: Navigation Between Days and Weeks**
The system shall allow users to navigate to the next or previous day.
The system shall allow users to navigate to the next or previous week.



Task Management – Functional Requirements

- **FR-Task-1: Create Goal**
The system shall allow a user to create a goal by specifying a title and a deadline.
The system shall validate all required fields before saving a goal.
- **FR-Task-2: Create Task**
The system shall allow a user to create a task by entering a task title.
The system shall create tasks independently unless they are explicitly assigned to a goal.
- **FR-Task-3: Assign Task to Goal (Drag and Drop)**
The system shall allow a user to assign a task to a goal using a drag-and-drop interaction.
The system shall update the task-to-goal assignment immediately after the action.
- **FR-Task-4: Move Task Between Goals**
The system shall allow a user to move a task between different goals using drag-and-drop.
- **FR-Task-5: Edit Task**
The system shall allow a user to edit an existing task.
The system shall require confirmation before saving task modifications.
- **FR-Task-6: Mark Task as Completed**
The system shall allow a user to mark a task as completed.
The system shall visually indicate the completion status of the task.
- **FR-Task-7: Goal Progress Calculation**
The system shall update the progress of a goal when associated tasks are marked as completed or uncompleted.
The system shall calculate goal progress as a percentage of completed tasks relative to the total number of tasks assigned to that goal.
- **FR-Task-8: Remove Task**
The system shall allow a user to delete a task.
The system shall display a confirmation prompt before task deletion.
- **FR-Task-9: Remove Goal**
The system shall allow a user to delete a goal.
The system shall handle associated tasks according to predefined system rules, such as unassigning or deleting dependent tasks.
- **FR-Task-10: View Goal Progress**
The system shall allow a user to view goal progress in percentage format.
The system shall update goal progress in real time as task statuses change.

Health Metrics – Functional Requirements

- **FR-Health-1: Input Body Measurements**
The system shall allow a user to input and update their body measurements, including weight and height.
The system shall validate that all input values fall within reasonable human ranges before storing them.

- **FR-Health-2: BMI Calculation and Classification**

The system shall automatically calculate the user's Body Mass Index (BMI) based on their recorded weight and height using the standard formula $BMI = \text{weight (kg)} / \text{height (m)}^2$. The system shall classify the calculated BMI into standard categories (e.g., Underweight, Normal, Overweight, Obese) and display both the value and its classification to the user. The system shall recalculate and update the displayed BMI whenever the user modifies their body measurements.

3.2.3 Non-Functional Requirements

Performance Requirements

- **NFR-Perf-1: Concurrent User Support**

The system shall support at least 1,000 concurrent users without significant performance degradation.

- **NFR-Perf-2: Response Time**

The system shall maintain an average response time of no more than 2 seconds for common operations, including viewing schedules, updating tasks, and accessing health metric data.

- **NFR-Perf-3: AI Scheduling Processing Time**

The AI-powered voice scheduling feature shall process the voice input and generate a structured schedule entry within 5 seconds under normal system load.

- **NFR-Perf-4: Calendar Data Loading**

Calendar data for the previous 12 months shall be loaded and rendered within 3 seconds under normal operating conditions.

- **NFR-Perf-5: Notification Timing Accuracy**

The system shall trigger schedule-based reminder notifications within 1 minute of their configured trigger time.

Reliability and Availability Requirements

- **NFR-Reli-1: System Availability**

The system shall maintain a minimum uptime of 99.5% per month, excluding scheduled maintenance periods.

- **NFR-Reli-2: Data Retention**

The system shall retain user data, including schedules, tasks, and personal information, for a minimum of 365 days after the user's last recorded activity unless deletion is explicitly requested.

Upon a user-initiated deletion request, all associated data shall be permanently removed within 7 days.

- **NFR-Reli-3: Fault Recovery**

The system shall provide automatic recovery mechanisms capable of restoring core functionalities within 5 minutes after an unexpected failure.

The system shall ensure that no committed transactions are lost during the recovery process.



- **NFR-Reli-4: Notification Reliability**

The notification service shall successfully deliver at least 99% of all scheduled reminders during normal operation.

The system shall ensure that reminder notifications are neither duplicated nor omitted, with delivery occurring within ± 1 minute of the scheduled trigger time.

- **NFR-Reli-5: Timezone Support**

The system shall support scheduling across multiple timezones, covering at least UTC−12 to UTC+12.

Events and reminders shall be displayed and triggered at the correct local time with zero offset error after timezone conversion.

Security and Privacy Requirements

- **NFR-Sec-1: Secure Communication**

All communication between client and server shall be encrypted using industry-standard protocols such as HTTPS/TLS.

- **NFR-Sec-2: Data Protection**

Sensitive user data, including schedules, tasks, health metrics, and personal information, shall be securely stored using appropriate encryption mechanisms.

- **NFR-Sec-3: Access Control**

The system shall enforce access control to ensure that each user can access only their own data.

- **NFR-Sec-4: Credential Storage**

Authentication credentials shall be stored using secure hashing algorithms consistent with industry best practices.

- **NFR-Sec-5: Privacy Compliance**

The system shall adhere to data protection principles, including user consent, data minimization, and secure data deletion upon user request.

- **NFR-Sec-6: Notification Privacy**

The system shall suppress schedule-based notifications during user-defined protected periods, including focus sessions and scheduled meetings.

Usability and User Experience Requirements

- **NFR-Usab-1: Interface Consistency**

The system shall provide a consistent and uniform user interface across all functional modules.

At least 90% of UI components, including buttons, icons, and navigation elements, shall follow a unified design style and interaction behavior across modules.

- **NFR-Usab-2: Learnability**

A new user shall be able to perform core operations, including creating schedules, managing tasks, and viewing health metrics, within 10 minutes without external guidance.

This criterion shall be satisfied by at least 80% of first-time users during usability testing.



- **NFR-Usab-3: Theme Consistency**

The system shall apply a unified light mode theme consistently across all screens and functional modules.

The interface shall maintain visual consistency in color, typography, and component styling without requiring any user-side theme configuration.

- **NFR-Usab-4: Accessibility**

The user interface shall comply with basic accessibility standards, including readable font sizes, sufficient color contrast, and screen reader compatibility.

At least 90% of UI screens shall meet WCAG 2.1 Level AA contrast and readability guidelines.

- **NFR-Usab-5: Schedule Visualization**

The system shall provide multiple schedule views, including daily and weekly views, to support user navigation.

Users shall be able to switch between schedule views within 2 seconds.

- **NFR-Usab-6: Voice Interaction Usability**

The AI-based voice scheduling feature shall be operable without requiring technical knowledge or prior training.

At least 80% of voice input commands issued by first-time users shall be correctly interpreted and converted into structured schedule entries on the first attempt.



3.3 Use-case Design

3.3.1 Use-case

General Use-case Figure 3.1 illustrates the general use case of the entire system. The system involves various actors, including the User, Admin, and several AI models: Google Gemini Flash 3.1 Lite, OpenAI GPT Oss 120B, OpenAI Whisper Large V3, ElevenLabs Eleven V3, Carnopy Labs Orpheus V1, and Llama V4 Scout. Furthermore, the system is composed of five primary modules:

- **Authentication and Personal Information:** Manages user authentication and the administration of personal profiles.
- **Task Management:** Facilitates task-related services, such as the creation and modification of tasks and goals.
- **Schedule Management:** Provides scheduling services, including the creation and editing of user schedules.
- **Speech to Schedule:** Integrates speech processing services to automate scheduling via voice input.
- **Image to Schedule:** Supports image processing services to extract scheduling data from visual inputs.

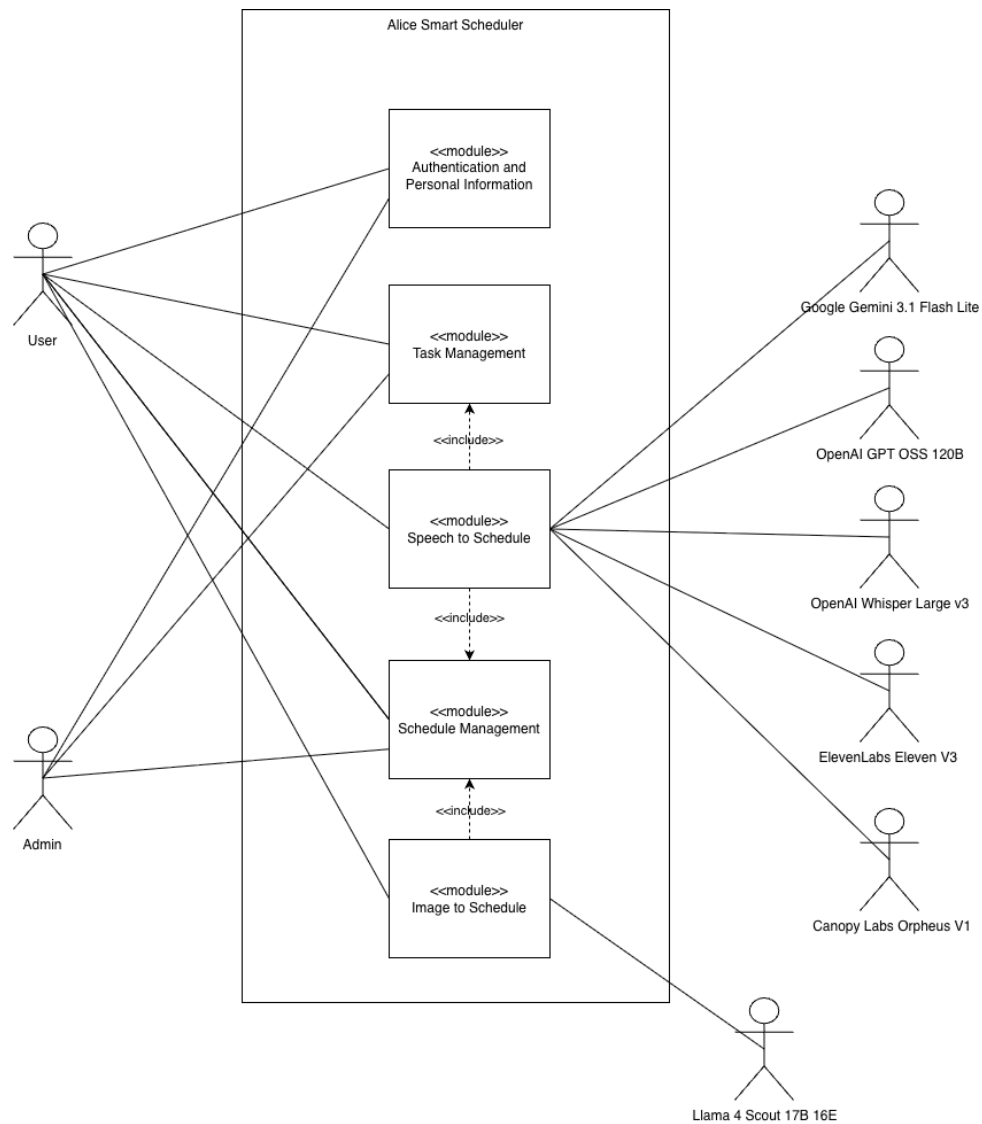


Figure 3.1: General Use-case view

Figure 3.2 illustrates the use case diagram for the Authentication and Personal Information module. Users can log in, register a new account manually or via Google, and update their personal profiles. Administrators are responsible for user management. Furthermore, the NG-
INX server and the Frontend components utilize the verification services provided by this module.

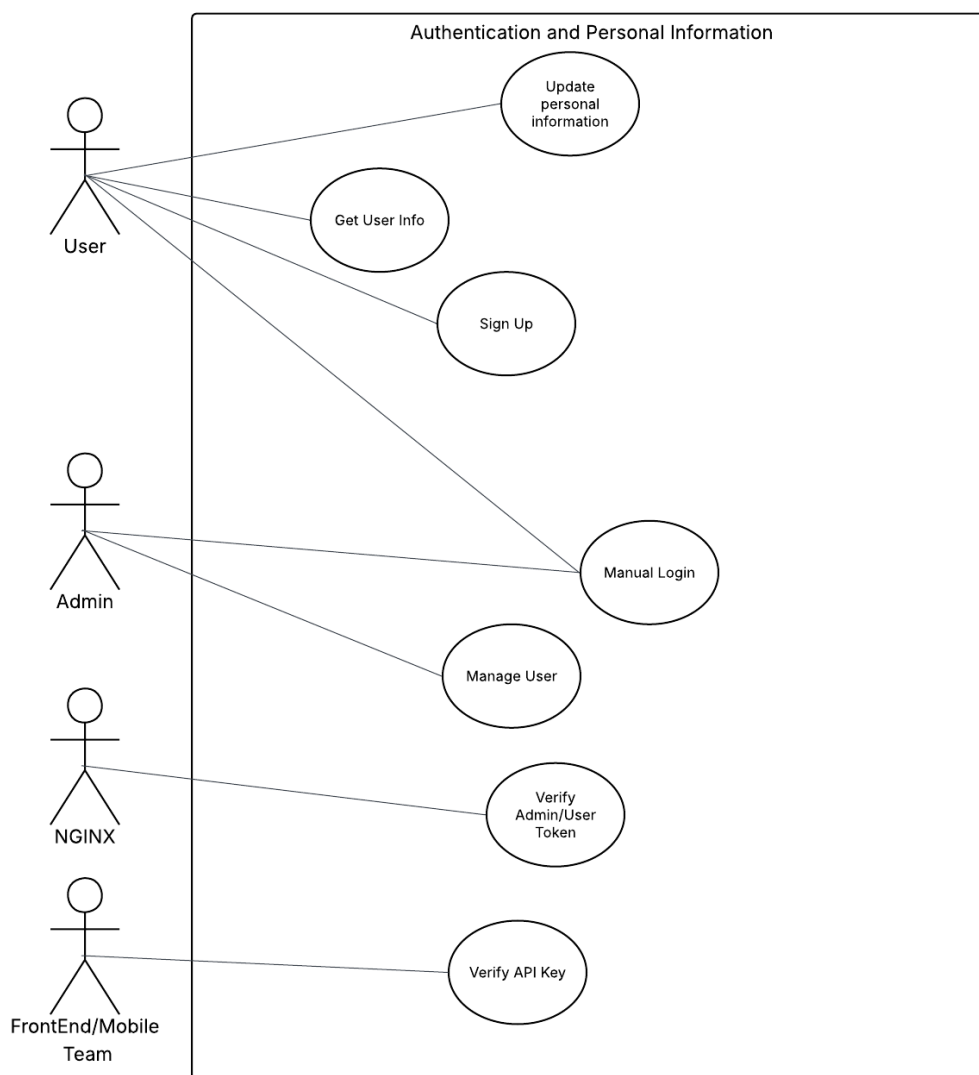


Figure 3.2: Use-case Authentication and Personal Information

Figure 3.3 illustrates the use cases for the Schedule Management module. Users can perform CRUD operations on events, including creating, viewing, updating, and deleting entries. Additionally, they can manage event-specific attributes such as Categories and Repeat Options. This module also serves as a core service for the *Speech to Schedule* and *Image to Schedule* modules. Administrators have the authority to oversee and manage all user schedules and events within the system.

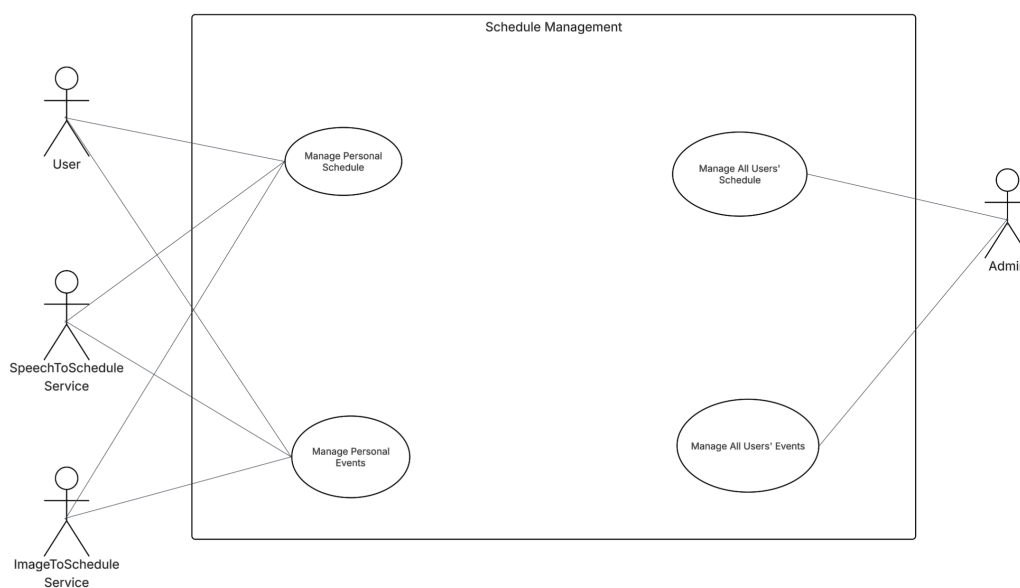


Figure 3.3: Use-case Schedule Management

Figure 3.4 illustrates the use cases for the Task Management module. Users can perform CRUD operations on their personal tasks and goals, while Administrators have the authority to manage tasks and goals across all user accounts. Furthermore, the *Speech to Schedule* module utilizes the services of this module to facilitate task-driven scheduling.

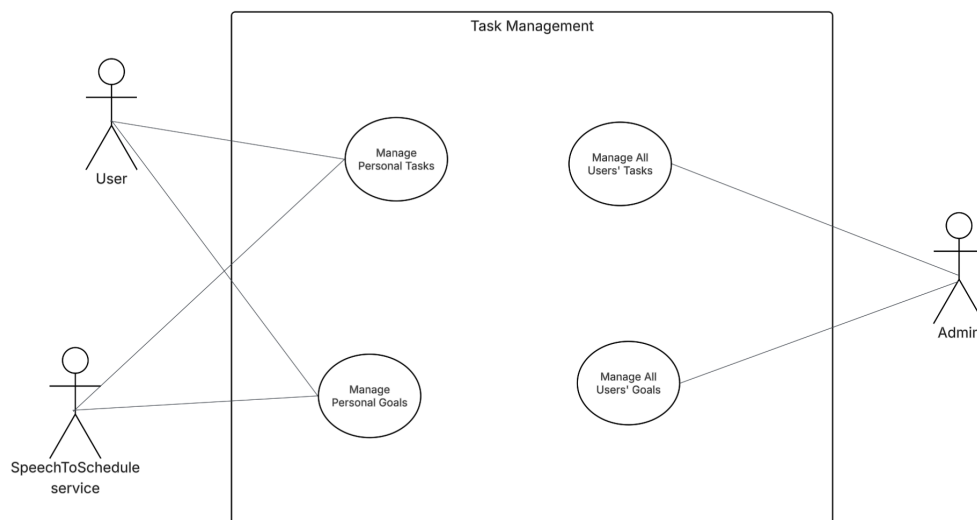


Figure 3.4: Use-case Task Management

Figure 3.5 depicts the use cases for the Speech to Schedule module. The *Automatic Speech Recognition (ASR)* use case serves as the core functionality, encompassing speech-to-text processing, prompt engineering for entity extraction, and subsequent text-to-speech synthesis. This module functions by communicating directly with the *Task Management* and *Schedule Management* modules to execute user requests. Additionally, Administrators can monitor usage metrics and generate RAG (Retrieval-Augmented Generation) datasets to further improve system performance.

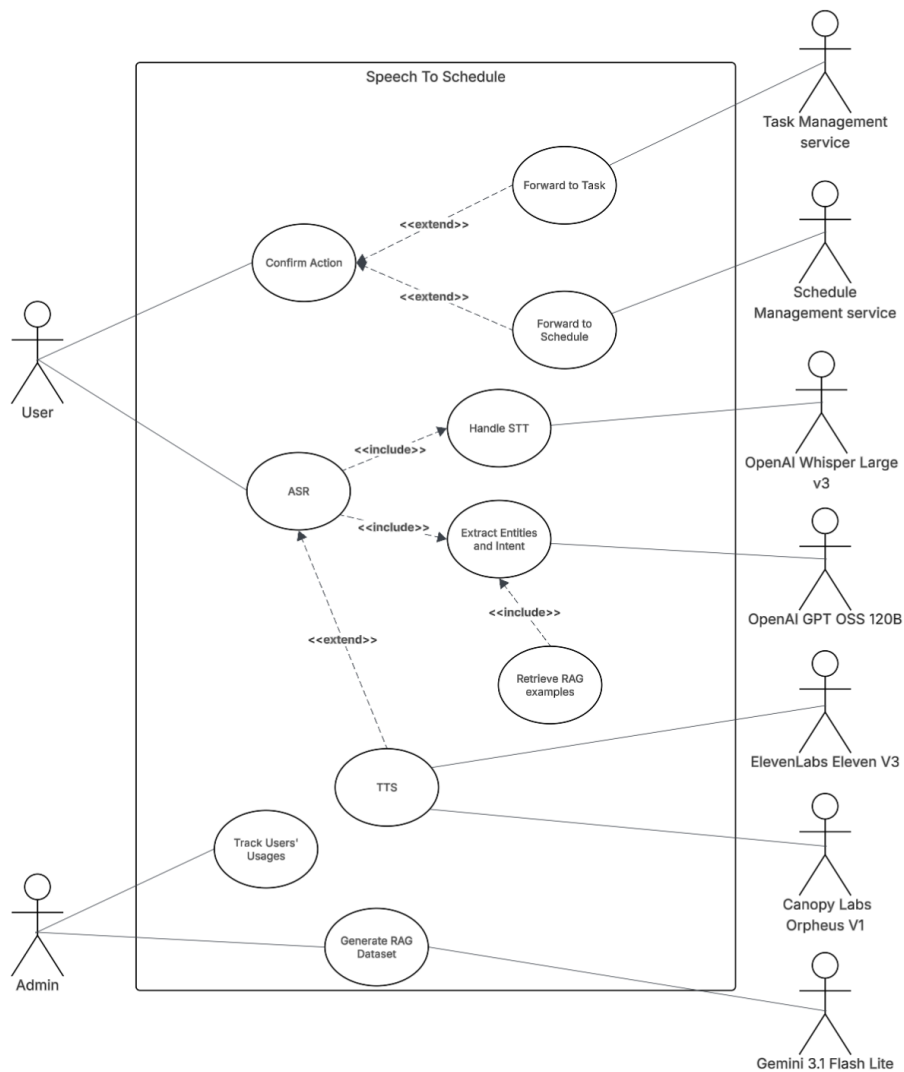


Figure 3.5: Use-case Speech To Schedule

Figure 3.6 illustrates the use cases for the Image to Schedule module. Users can leverage the integrated image processing services to automate the creation of events from visual data, such as photos of physical planners or digital screenshots. Furthermore, Administrators have the ability to monitor usage statistics and track service consumption across the platform.

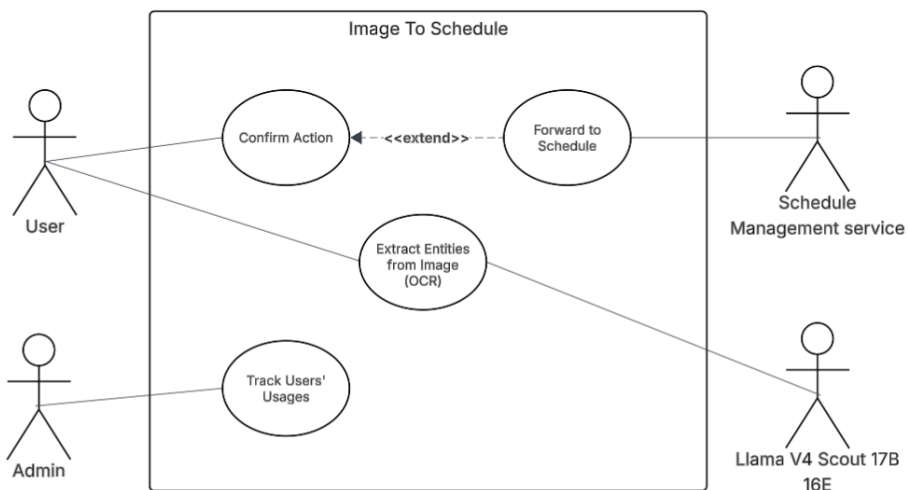


Figure 3.6: Use-case Image To Schedule

3.3.2 Use-case Descriptions

Add Personal Event

Table 3.2 is a description table of the use case Add Personal Event. Through this use case, the user can create a new personal event by entering relevant details such as the event name, time, location, and category. If the process is completed successfully, the new event is stored in the system and immediately displayed in the timetable. Nevertheless, if a network or service error occurs, the system keeps the entered information so the user can try again without losing data.

Use Case ID	UC-01
Use Case Name	Add Personal Event
Primary Actor(s)	User Secondary Actor(s) None
Description	The user manually creates a new event to be tracked in their personal schedule.
Trigger	The user clicks the “Add Event” button (+ icon) in the client application.
Preconditions	PRE-1. The user is securely logged in with a valid authorization token. PRE-2. The user is actively viewing the Calendar/Timetable tab.
Postconditions	POST-1. The new event is successfully saved in the Schedule Management database. POST-2. The client application updates the timetable UI to display the newly created event.
Normal Flow	1. User clicks the “+” button. 2. System (Client App) displays the event entry form overlay. 3. User enters the event details (Name, Start/End Time, Location, Category). 4. User clicks “Save” to submit. 5. System validates the inputted data and sends a RESTful request to the Schedule Management service. 6. Schedule Management service processes the request and persists the event in the database. 7. System closes the form overlay and refreshes the timetable view.
Exceptions	6a. API / Network Error (Schedule Management service fails to save): 6a.1. System displays an error message to the user. 6a.2. System keeps the form open with the user’s data retained to prevent data loss. 6a.3. Resume at Step 4.

Table 3.2: Use Case UC-01: Add Personal Event



Add Personal Goal

Table 3.3 is a description table of the use case Add Personal Goal. Through this use case, the user can create a new personal goal by providing information such as the goal name, description, deadline, logo, and color. If everything proceeds correctly, the goal is saved and shown in the user's goal list. Nevertheless, if the Task Management service cannot process the request, the system preserves the entered information so the user can retry later.

Use Case ID	UC-02
Use Case Name	Add Personal Goal
Primary Actor(s)	User Secondary Actor(s) None
Description	The user creates a new personal goal to be tracked within the system.
Trigger	The user clicks the “+” button in the 'Your Goals' section of the Goal and Task page.
Preconditions	PRE-1. The user is securely logged in with a valid authorization token. PRE-2. The user is actively viewing the Goal and Task page.
Postconditions	POST-1. The new goal is successfully saved in the Task Management database. POST-2. The client application updates the 'Your Goals' section to display the newly created goal.
Normal Flow	1. User clicks the “+” button in the 'Your Goals' section. 2. System (Client App) displays the goal entry form overlay. 3. User enters the goal details (Name, Description, Deadline, Logo, Color). 4. User clicks “Save” to submit. 5. System validates the inputted data and sends a RESTful request to the Task Management service. 6. Task Management service processes the request and persists the goal in the database. 7. System closes the form overlay and refreshes the 'Your Goals' section.
Exceptions	6a. API / Network Error (Task Management service fails to save): 6a.1. System displays an error message to the user. 6a.2. System keeps the form open with the user's data retained to prevent data loss. 6a.3. Resume at Step 4.

Table 3.3: Use Case UC-02: Add Personal Goal



Add Personal Task

Table 3.4 is a description table of the use case Add Personal Task. Through this use case, the user can create a new personal task from the Today's Tasks section by entering the task information and submitting it to the system. If the operation succeeds, the task is stored in the database and displayed immediately in the task list. Nevertheless, if an API or network problem happens, the system keeps the user's input so the task can be submitted again later.

Use Case ID	UC-03
Use Case Name	Add Personal Task
Primary Actor(s)	User Secondary Actor(s) None
Description	The user creates a new personal task to be tracked and completed within the system.
Trigger	The user clicks the "+" button in the 'Today's Tasks' section of the Goal and Task page.
Preconditions	PRE-1. The user is securely logged in with a valid authorization token. PRE-2. The user is actively viewing the Goal and Task page.
Postconditions	POST-1. The new task is successfully saved in the Task Management database. POST-2. The client application updates the 'Today's Tasks' section to display the newly created task.
Normal Flow	1. User clicks the "+" button in the 'Today's Tasks' section. 2. System (Client App) displays the task entry form (overlay or inline input). 3. User enters the task details (e.g., Task Name). 4. User presses "Enter" or clicks the submit button. 5. System validates the inputted data and sends a RESTful request to the Task Management service. 6. Task Management service processes the request and persists the task in the database. 7. System closes the form/input and refreshes the 'Today's Tasks' section.
Exceptions	6a. API / Network Error (Task Management service fails to save): 6a.1. System displays an error message to the user. 6a.2. System keeps the input form open with the user's data retained to prevent data loss. 6a.3. Resume at Step 4.

Table 3.4: Use Case UC-03: Add Personal Task



Edit Personal Event(s)

Table 3.5 is a description table of the use case Edit Personal Event(s). Through this use case, the user can modify an existing event in the personal schedule, including updating event information and deciding whether the change applies to a single occurrence or all occurrences. If the update is processed successfully, the timetable is refreshed with the latest event data. Nevertheless, if the update fails because of a service or network issue, the form remains available so the user can correct or resubmit the request.

Table 3.5: Use Case UC-04: Edit Personal Event(s)

Use Case ID	UC-04
Use Case Name	Edit Personal Event(s)
Primary Actor(s)	User Secondary Actor(s) None
Description	The user manually modifies the details of an existing event in their schedule.
Trigger	The user clicks or taps an existing “Event” object on the timetable.
Preconditions	PRE-1. The user is securely logged in with a valid authorization token. PRE-2. There is at least one existing event in the user’s schedule. PRE-3. The user is actively viewing the Calendar tab.
Postconditions	POST-1. The specified event(s) are updated in the Schedule Management database. POST-2. The client application updates the timetable UI to reflect the changes.
Normal Flow	1. User taps/clicks on an event on the schedule. 2. System (Client App) displays the event information overlay. 3. User clicks the “Edit” button (pencil icon). 4. System replaces the view with an editable form populated with current event data. 5. User modifies the event information (e.g., Time, Name). 6. User clicks “Update”. 7. System prompts a confirmation dialog with three options: “Edit this event”, “Edit all events”, and “Cancel”. 8. User clicks “Edit all events”. 9. System validates the updated data and sends a RESTful request to the Schedule Management service. 10. Schedule Management service processes the update and persists the changes in the database. 11. System closes the form and refreshes the timetable view.



Alternative Flow	Alternative at Step 8: Edit single instance 8a.1 User clicks “Edit this event”. 8a.2 Resume at Step 9 (System sends request to update only the specific instance). Alternative at Step 8: Cancel edit 8b.1 User clicks “Cancel”. 8b.2 System closes the dialog but keeps the edit form open with retained data. 8b.3 Resume at Step 5.
Exceptions	10a. API / Network Error (Schedule Management service fails to update): 10a.1 System displays an error message. 10a.2 System keeps the form open with user data retained. 10a.3 Resume at Step 6.



Assign Personal Task to Goal

Table 3.6 is a description table of the use case Assign Personal Task to Goal. Through this use case, the user can associate an existing standalone task with a specific goal by using a drag-and-drop interaction. If everything works correctly, the task is linked to the selected goal and the interface is updated accordingly. Nevertheless, if the linking request fails, the system restores the task to its original position and allows the user to try again.

Use Case ID	UC-05
Use Case Name	Assign Personal Task to Goal
Primary Actor(s)	User Secondary Actor(s) None
Description	The user associates an existing standalone task with a specific personal goal using a drag-and-drop interface.
Trigger	The user drags a task card from the 'Today's Tasks' section and drops it onto a goal card.
Preconditions	PRE-1. The user is securely logged in with a valid authorization token. PRE-2. The user is actively viewing the Goal and Task page. PRE-3. At least one standalone task and one goal exist in the user's view.
Postconditions	POST-1. The task is successfully linked to the target goal within the Task Management database. POST-2. The client application visually updates to reflect the task's new assigned state.
Normal Flow	1. User clicks and drags a specific task from the 'Today's Tasks' section. 2. User drops the task onto a target goal in the 'Your Goals' section. 3. System (Client App) sends a RESTful request to the Task Management service to update the task's parent goal association. 4. Task Management service processes the assignment and persists the relationship in the database. 5. System refreshes the UI, visually moving or linking the task to the selected goal.
Exceptions	3a. API / Network Error (Task Management service fails to link): 3a.1. System displays an error message (toast/snackbar notification) to the user. 3a.2. System visually reverts the dragged task back to its original position in 'Today's Tasks'. 3a.3. Resume at Step 1.

Table 3.6: Use Case UC-05: Assign Personal Task to Goal



ASR

Table 3.7 is a description table of the use case Process Voice Command (ASR). Through this use case, the user can speak a command, have the audio transcribed and analyzed, and then receive either a structured action preview or a conversational voice response. If the speech is recognized and processed correctly, the extracted intent is returned to the client for confirmation. Nevertheless, if the speech is unclear or the language model cannot process the request, the system shows an error message and the user may try again.

Table 3.7: Use Case UC-06: Process Voice Command (ASR)

Use Case ID	UC-06
Use Case Name	Process Voice Command (ASR)
Primary Actor(s)	User
Secondary Actor(s)	OpenAI Whisper Large v3, OpenAI GPT OSS 120B, ElevenLabs Eleven V3, Canopy Labs Orpheus V1.
Description	The user records a voice command which is processed through a hybrid AI pipeline to extract intents, entities, or generate conversational audio responses.
Trigger	The user taps the voice/microphone button in the client application.
Preconditions	PRE-1. The user is securely logged in with a valid authorization token. PRE-2. Internet connection is active. PRE-3. The user is on the Alice voice assistant interface.
Postconditions	POST-1. The user's intent is successfully extracted and routed for confirmation, OR a synthesized audio response is played.
Normal Flow	1. User taps on the voice button to start recording. 2. User speaks their command or query to the device. 3. User clicks on the stop button. 4. System (Client App) captures the audio record and sends a RESTful request to the Speech To Schedule service. 5. Speech To Schedule service routes the audio to GroqCloud to perform Speech-to-Text (STT) using Whisper Large v3. 6. Service retrieves relevant RAG examples from the vector database to construct the prompt context. 7. Service sends the transcribed text and RAG context to GroqCloud for entity and intent extraction using GPT OSS 120B. 8. Service receives the extracted JSON response and saves the raw transcribed text and extracted entities into the Speech To Schedule database. 9. Speech To Schedule service analyzes the extracted intent. 10. System determines the intent requires a CRUD operation (Schedule or Task). 11. Service returns the structured data to the Client App, which displays a dialog asking the user to confirm the extracted action.



Alternative Flow	Alternative at Step 10: Conversational / Non-Actionable Intent 10a.1 System determines the intent does not communicate with Schedule or Task Management (e.g., health advice, general chat). 10a.2 Speech To Schedule service sends the LLM's text response to ElevenLabs / Canopy Labs to perform Text-to-Speech (TTS). 10a.3 Service receives the generated audio record and returns it to the Client App. 10a.4 Client App automatically plays the synthesized audio to the user. Use-case ends here.
Exceptions	5a. Unrecognizable Speech / Empty Audio: 5a.1 Whisper returns an empty or extremely low-confidence transcription. 5a.2 System displays/plays an error: "Sorry, I didn't catch that." 5a.3 Resume at Step 1. 7a. LLM Parsing / Network Error: 7a.1 GPT OSS 120B times out or fails to return a valid JSON structure. 7a.2 System displays a processing error message to the user. Use-case ends here.



OCR

Table 3.8 is a description table of the use case Extract Entities from Image (OCR). Through this use case, the user can upload or capture an image that contains schedule-related information and let the system extract structured data from it. If the image is analyzed successfully, the extracted schedule entities are returned for confirmation before being applied. Nevertheless, if the image is unreadable or the service fails to respond, the system keeps the current image and allows the user to try again.

Table 3.8: Use Case UC-07: Extract Entities from Image (OCR)

Use Case ID	UC-07
Use Case Name	Extract Entities from Image (OCR)
Primary Actor(s)	User
Secondary Actor(s)	Meta Llama 4 Scout 17B 16E (via GroqCloud)
Description	The user uploads or captures an image containing schedule information, which is processed by a multimodal LLM to extract structured schedule events.
Trigger	The user taps the image upload or camera button in the client application.
Preconditions	PRE-1. The user is securely logged in with a valid authorization token. PRE-2. Internet connection is active. PRE-3. The user is on the Alice image assistant interface.
Postconditions	POST-1. The schedule data is successfully extracted from the image and routed to the user for confirmation.
Normal Flow	1. User taps on the "Upload Photo" button. 2. System (Client App) opens the device's photo library. 3. User chooses an image from the library. 4. System loads and displays the image in a preview box. 5. User clicks the 'Analyze Image' button. 6. System sends a RESTful request with the image payload to the Image To Schedule service. 7. Image To Schedule service routes the image to GroqCloud for OCR and entity extraction using Meta Llama 4 Scout. 8. Service receives the extracted JSON schedule data and returns it to the Client App. 9. System displays a dialog asking the user to confirm the extracted schedule actions.
Alternative Flow	Alternative at Step 1: Capture via Camera 1a.1 User taps the "Take Photo" button instead of "Upload Photo". 1a.2 System opens the device's camera interface. 1a.3 User captures a new photo. 1a.4 Resume at Step 4.



Exceptions	7a. Unreadable Image / No Data Found: 7a.1 Llama 4 Scout successfully processes the image but cannot identify any schedule entities (e.g., image is blurry or irrelevant). 7a.2 Image To Schedule service returns an empty array/error to the Client App. 7a.3 System displays a warning to user. 7a.4 Resume at Step 4. 8a. API / Network Error (Service or GroqCloud fails to respond): 8a.1 System displays a processing error message to the user. 8a.2 System retains the uploaded image in the preview box. 8a.3 Resume at Step 4.
------------	---



Confirm ASR Action

Table 3.9 is a description table of the use case Confirm ASR Action. Through this use case, the user reviews the extracted result from a voice command and confirms whether the action should be executed by the appropriate backend service. If the confirmation is accepted and the target service responds successfully, the requested schedule or task data is saved and reflected in the interface. Nevertheless, if the user cancels the action or a target service error occurs, the system stops the operation and informs the user accordingly.

Table 3.9: Use Case UC-08: Confirm ASR Action

Use Case ID	UC-08
Use Case Name	Confirm ASR Action
Primary Actor(s)	User
Secondary Actor(s)	Schedule Management service, Task Management service
Description	The user reviews and confirms the extracted intent and entities from their voice command, triggering the backend to route the data to the appropriate microservice.
Trigger	The user clicks the 'Confirm' button on the action preview dialog.
Preconditions	PRE-1. The user is securely logged in with a valid authorization token. PRE-2. Internet connection is active. PRE-3. The user is on the Alice voice assistant interface. PRE-4. A voice command was successfully processed and the confirmation dialog is currently displayed.
Postconditions	POST-1. The action is successfully executed and saved in the corresponding target database (Schedule or Task). POST-2. The client application UI is updated to reflect the new data.
Normal Flow	1. User clicks the 'Confirm' button in the dialog. 2. System (Client App) sends a RESTful request containing 'confirm: true' and the 'objectId' to the Speech To Schedule service. 3. Speech To Schedule service receives the request and retrieves the cached extracted object from its database using the 'objectId'. 4. Service determines the object's intent is a Schedule action. 5. Speech To Schedule service forwards the object via a RESTful POST request to the Schedule Management service. 6. Schedule Management service processes the data and persists the event in its database. 7. System closes the dialog and refreshes the user's schedule view.



Alternative Flow	Alternative at Step 4: Task Intent 4a.1 Service determines the object's intent is a Task action. 4a.2 Speech To Schedule service forwards the object via a RESTful POST request to the Task Management service. 4a.3 Task Management service processes the data and persists the task in its database. 4a.4 Resume at Step 7. Alternative at Step 1: User Cancels Action 1b.1 User clicks the 'Cancel' button in the dialog. 1b.2 System sends a request containing 'confirm: false' and the 'objectId'. 1b.3 Speech To Schedule service drops/deletes the cached object. 1b.4 System closes the dialog without making changes. Use-case ends here.
Exceptions	6a. Target Service API Error (Schedule/Task service is down or fails to save): 6a.1 The target service returns an error code to the Speech To Schedule service. 6a.2 The Speech To Schedule service forwards the error state to the Client App. 6a.3 System displays an error message ("Failed to save action") and keeps the dialog open. Use-case ends here.

3.4 Architecture Design

3.4.1 Architecture Strategy and Pattern Selection

Software Architecture delineates the high-level structure of a system, defining its components, their interrelationships, and the governing rules for their interaction. Several notable architectural styles are relevant to this project:

Client-Server Architecture: This fundamental style segregates system functionality into two distinct partitions: resource providers (servers) and service requesters (clients) [8].

This architecture offers several significant advantages. First, it enables centralization of control, security policies, and data storage, thereby ensuring data integrity and simplified updates. Second, it provides enhanced scalability, as server resources can be scaled vertically or horizontally independently of the client hardware. Third, it ensures broad accessibility, allowing clients to access server resources from diverse locations and platforms without platform-specific constraints.

However, the architecture also presents notable disadvantages. The most critical concern is the single point of failure, where server downtime renders the entire application unavailable to clients. Additionally, the architecture exhibits strong network dependency, with performance strictly bound by network latency and bandwidth, meaning poor connectivity directly degrades user experience. Furthermore, managing centralized servers requires dedicated infrastructure and maintenance efforts, resulting in considerable maintenance overhead.

Layered Architecture: Also known as N-tier architecture, this pattern organizes the system into horizontal layers, where each layer has a specific role (e.g., presentation, business logic, data access). It is widely adopted for its clear separation of concerns [8].

The primary advantages of this approach include effective separation of concerns, wherein each layer focuses on a specific aspect, making the system easier to develop and maintain. The architecture also enhances testability, as individual layers can be mocked and tested in isolation. Moreover, it provides abstraction benefits, ensuring that changes in one layer, such as switching databases, rarely impact other layers like the user interface.

Despite these benefits, layered architecture introduces certain drawbacks. Performance overhead becomes apparent as data must travel through multiple layers to reach the database, which can introduce latency and lead to the "architectural sinkhole" anti-pattern. Additionally, tight coupling often occurs in practice, as changes to business rules frequently require cascading changes across both the database and UI layers.

Microservices Architecture: This style structures an application as a collection of loosely coupled, independently deployable services. Each service is organized around a specific business capability (Bounded Context) and communicates via lightweight protocols [9].

The advantages of microservices architecture are substantial. It provides fault isolation, ensuring that a failure in one service, such as Task Management, does not necessarily crash the entire system. The architecture enables technology heterogeneity, allowing different services to be written in different languages or use different databases best suited for their specific task. Moreover, it promotes agility through smaller codebases that allow for faster deployment cycles and easier



understanding for new developers.

Conversely, microservices architecture presents significant challenges. It requires robust DevOps culture to manage the operational complexity of deployment, monitoring, and orchestration of many services. Additionally, implementing transactions that span multiple services, known as distributed transactions, is complex and difficult to manage, creating data consistency challenges.

Hybrid Architecture Strategy: **Figure 3.7** illustrates the architecture of this project. To optimize system performance and scalability, this project adopts a **Hybrid Architecture** that combines multiple architectural styles. At the highest level, the system employs Client-Server architecture to separate the user interface (Mobile App) from the backend logic, ensuring clear boundaries between the client and server. The server-side is decomposed into independent modules using a Microservices architecture, allowing for autonomous deployment and scaling. Internally, each microservice utilizes a Layered Architecture to organize the codebase, ensuring separation of concerns between business logic and data access.

The three core modules—**Authentication and Personal Information**, **Task Management**, **Schedule Management**, **Speech To Schedule**, and **Image To Schedule**—adhere to a uniform internal architecture. To ensure separation of concerns, each module is structured into four distinct layers. The Presentation Layer manages the interface for client interactions, including API endpoints, URL routing, and Views. The Service Layer encapsulates the core business logic and functional requirements provided by the module. The Persistence Layer handles data persistence logic through abstractions such as Models and Repositories. Finally, the Database Connection Layer consists of a dedicated, isolated database for each module to ensure data autonomy.

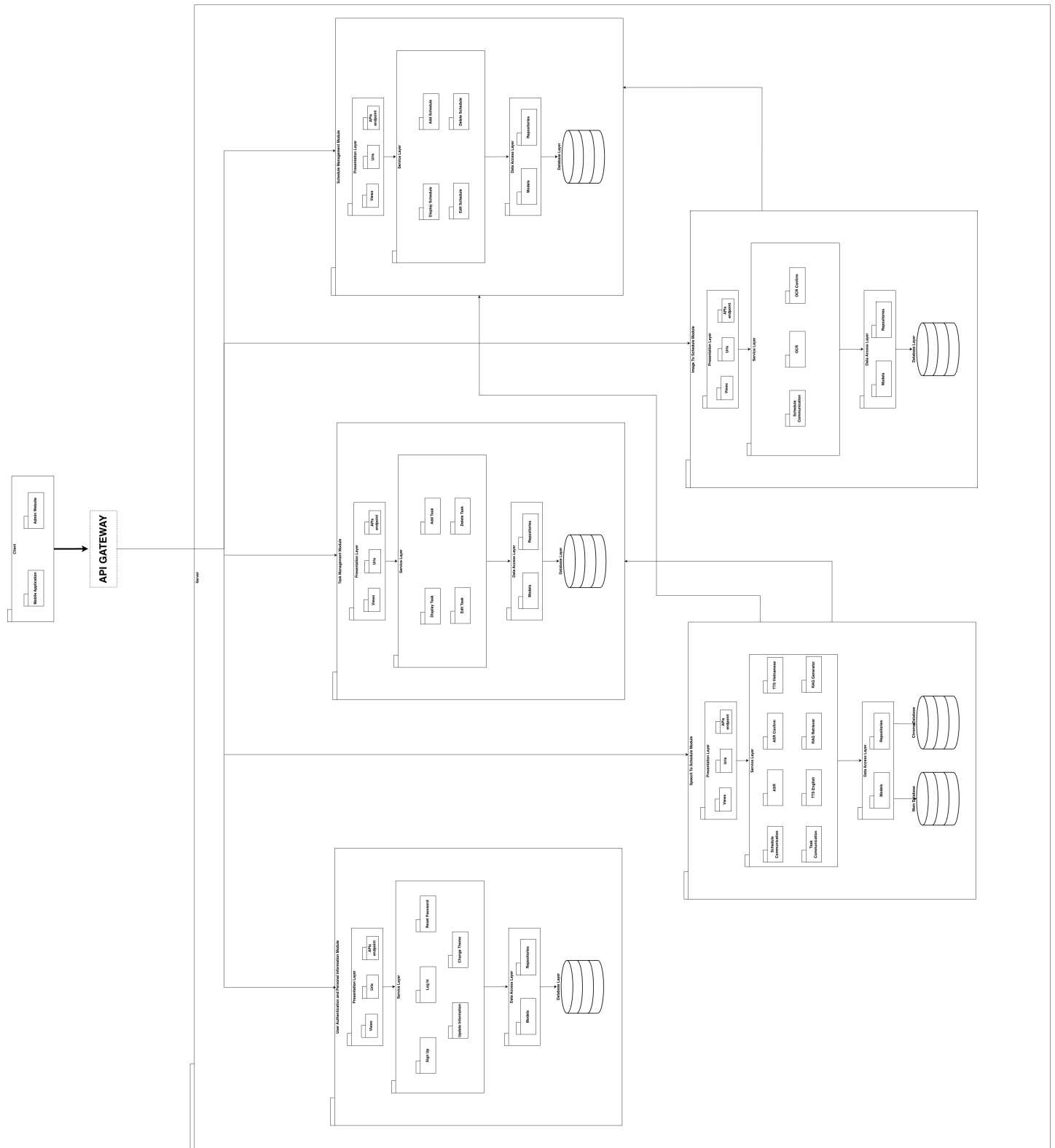


Figure 3.7: Architecture Diagram

The architecture diagram is too large, so we decided to crop it into specific views. First, **Figure 3.8** shows the client part, illustrating the clients and the gateway.

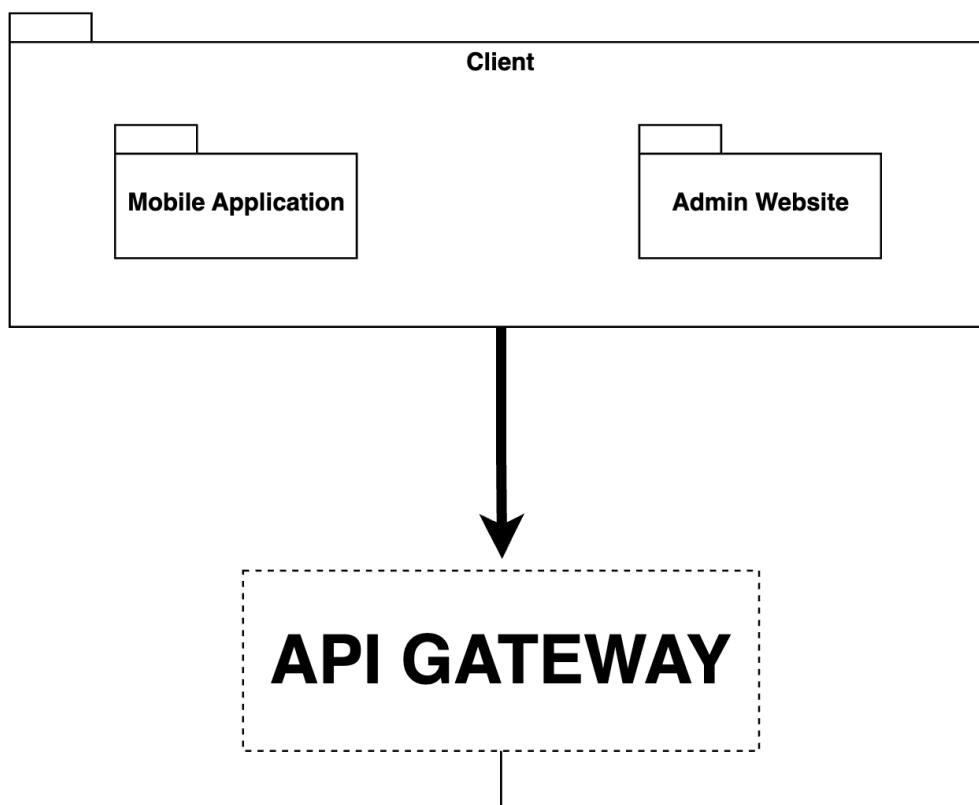


Figure 3.8: Client and Gateway architecture view

Figure 3.9 illustrates the Schedule module.

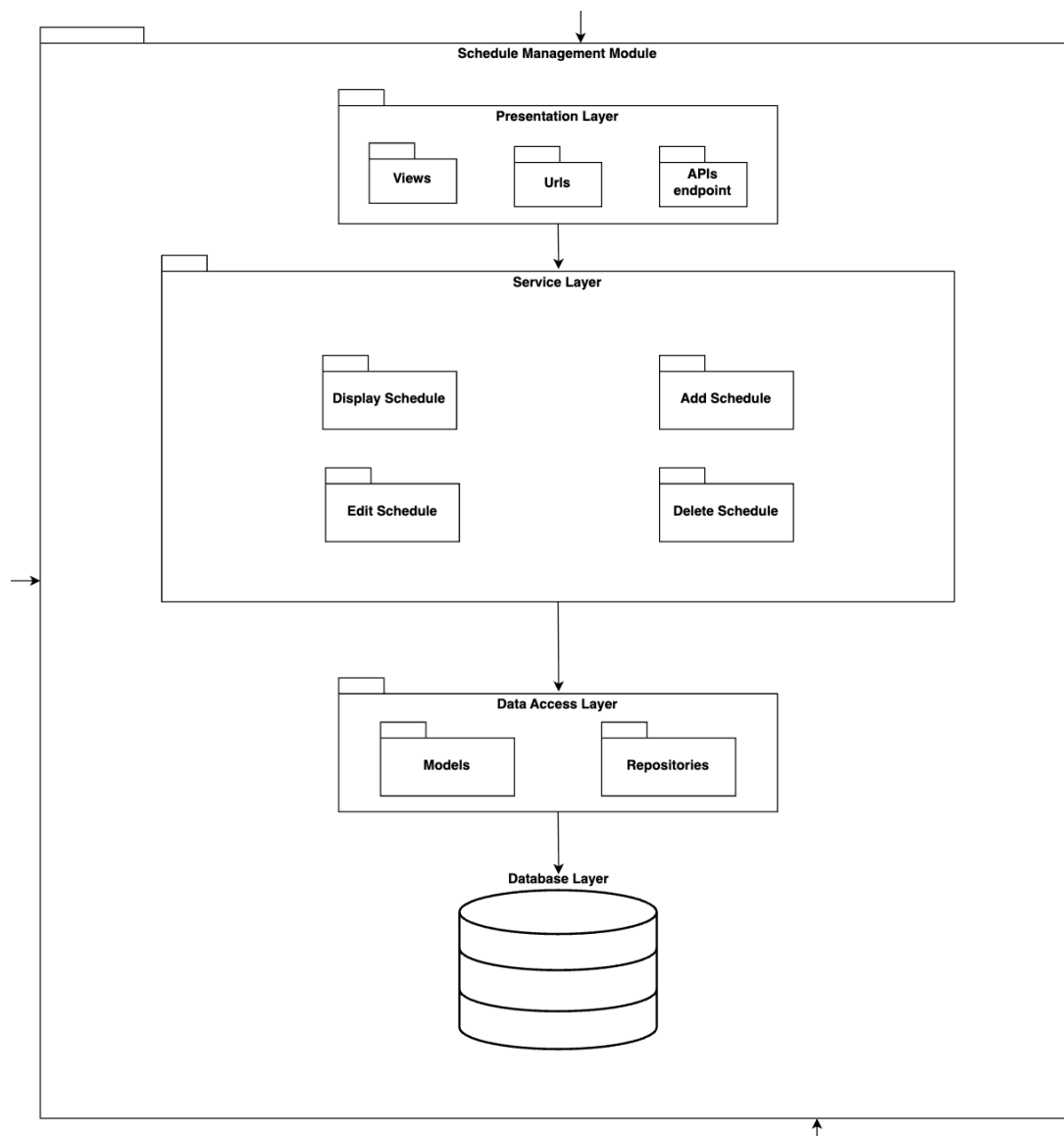


Figure 3.9: Schedule architecture view

Figure 3.10 illustrates the Task module.

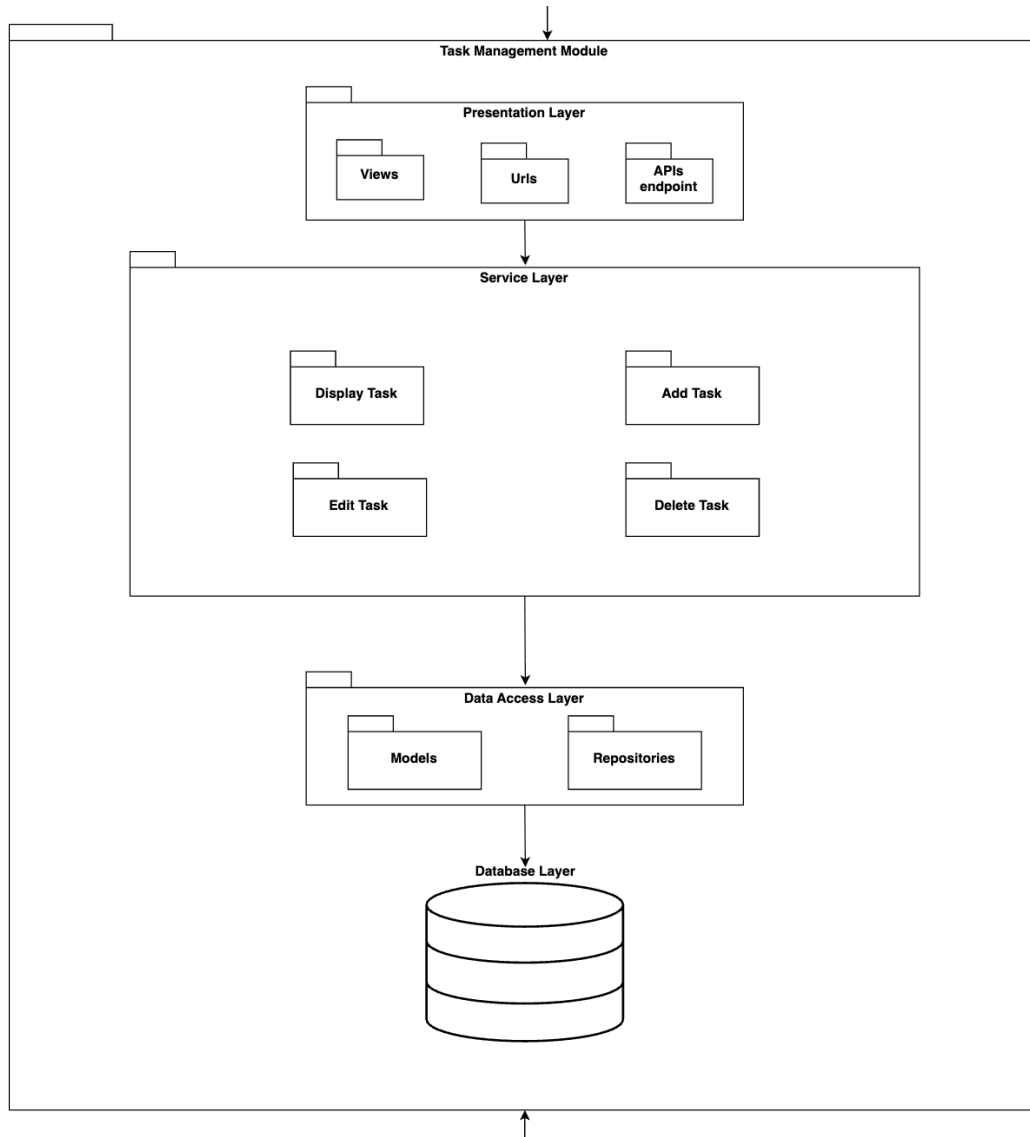


Figure 3.10: Task architecture view

Figure 3.11 illustrates the Authentication and Personal Information module.

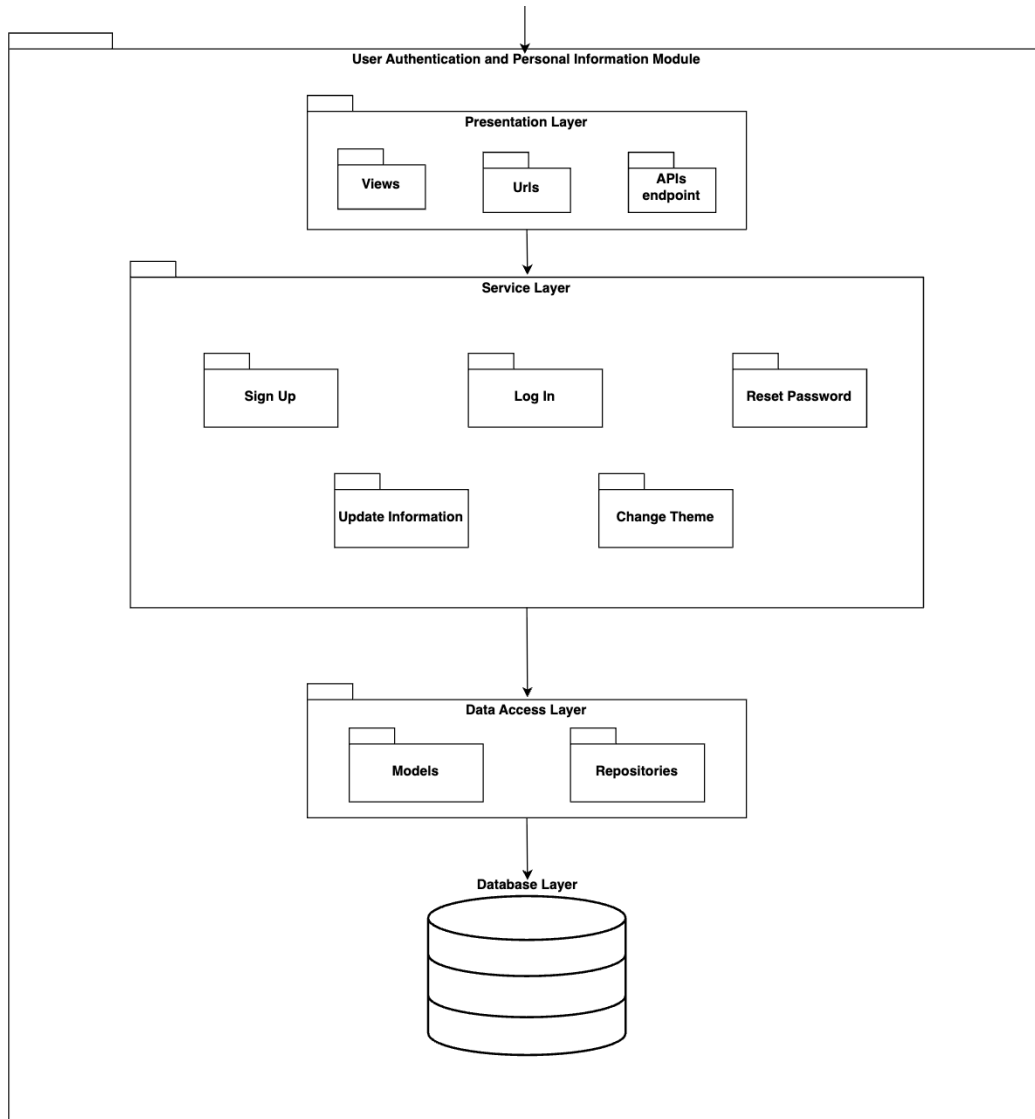


Figure 3.11: Authentication and Personal Information architecture view

Figure 3.12 illustrates the Speech To Schedule module.

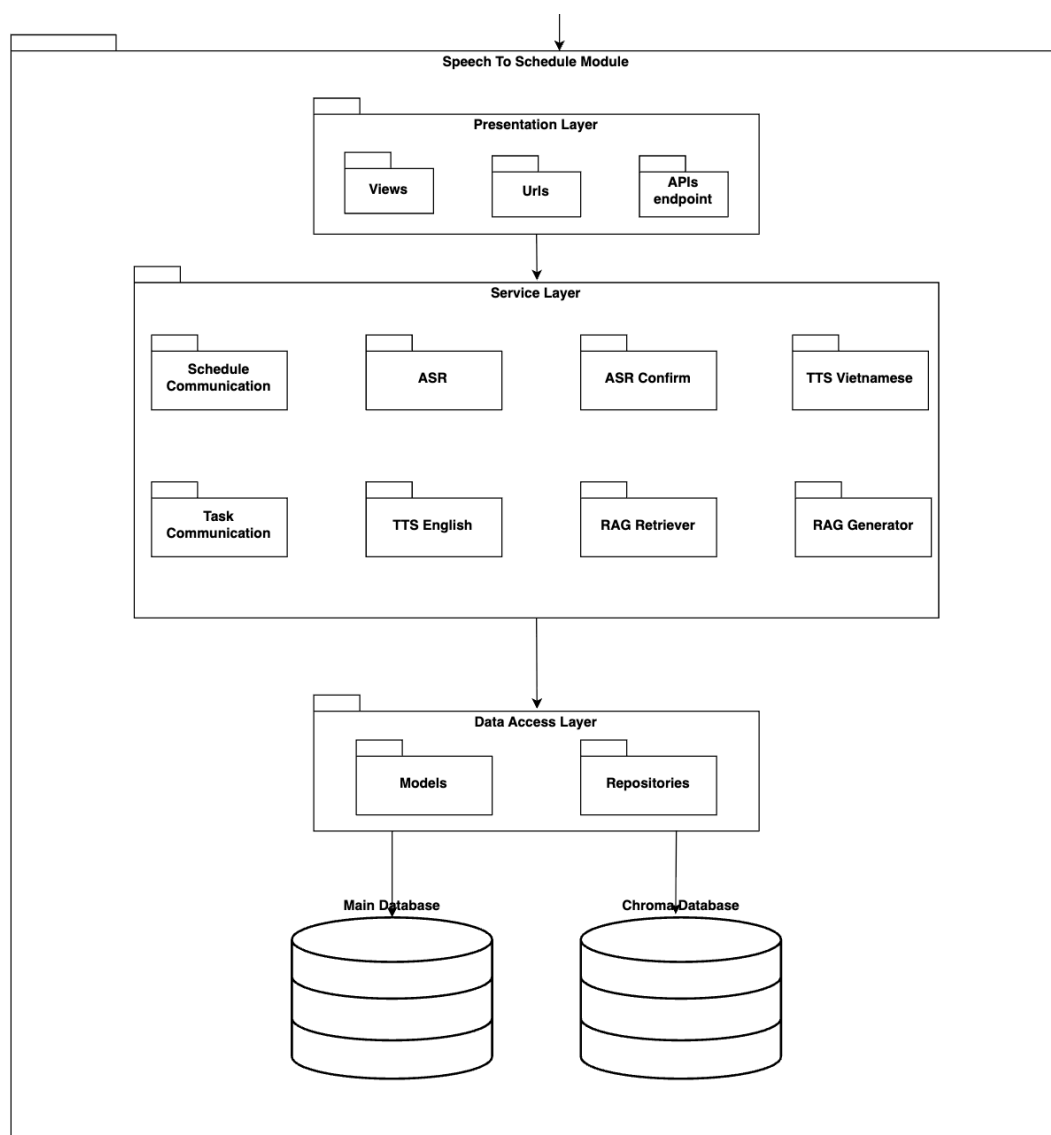


Figure 3.12: Speech To Schedule architecture view

Figure 3.13 illustrates the Image To Schedule module.

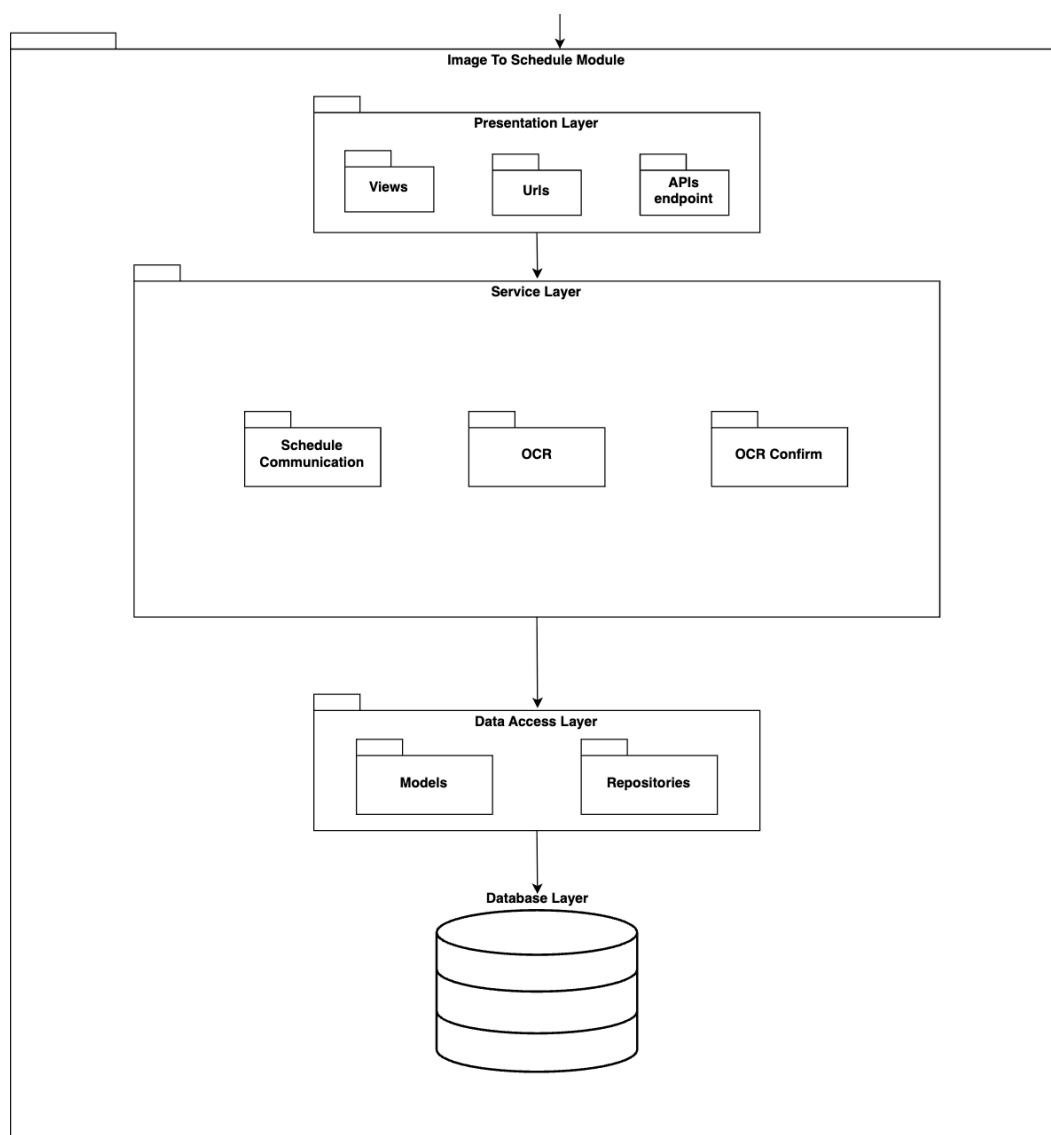


Figure 3.13: Image To Schedule architecture view

3.4.2 Designing Activity Diagrams

Add Event

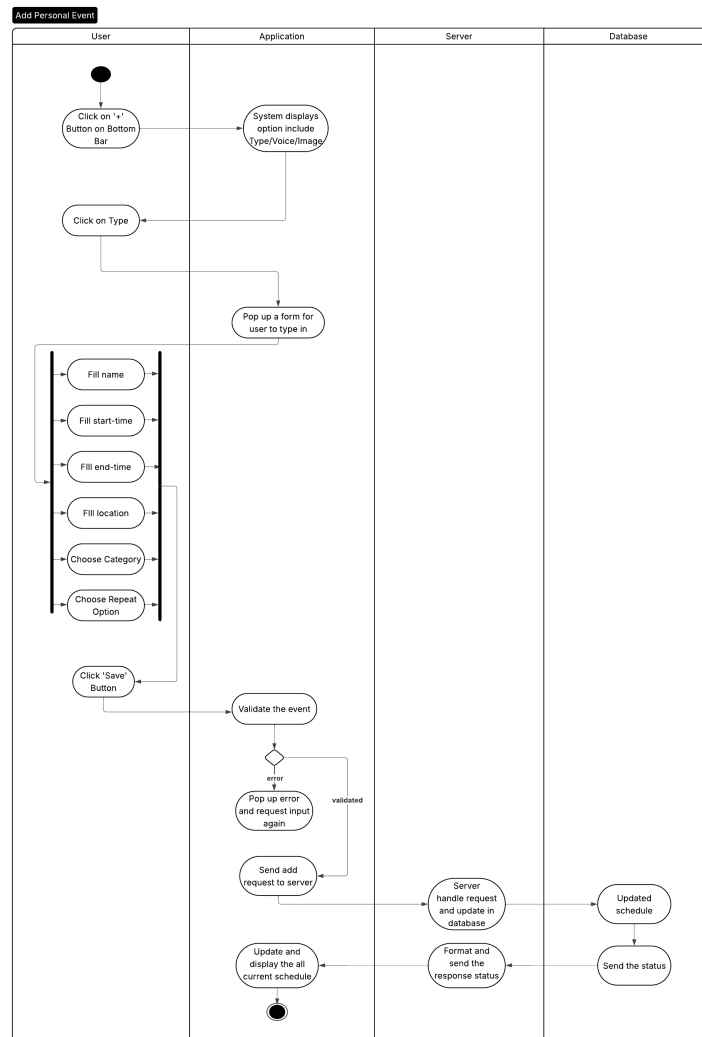


Figure 3.14: Add Event Activity Diagram

Figure 3.14 illustrates the workflow for adding a new event to the calendar. The process initiates when the user interacts with the '+' button on the navigation bar, subsequently entering the required event details and selecting relevant configurations. Upon clicking 'Save', the system validates the input; if an error occurs, an alert is triggered to notify the user. Otherwise, the successfully synchronized event is displayed on the calendar view.

Add Goal

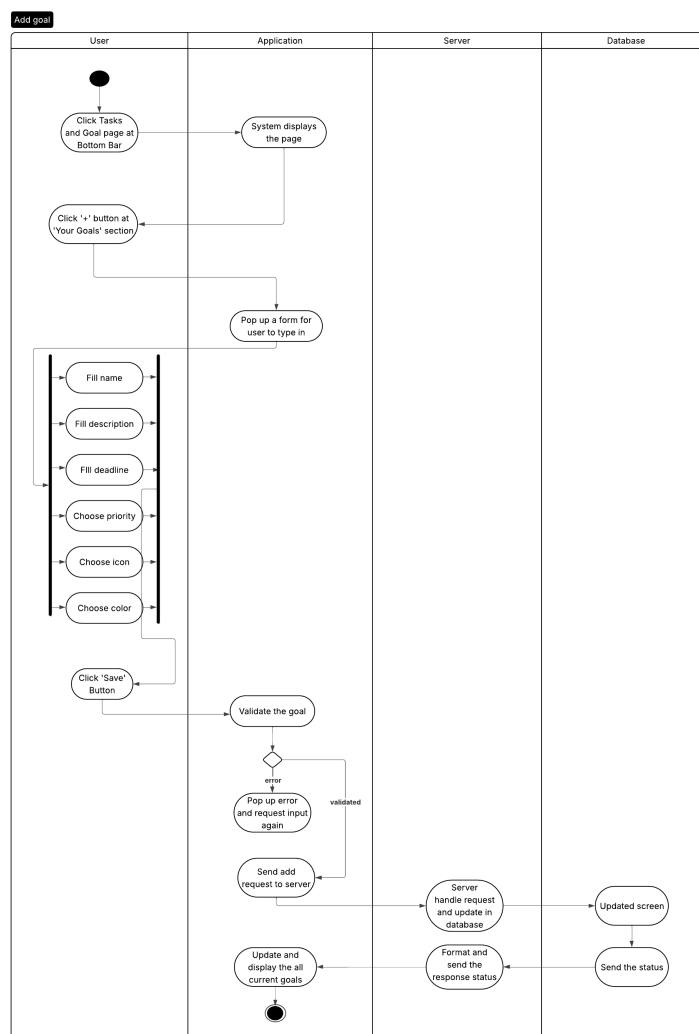


Figure 3.15: Add Goal Activity Diagram

Figure 3.15 illustrates the workflow for adding a new goal to the system. The process begins when the user navigates to the *Task and Goal* page via the bottom navigation bar and selects the '+' button within the 'Your Goals' section. The user then inputs the necessary goal parameters and commits the changes by clicking 'Save'. If the validation is successful, the newly created goal is rendered on the screen; otherwise, an error notification is displayed to inform the user of the failure.

Add Task

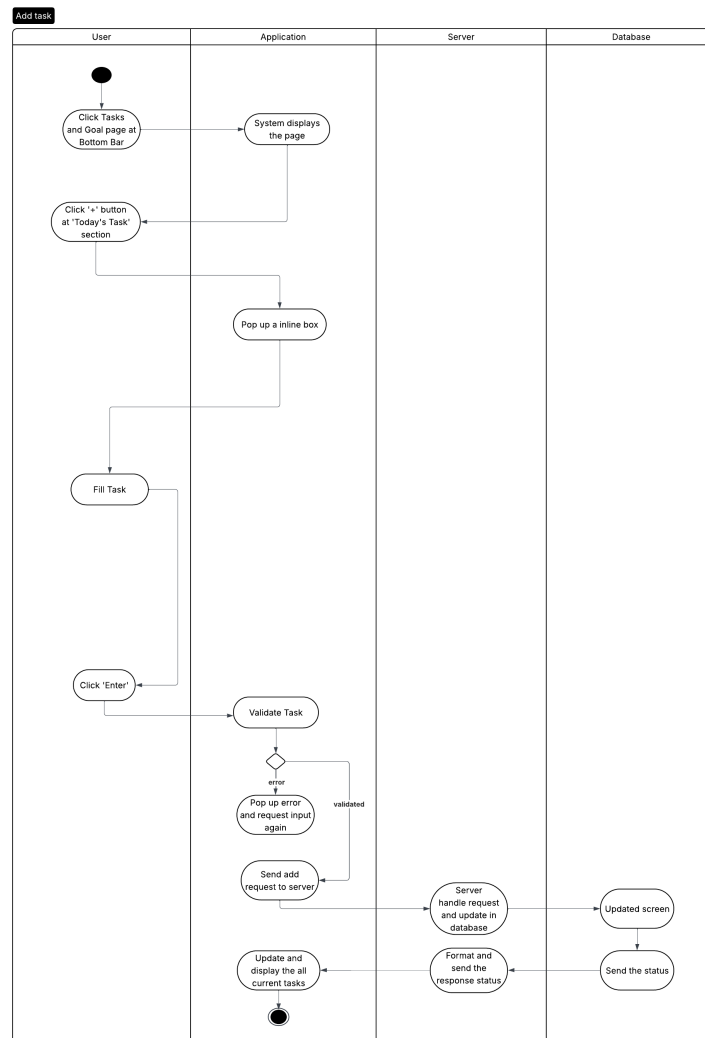


Figure 3.16: Add Task Activity Diagram

Figure 3.16 illustrates the work-flow for adding a new task to the system. The process begins when the user navigates to the *Task and Goal* page via the bottom navigation bar and selects the '+' button within the 'Today's Tasks' section. The user only need to input the content of the task and click 'Save'. If the validation is successful, the newly created task is displayed on the screen; else, an error snackbar is showed to notify the failure.

Edit Event(s)

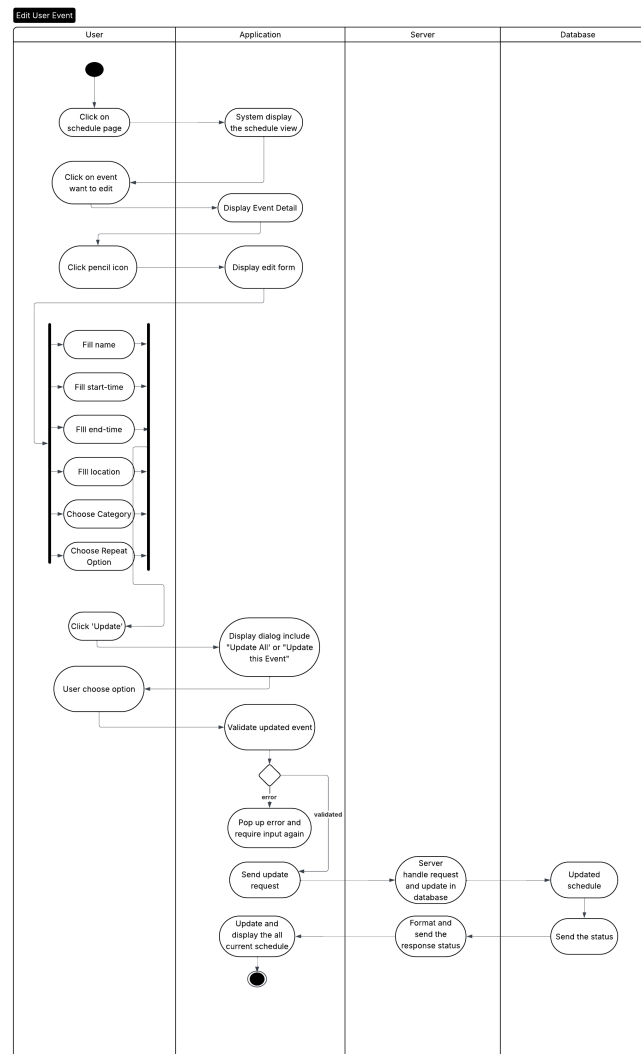


Figure 3.17: Edit Event Activity Diagram

Figure 3.17 illustrates the workflow for updating single or recurring events. The process begins when the user selects a specific event from the schedule and taps the edit (pencil) icon to modify the event details. Upon clicking 'Update', a selection dialog appears, prompting the user to specify whether the changes should apply only to the selected instance or to all events in the series.

Assign Task To Goal

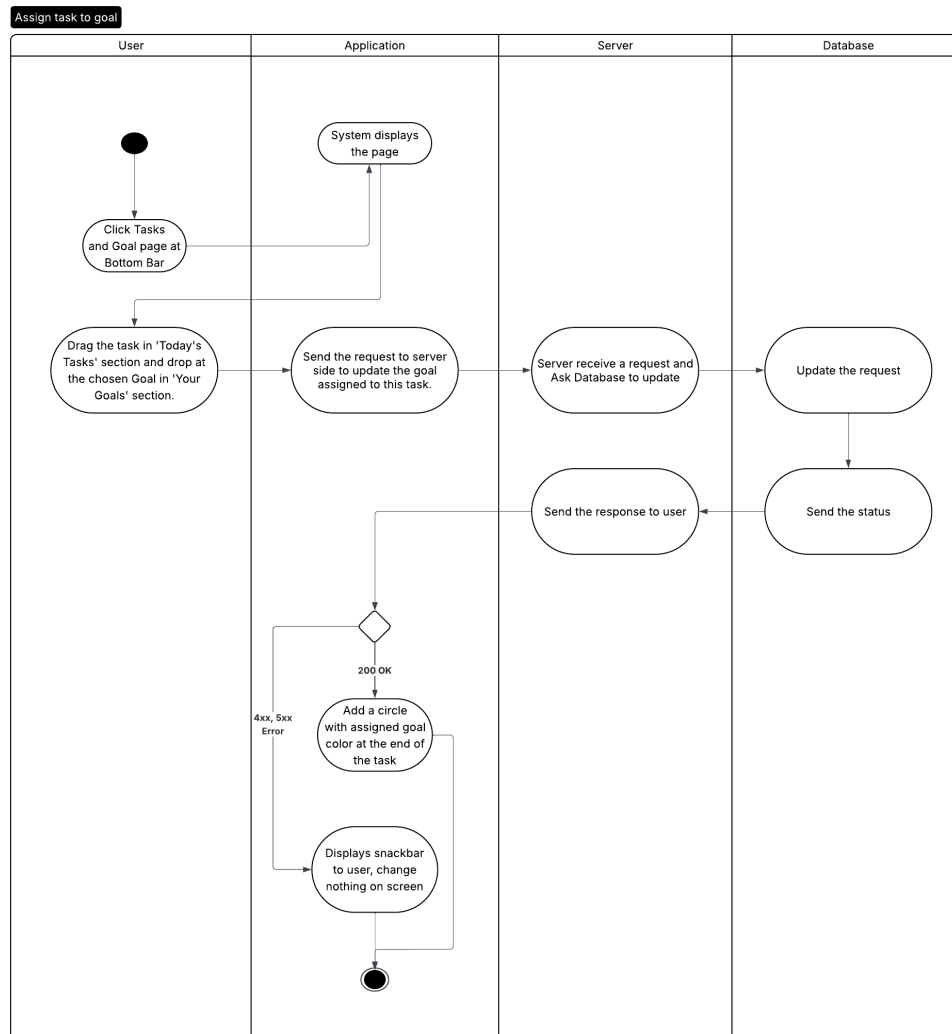


Figure 3.18: Assign Task To Goal Activity Diagram

Figure 3.18 illustrates the workflow for assigning a specific task to a goal. The process begins with the user navigating to the *Task and Goal* page. Within this view, the user assigns a task by performing a drag-and-drop gesture, moving the desired task card onto the target goal. This action triggers the system to establish a logical association between the two entities.

Automatic Speech Recognition (ASR)

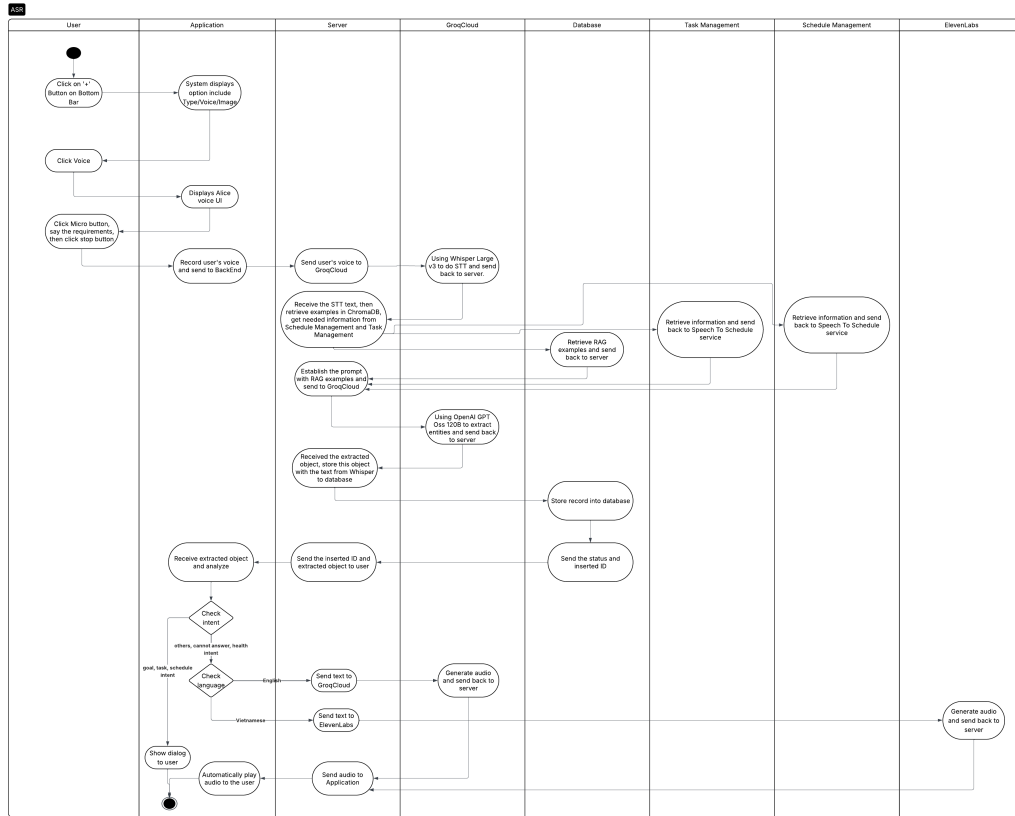


Figure 3.19: ASR Activity Diagram

Figure 3.19 illustrates the workflow for the voice command service. The activity initiates when the user selects the '+' button and chooses the *Voice* option. Upon interacting with the voice interface and speaking a command, the mobile application records the audio and transmits it to the backend. The backend utilizes the *GroqCloud* service for high-performance *Speech-to-Text (STT)* processing. Following transcription, the backend performs a retrieval step to gather relevant context, applies *few-shot prompting* within a *RAG (Retrieval-Augmented Generation)* framework, and prompts the *LLM* via *GroqCloud* to perform entity extraction. Once the structured entity document is retrieved, it is returned to the mobile application for display through a dialog interface. Finally, the application invokes the *Text-to-Speech (TTS)* service to provide auditory feedback if necessary.

Optical Character Recognition (OCR)

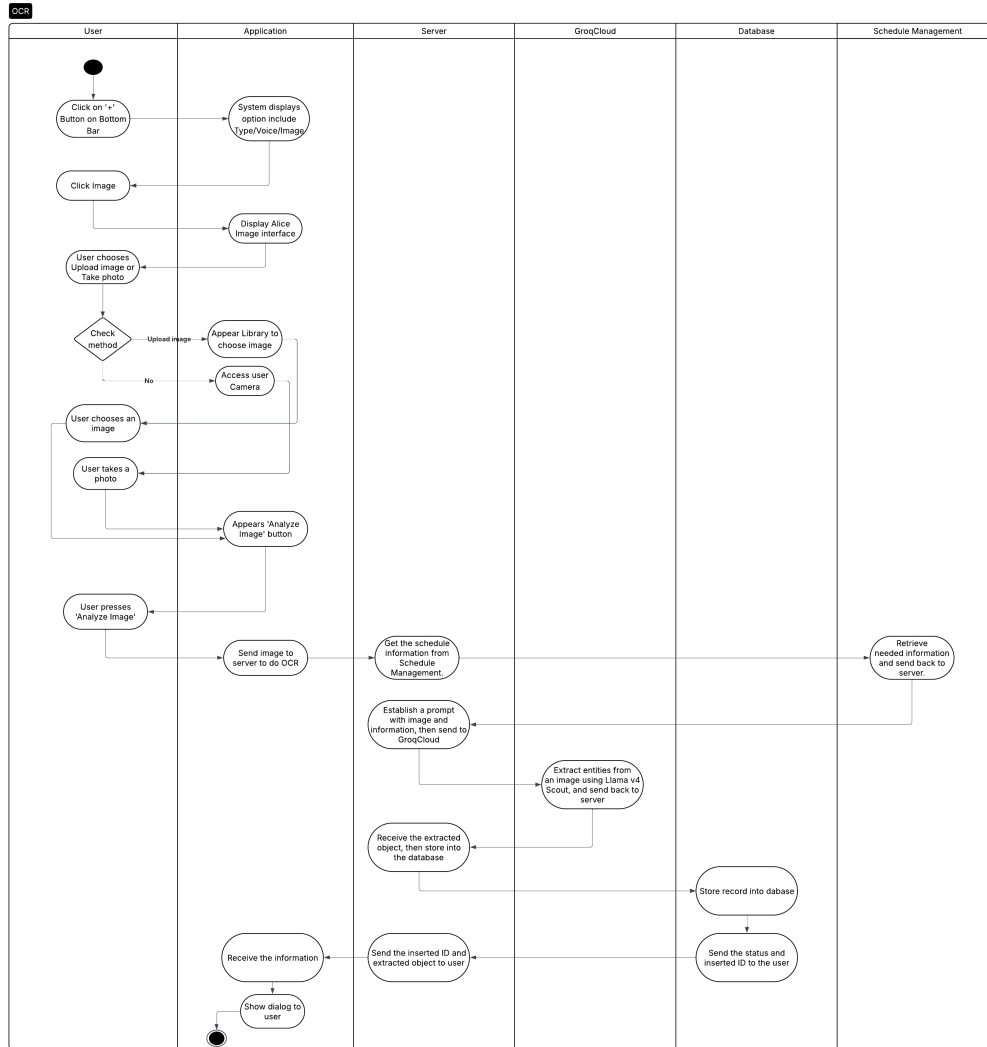


Figure 3.20: OCR Activity Diagram

Figure 3.20 illustrates the workflow for utilizing the image-based event creation service. The process initiates when the user selects the '+' button and chooses the *Image* option, prompting them to either upload an existing file or capture a new photo. Once the user triggers the 'Analyze' action, the image is transmitted to the backend. The backend subsequently orchestrates multiple internal services to retrieve relevant context and construct a comprehensive prompt. This prompt, along with the image data, is dispatched to *GroqCloud* for multimodal analysis and entity extraction. Upon receiving the structured results, the backend returns the extracted data to the mobile application, where it is presented to the user for final review and confirmation.

Confirm ASR Action

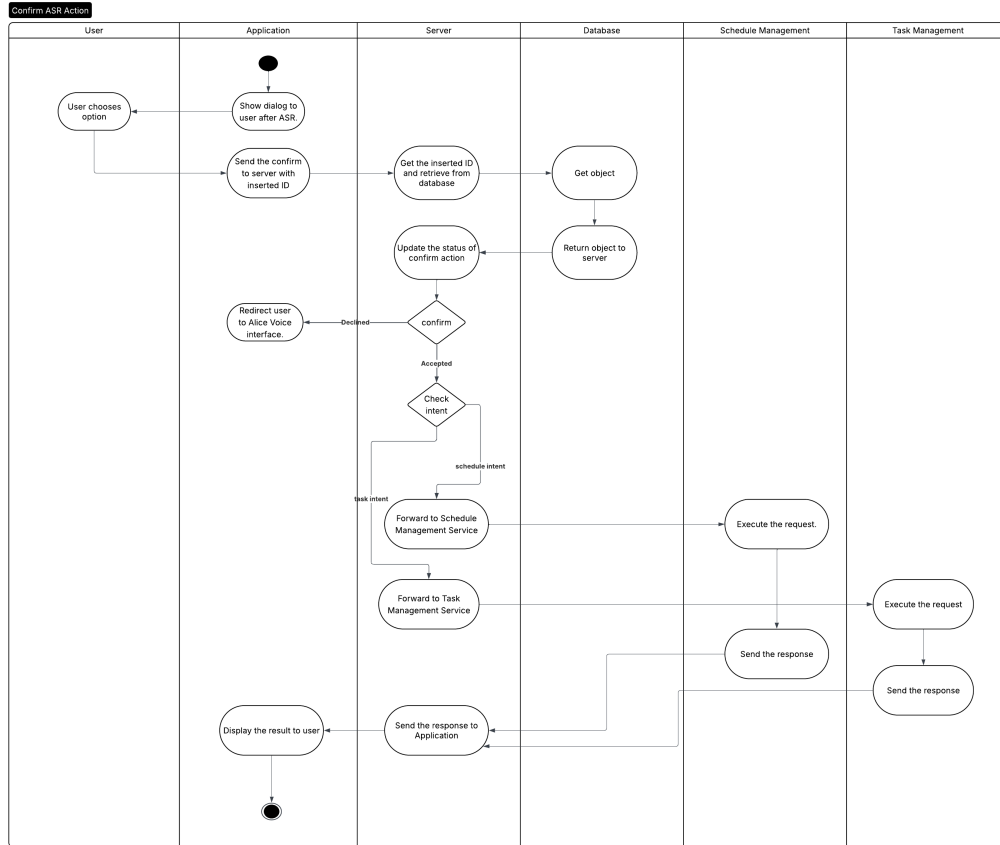


Figure 3.21: Confirm ASR Action Activity Diagram

Figure 3.21 illustrates the confirmation workflow following successful speech-to-text processing. The flow initiates once the mobile application displays the structured data within a dialog. If the user selects the 'Accept' option, the unique identifier (ID) of the logged document is transmitted back to the backend. The backend subsequently updates the document's status within the database and orchestrates the necessary internal services to execute the requested actions, such as scheduling an event or committing a task.



3.4.3 Component Diagram

Figure 3.22 illustrates the component diagram of the system’s architecture. The system adopts a modular design in which the *Mobile Application* serves as the client-side entry point, facilitating user interaction with backend services. An *API Gateway* acts as a centralized intermediary, responsible for routing incoming requests from the application to the appropriate server-side modules.

The backend is organized into distinct, encapsulated subsystems to ensure separation of concerns. The *Authentication and Personal Information* module manages user identity and profile data, utilizing a dedicated repository and database for persistence. The *Task Management* module handles the complete lifecycle of user tasks and goals, maintaining its own isolated database. Similarly, the *Schedule Management* module organizes calendar events through a layered architecture. Finally, the AI-driven components—*Speech to Schedule* and *Image to Schedule*—manage specialized workloads, including speech processing and optical character recognition integration features, respectively.

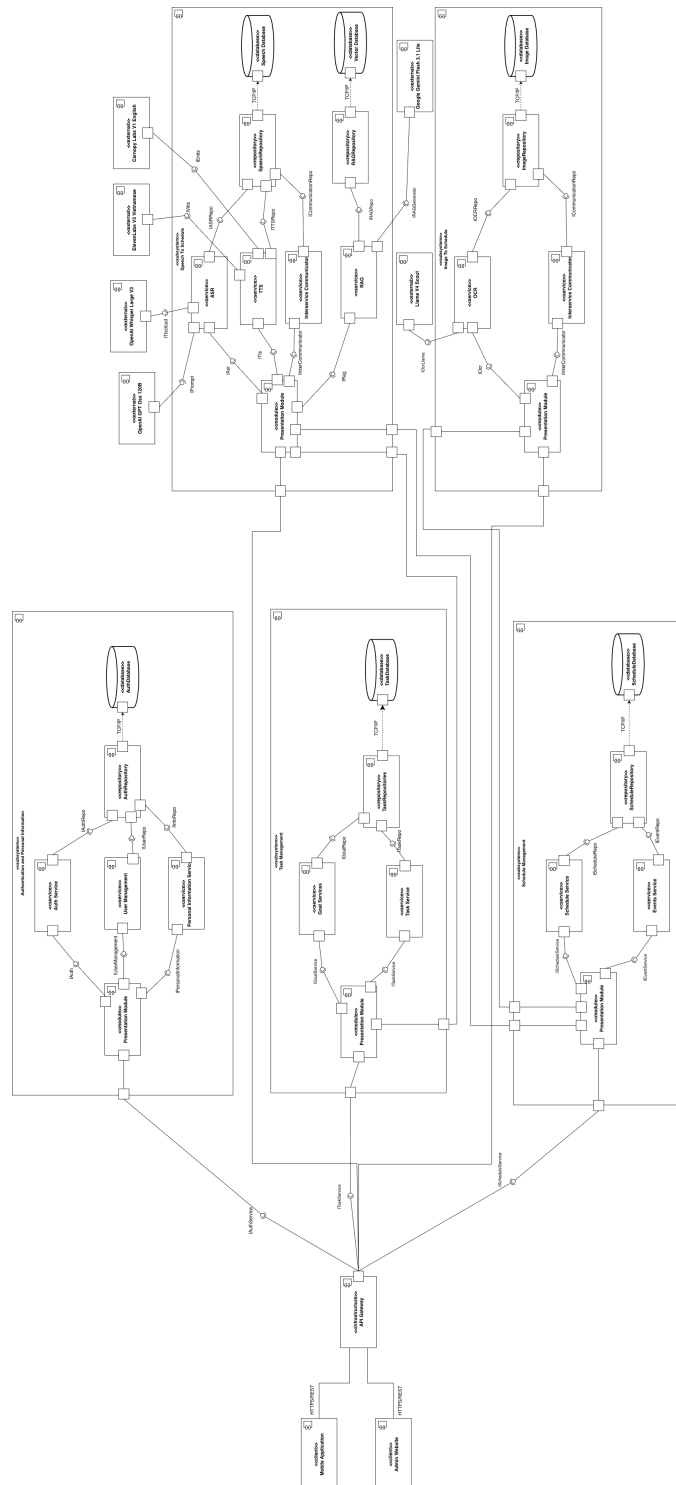


Figure 3.22: Component Diagram

The Component Diagram is too large, so we split it into specific views. First, **Figure 3.23** illustrates the Client and Gateway part.

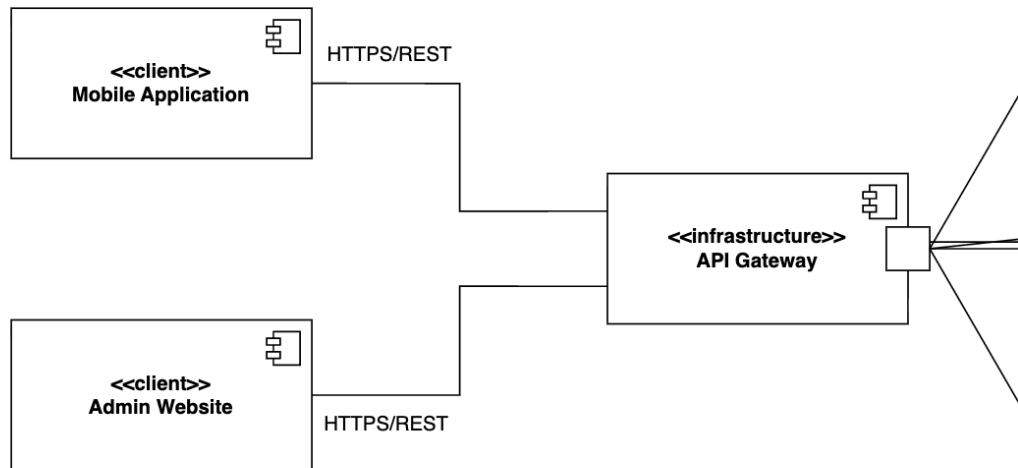


Figure 3.23: Client and Gateway component view

Figure 3.24 illustrates the Schedule module.

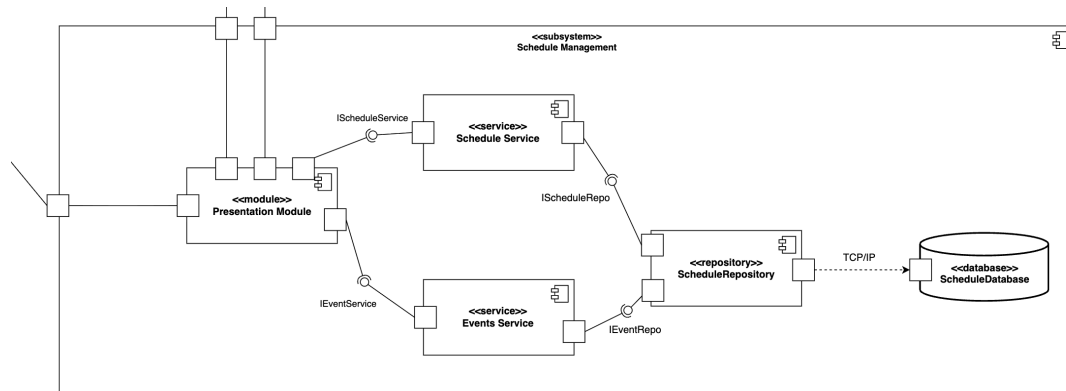


Figure 3.24: Schedule component view

Figure 3.25 illustrates the Task module.

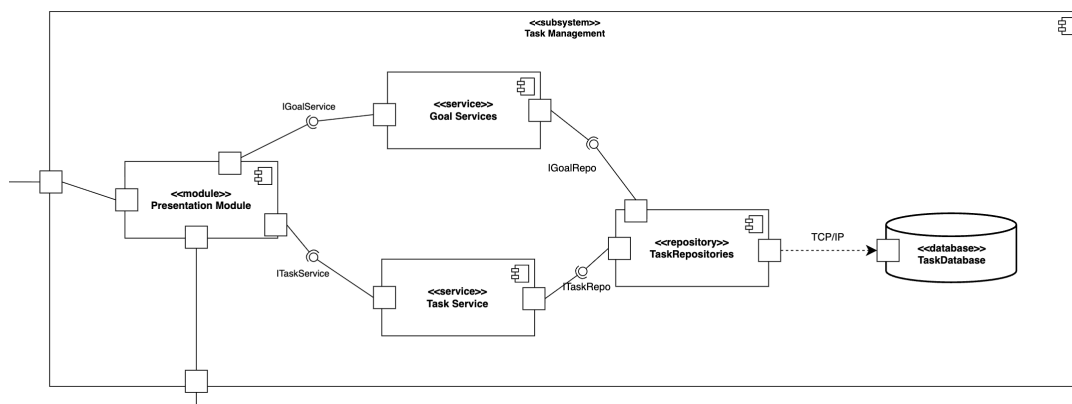


Figure 3.25: Task component view

Figure 3.26 illustrates the Authentication and Personal Information module.

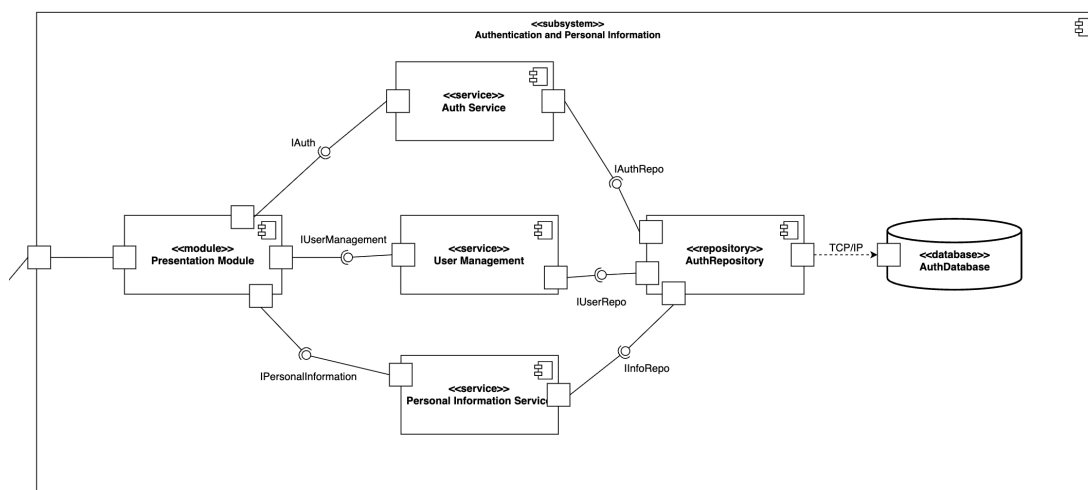


Figure 3.26: Authentication and Personal Information component view

Figure 3.27 illustrates the Speech To Schedule module.

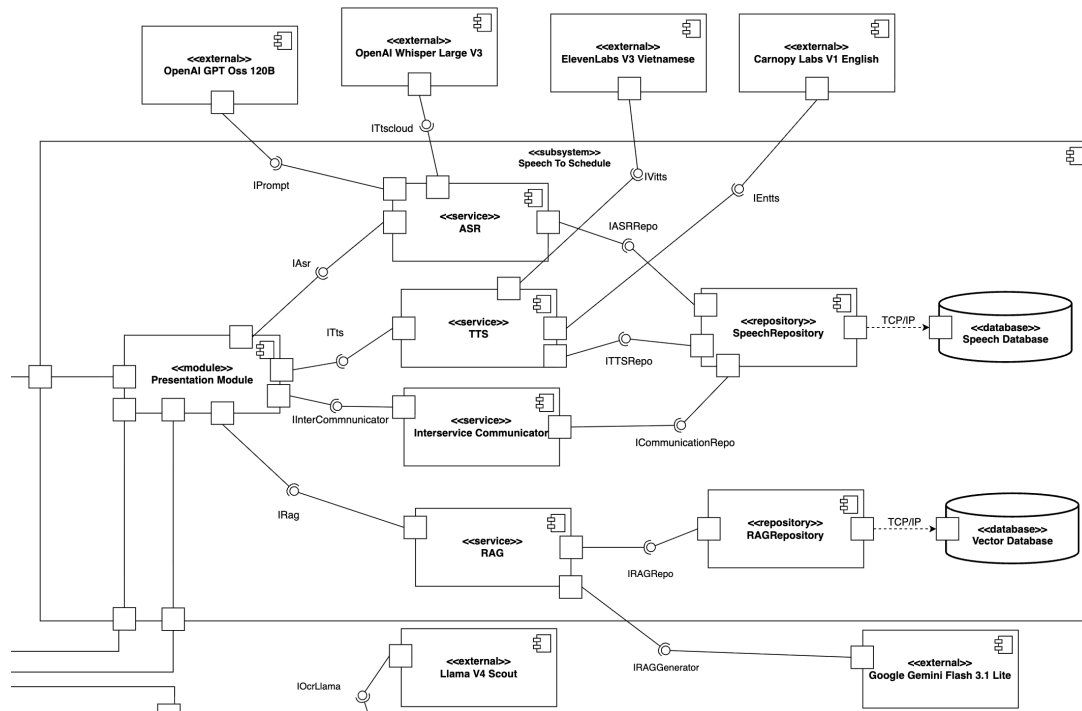


Figure 3.27: Speech To Schedule component view

Figure 3.28 illustrates the Image To Schedule module.

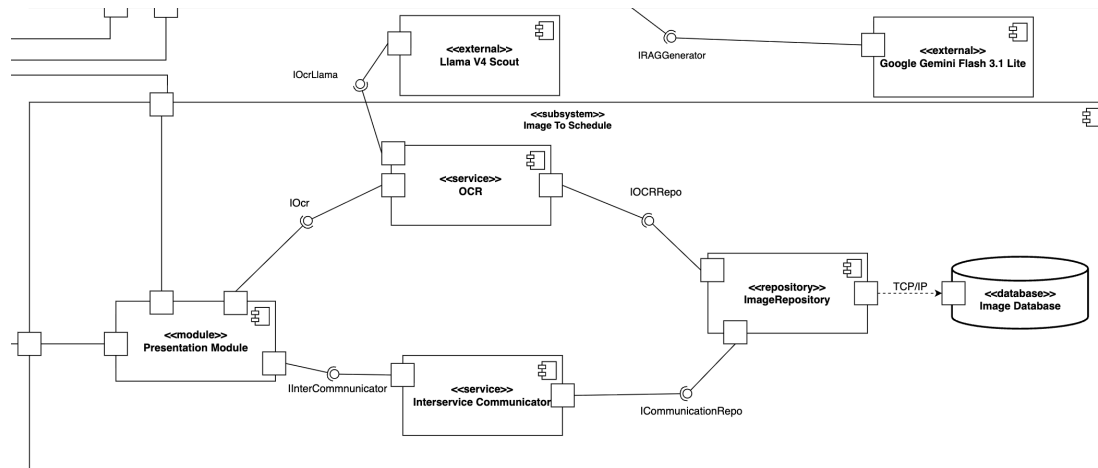


Figure 3.28: Image To Schedule component view

3.5 Database Design

3.5.1 Class Diagram Analysis

System architecture is illustrated through a comprehensive class diagram in **Figure 3.29** that organizes data into several functional domains, prioritizing scheduling, task management, and biometric data tracking. At the core of the system is the Scheduling and Event module, where the **Schedule** class acts as a central hub for managing various **Event** and **EventInstance** objects. These entities store critical temporal information, location data, and recurrence rules, while being organized into distinct categories via the **Category** class to enhance user data filtering and organization.

For task management, the system distinguishes between short-term actions and long-term objectives through the **TaskBase** and **GoalBase** classes. These components are tightly integrated with standardized enumerations such as **TaskStatus** and **TaskPriority**, ensuring that every task has a clear lifecycle and urgency level within the database. Furthermore, the system incorporates an intelligent logging mechanism through the **ImageLog** and **SpeechLog** entities. These classes handle complex data types such as image recognition confidence scores and speech-to-text categories, enabling the system to trigger specific **Reminder** instances based on visual or auditory inputs.

The Authentication and User profile module provides a robust foundation for personalization by storing not only standard credentials but also detailed physical attributes in the **User** class. By including fields for age, gender, height, and weight, the data model supports advanced health-tracking features or personalized scheduling recommendations. All these entities are linked through precise structural relationships, where the **Repository** pattern is utilized for data persistence, and various enums are employed to maintain data integrity across the entire application ecosystem.

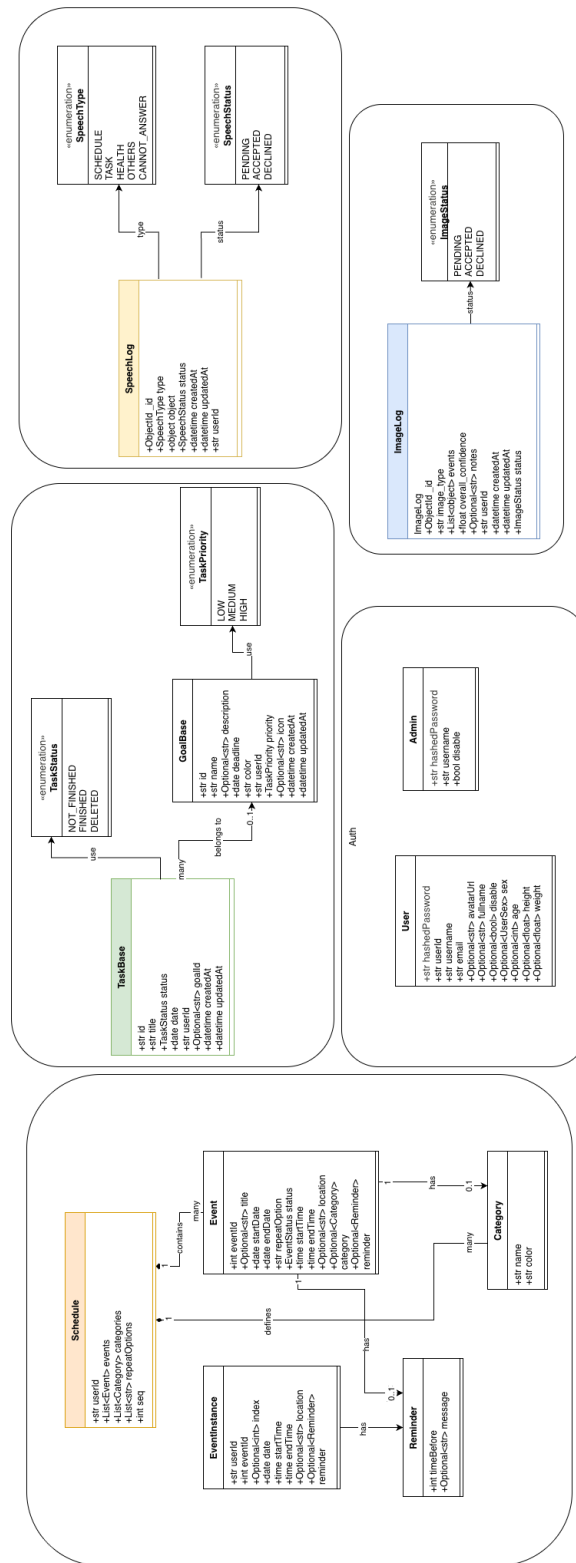


Figure 3.29: Detailed System Class Diagram

Chapter 4

System Implementation

Chapter 4 presents the concrete implementation of the proposed system, translating the design specifications outlined in the previous chapter into a fully functional application. The chapter begins by describing the development environment, tools, and frameworks adopted throughout the implementation process. It then details the front-end and back-end construction, including the implementation of core modules such as user authentication, calendar management, and real-time data synchronization. Special attention is given to the speech-to-schedule module, where the integration of speech recognition APIs and natural language processing pipelines is thoroughly explained. The chapter also covers the implementation of the database layer, API endpoints, and the communication flow between system components. Finally, key challenges encountered during development and the solutions adopted to address them are discussed.

4.1 Back-End Implementation

4.1.1 Service Structure

Within the Alice backend ecosystem, each microservice maintains an isolated codebase to ensure loose coupling and independent deployability. As illustrated in the architecture diagram, every service adheres to a strict Layered Architecture, structured as follows:

```
Service-root
|-- .env                # Local environment variables
|-- Dockerfile          # Containerization instructions
|-- requirements.txt     # Python dependencies
'-- app/                # Core application source code
    |-- config.py       # Environment configuration management
    |-- main.py         # Application entry point (FastAPI)
    |-- schemas.py      # Data Transfer Objects (Pydantic)
    |-- database/       # Database Connection Layer
    |   '-- connection.py
    |-- endpoints/      # Presentation Layer
    |   '-- apis.py
    |-- persistence/    # Data Access Layer
    |   '-- repository.py
    '-- services/       # Business Logic Layer
        '-- services.py
```



This four-tier architecture effectively separates computational concerns. The core components function as follows:

- **.env:** Stores environment variables exclusively for local development and testing (this file is strictly excluded from production environments).
- **Dockerfile:** Contains the containerization instructions required to build the minimal, production-ready Docker image for Kubernetes deployment.
- **requirements.txt:** Declares the explicit Python dependencies and libraries required to execute the service.
- **app/:** The root directory containing the modularized application code.
- **config.py:** Manages global configurations and securely loads environment variables into the application state.
- **schemas.py:** Defines Pydantic models acting as Data Transfer Objects (DTOs) for strict request validation and JSON response serialization.
- **main.py:** The entry point of the FastAPI application, responsible for initializing the ASGI server, configuring middleware, and registering API routers.
- **connection.py:** Handles the asynchronous connection pool and lifecycle management with the MongoDB database instance.
- **repository.py:** Implements the Repository Pattern to abstract database operations (CRUD), isolating direct data access queries from the overarching business rules.
- **services.py:** Encapsulates the core business logic, acting as the intelligent intermediary between the presentation layer and the data persistence layer.
- **apis.py:** The presentation layer that defines the RESTful endpoints, handles HTTP requests, and routes traffic to the appropriate service functions.

The structure above is the general structure of all service in Alice Back End, however, there are still some differences about the structure of each service. In Alice Speech To Schedule and Image To Schedule services, there are many files in services layer to handle the actions.

4.1.2 API Design

The Alice Backend exposes a comprehensive suite of application programming interfaces (APIs) to facilitate communication between the client applications and the underlying microservices. These endpoints are strictly designed in accordance with RESTful architectural principles, ensuring statelessness, standardized HTTP methods, and predictable resource-oriented URLs.

Figure 4.1 illustrates the standardized API structure implemented across the backend ecosystem. To ensure a clear separation of concerns and to facilitate seamless reverse proxy routing through the NGINX API Gateway, all endpoints follow a unified Uniform Resource Identifier (URI) naming convention: `/api/{service-name}/...`

This prefix-based routing strategy allows the system to easily distinguish and direct incoming traffic to the appropriate containerized microservice. Currently, the architectural boundaries dictate the following service prefixes: **tasks**, **schedules**, **auth**, **speech**, and **image**.



Task Management API 1.0.0 OAS 3.1

/api/tasks/openapi.json

FastAPI with MongoDB

tasks			^
GET	/api/tasks/health	Health Check	▼
POST	/api/tasks/	Create Task	▼
GET	/api/tasks/	List Tasks	▼
PATCH	/api/tasks/{task_id}	Update Task	▼
DELETE	/api/tasks/{task_id}	Delete Task	▼
POST	/api/tasks/goals	Create Goal	▼
GET	/api/tasks/goals	List Goals	▼
PATCH	/api/tasks/goals/{goal_id}	Update Goal	▼
DELETE	/api/tasks/goals/{goal_id}	Delete Goal	▼

Figure 4.1: RESTful API design and documentation for the Alice Task Module

4.1.3 Generative AI Integration in Alice

To empower the highly specialized **speech-service** (s2s) and **image-service** (i2s) modules, the Alice Backend extensively leverages Generative Artificial Intelligence. This is achieved by integrating third-party Software Development Kits (SDKs) from **GroqCloud**, **Google AI Studio**, and **ElevenLabs** directly into the business logic layer.

4.1.3.1 Speech-To-Schedule (s2s) Workflow

Within the **speech-service**, the workflow initiates when the mobile application captures the user's voice and transmits the audio payload to the `/api/speech/asr` endpoint. Upon receipt, the audio stream is routed to GroqCloud for high-speed Speech-to-Text (STT) processing utilizing the **Whisper Large V3** model.

While the initial architectural design incorporated a localized Whisper deployment, the resource constraints of containerized environments limited the system to the "medium" model size, which resulted in excessively heavy Docker image builds. Consequently, the STT workload was migrated to GroqCloud's managed API to ensure rapid and accurate transcription without compromising cluster resources.

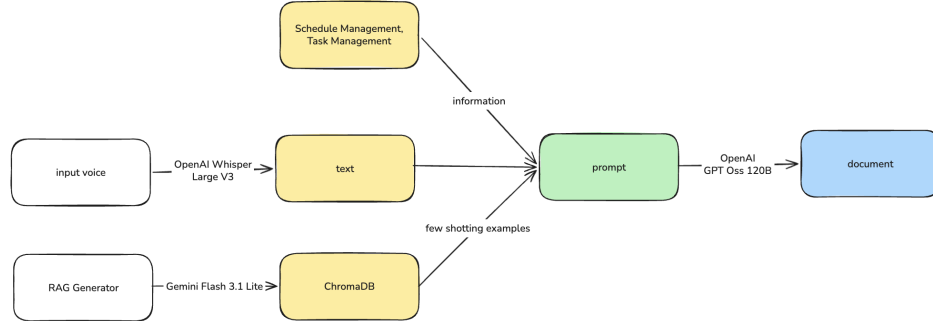


Figure 4.2: Architecture and execution flow of the Speech-to-Schedule module using RAG

As illustrated in **Figure 4.2**, once the raw text is transcribed, the service constructs a dynamic prompt. This operational context is enriched by retrieving relevant data from the **Schedule Management** and **Task Management**. To further enhance the extraction accuracy and context awareness, the system employs a Retrieval-Augmented Generation (RAG) architecture paired with few-shot prompting. Contextual examples are dynamically retrieved from the isolated **ChromaDB** vector store, where semantic embeddings are generated via **Gemini Flash 3.1 Lite**.

The finalized prompt is then dispatched to GroqCloud, where the **OpenAI GPT OSS 120B** Large Language Model performs entity recognition and intent extraction to output a structured JSON document. Following successful parsing, the corresponding database actions are executed. If auditory feedback is required by the user interface, the system synthesizes natural speech using **ElevenLabs V3** for Vietnamese interactions and **Canorpy Labs Orpheus V1** for English interactions.

4.1.3.2 Image-To-Schedule (i2s) Workflow

Conversely, the **image-service** handles visual scheduling inputs. When a user uploads a schedule image, the binary payload is transmitted to the backend and securely forwarded to the Groq-Cloud Vision API.

The service utilizes the **Meta Llama V4 Scout** vision model, accompanied by strict deterministic prompting, to perform Optical Character Recognition (OCR) and extract structured entity data (such as dates, times, locations, and event titles). Upon successful parsing and Pydantic validation, the mapped entities trigger the appropriate database insertion workflows to finalize the event creation. The flow is depicted as in the **Figure 4.3**.

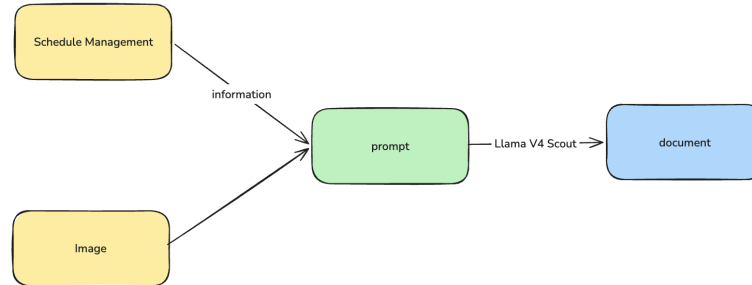


Figure 4.3: Execution flow of the Image-to-Schedule module using Vision AI

4.1.3.3 RAG Database Generation

In the Alice s2s workflow, Retrieval-Augmented Generation (RAG) is applied to every prompt to improve performance. Specifically, three contextual examples are retrieved from ChromaDB and appended to the prompt prior to execution.

For generating the datasets stored within ChromaDB, the system utilizes **Google Gemini Flash 3.1 Lite** rather than the **OpenAI GPT OSS 120B** model used in the primary s2s module. According to the Artificial Analysis Intelligence Index, the Google model scores higher than the OpenAI model in this domain [39]. However, Gemini is exclusively used for this offline generation task because it introduces significant latency, which would negatively impact the real-time user experience if deployed in the active speech pipeline. Internal team evaluations have confirmed that the objects returned by Gemini are more accurate than those generated by OpenAI.

This database generation and validation process is managed exclusively by system administrators.

4.2 Mobile Application Implementation

4.2.1 Application Structure

This feature-based architecture separates the application into clear modules and layers. The core components of the Flutter project are organized as follows:

- **lib/**: The root directory containing the main application source code.
- **app/**: Handles app-level setup such as **AppShell**, tab layout, global routing, dependency initialization, and shared navigation configuration.
- **components/**: Stores reusable UI components such as buttons, dialogs, cards, tiles, and shared input widgets. These components should not contain business logic.
- **core/**: Contains shared infrastructure used across multiple features, including network clients, local storage, constants, base classes, utilities, and common helpers.



- **features/**: The main directory for feature modules. Each feature is responsible for its own data, domain, and presentation logic.
- **features/calendar/**: Handles the calendar module, including calendar views, event display, event creation, editing, deletion, filtering, and navigation inside the calendar feature.
- **features/calendar/data/**: Contains the data layer of the calendar feature, including API services, request/response models, DTOs, and repository implementations.
- **features/calendar/domain/**: Contains the business layer of the calendar feature, including entities, use cases, and pure business rules that should not depend on the UI.
- **features/calendar/presentation/**: Contains the UI layer of the calendar feature, including pages, widgets, state handling, and presentation-specific helpers.
- **features/calendar/presentation/pages/**: Stores calendar screens. Each page can be separated into a UI file and a state file, for example `calendar_homepage.dart` and `calendar_homepage_state.dart`.
- **features/calendar/presentation/widgets/**: Stores smaller reusable widgets used by calendar pages, such as event cells, month event boxes, headers, view selectors, and empty states.
- **features/calendar/presentation/helper/**: Stores helper classes and functions for formatting, mapping, layout calculation, and UI-specific transformations.
- **features/calendar/presentation/calendar_navigator.dart**: Manages internal routing for the calendar module. This keeps calendar navigation separated from the global application shell.
- **features/task/**: Handles the task module, including task listing, task editing, completion status, drag interactions, and goal assignment.
- **features/auth/**: Handles authentication-related flows such as login, registration, session validation, token storage, and user authentication state.
- **features/health/**: Reserved for health-related features and future health integrations.
- **features/speech/**: Handles speech and voice interaction features.
- **features/image/**: Handles image analysis workflows, such as analyzing an uploaded image and converting it into calendar events or tasks.
- **features/recorder/**: Handles audio recording logic and related recording utilities.
- **theme/**: Defines the application design system, including colors, text styles, spacing, border radius, shadows, and other UI tokens.
- **main.dart**: The entry point of the Flutter application. It initializes the application and runs the root widget.

The structure above is the general structure of the Alice mobile application. Each feature follows the same layered organization to improve maintainability, reduce coupling, and make the code easier to scale. For larger features such as Calendar, the presentation layer is further divided into **pages**, **widgets**, and **helper** folders so that screen layout, reusable UI components, state handling, and helper logic remain clearly separated.



4.2.2 API Integration

4.2.2.1 Dio Client

API integration is an important part of the mobile implementation because the application needs to communicate with the backend server to retrieve, create, update, and delete data across different modules such as calendar scheduling, task management, image processing, and speech processing. To support these requirements, the application uses Dio as the main HTTP client for backend communication.

Dio was selected because it provides a more scalable and maintainable networking approach compared with a basic HTTP client. A reusable Dio client allows the application to centralize common request configurations such as base URL, headers, authentication tokens, timeout settings, and response handling. This helps reduce duplicated networking logic across features and keeps API communication separated from the UI layer.

One of the main advantages of Dio is its interceptor mechanism. Interceptors make it possible to handle shared request and response behavior in one place, such as attaching the authentication token, logging request and response data, handling unauthorized responses, and converting API errors into a consistent format. This is especially useful in a multi-feature application where many API calls require the same authentication and error-handling logic.

Dio also improves development and debugging through structured error handling and request logging. Different failure cases such as connection timeout, server error, invalid response, or cancelled requests can be handled more consistently. In addition, logging helps verify request payloads and response data during development, especially for features that depend on accurate data submission, such as calendar event creation and event time updates.

Another important reason for using Dio is its convenient support for query parameters and file upload. Query parameters are useful when retrieving filtered data, such as loading calendar events within a specific date range. File upload support through FormData is also suitable for this application because image and speech-related features require sending image files or audio files to the backend for processing.

Overall, Dio supports a cleaner API integration layer by centralizing network configuration, improving error handling, supporting request logging, and simplifying file upload. This makes it suitable for the current application architecture and helps keep the mobile implementation more maintainable and scalable.

4.2.2.2 API Integration Architecture

In this project, the API integration layer is not implemented directly inside the user interface. Instead, the application separates API-related responsibilities into different components, including service classes, controller classes, and DTO objects. This structure helps keep the networking logic organized and prevents the UI layer from depending directly on low-level HTTP requests. As a result, the application becomes easier to maintain, extend, and debug as the number of features increases.

The service layer is responsible for communicating directly with the backend server through the Dio client. Each service class defines the API endpoints and request methods required by

a specific feature. For example, the authentication service handles API calls such as login, registration, user information retrieval, and token validation, while the schedule service handles calendar-related APIs such as retrieving events, creating events, updating events, deleting events, managing categories, and retrieving repeat options. By placing these API calls inside service classes, the application avoids duplicating HTTP request logic across different screens and keeps backend communication centralized.

The controller layer acts as an intermediate layer between the service layer and the rest of the application. Instead of allowing the UI to work with raw API responses, the controller processes the data returned from the service and prepares it for application use. This includes parsing response data, converting response objects into application models, storing data in memory cache or local storage, and handling fallback logic when needed. For example, the schedule controller retrieves event data from the schedule service, maps the response into calendar event models, and stores the result locally. Similarly, the authentication controller manages user information by saving the latest user data into cache and local storage, allowing the application to reuse the data without repeatedly depending on direct API calls.

DTO objects are used to standardize the structure of request and response data exchanged between the mobile application and the backend. Instead of passing raw maps throughout the application, DTO classes define clear data formats for operations such as creating an event, updating an event, updating a recurring event instance, or reading an event response. This reduces the risk of incorrect JSON keys, improves readability, and makes the API layer easier to modify when the backend data structure changes. DTOs also help separate backend data formats from the models used internally by the application.

This API integration architecture supports the principles of Clean Architecture and Separation of Concerns. The presentation layer does not need to know how HTTP requests are created, which endpoint is called, or how backend responses are structured. The service layer focuses on backend communication, the controller layer focuses on data flow and application logic, and DTOs define the data contract between the mobile app and the backend. This separation makes the codebase more scalable and maintainable, especially as the application includes multiple modules such as authentication, calendar scheduling, task management, image processing, and speech processing.

4.2.3 Data Management Service

4.2.3.1 Hive - Offline Service

To support offline mode, the application uses Hive as the local storage solution. Since the mobile application depends on backend APIs for data such as user information, calendar events, categories, and repeat options, storing selected data locally helps improve reliability when the network connection is unstable or unavailable.

One of the main purposes of using Hive is to allow the application to continue displaying previously loaded data when an API request fails. After data is successfully retrieved from the backend, it can be saved into local storage and reused later as cached data. For example, calendar events, category lists, repeat options, and user profile information can still be available even when the device temporarily loses connection to the server.

Hive also provides fast local data access, which is suitable for mobile applications that need quick loading and smooth user interaction. Instead of waiting for every screen to reload data from the backend, the application can read cached data from local storage when appropriate. This helps improve the perceived performance of the application and provides a more stable user experience.

Another advantage of Hive is that it reduces the application's dependency on constant network availability. If the backend server is unreachable or the network connection is weak, the application can fall back to the latest stored data instead of showing an empty state or interrupting the user flow completely. This is especially useful for features such as calendar scheduling, where users may still need to check their existing events even without an active internet connection.

Hive is also integrated into the application's layered architecture. The local storage logic is separated into dedicated storage classes, while controllers decide when to retrieve data from the backend and when to use cached data from Hive. This keeps the UI layer independent from the local storage implementation and makes the codebase easier to maintain and extend.

Overall, Hive was selected because it supports offline mode, provides fast local data access, simplifies local caching, and improves the reliability of the application under unstable network conditions. By combining backend APIs with Hive-based local storage, the application can provide a smoother and more dependable user experience.

4.2.3.2 Cache Service

A key strength of the application's caching approach is that it does not rely only on one type of cache. Instead, the system combines both persistent local cache and in-memory cache to balance offline reliability and runtime performance. Persistent cache is handled by Hive through a dedicated local storage service, where calendar events, categories, and repeat options are saved into a separate storage box and restored when needed. This allows important schedule data to remain available even after the application is closed or when the device loses network access. For example, events are converted into cache-friendly DTO objects before being stored, while categories and repeat options are saved using their own cache keys, making the cached data organized and easy to retrieve.

Besides Hive storage, the application also uses a lightweight memory cache for frequently accessed data such as categories and repeat options. This helps reduce unnecessary API calls and avoids repeatedly reading the same data from local storage during a single app session. When data is successfully loaded from the backend, it is saved into both memory cache and Hive storage. When the API request fails, the controller can fall back to Hive and then update the memory cache with the local data. This creates a layered cache strategy: the backend remains the main source of fresh data, Hive acts as a persistent offline backup, and memory cache provides fast access while the application is running.

This design also follows the application's clean architecture direction. Network requests are handled by the schedule service, local persistence is isolated inside the Hive storage service, and cache coordination is managed by the controller instead of the UI layer. As a result, the user interface does not need to know whether data comes from the backend, memory cache, or Hive. This separation makes the application easier to maintain, improves performance, and provides a more dependable experience when users open the calendar under unstable network conditions.

4.2.4 Schedule Module

4.2.4.1 Calendar View By Week



Figure 4.4: Weekly calendar interface for schedule management

The weekly calendar screen is the main interface for managing and viewing scheduled events in the application. It presents events within a seven-day layout, helping users easily observe their schedule and interact with events in a clear and organized manner.



The calendar header displays the current month and year, together with the weekdays from Monday to Sunday. The current date is highlighted with a purple indicator to help users quickly identify today. Users can swipe left or right to navigate between weeks, while the calendar icon on the right side allows them to quickly return to the current date.

The main body of the screen is structured as a time-based grid. The vertical axis shows hourly intervals from 00:00 to 23:00, while the horizontal axis represents the days of the selected week. This layout allows events to be placed according to their date, start time, and end time, providing a clear overview of the weekly schedule.

4.2.4.2 Calendar View By Month

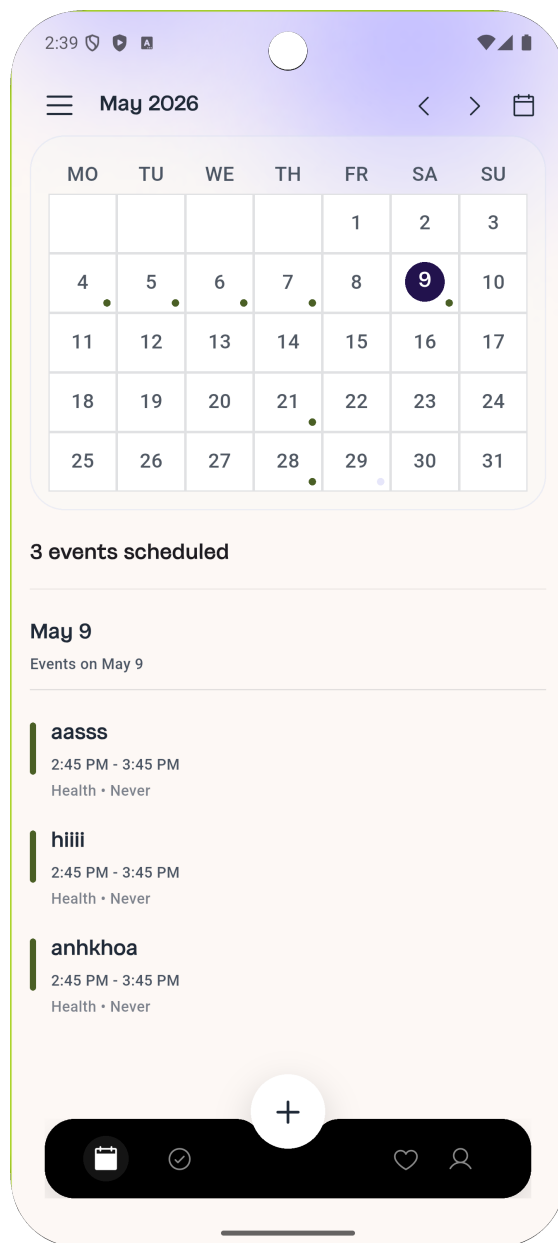


Figure 4.5: Weekly calendar interface for schedule management

The monthly calendar screen provides a broader overview of scheduled events within a selected month. Instead of focusing on hourly details, this view allows users to quickly observe which days contain events and navigate through the month in a simple and organized layout.

The calendar header displays the current month and year, together with navigation controls for



moving to the previous or next month. A calendar icon is also provided on the right side, allowing users to quickly return to the current date when needed.

The main calendar area is arranged as a monthly grid, with weekdays displayed from Monday to Sunday. Each cell represents a specific date in the selected month. The current or selected date is highlighted with a purple circular indicator, while small colored dots below certain dates indicate that events are scheduled on those days. This helps users recognize busy days without opening each date individually.

Below the monthly grid, the screen displays a list of scheduled events for the selected date. Each event item shows the event title, time range, category, and repeat information. A small colored indicator on the left side of each event reflects its category, making it easier for users to distinguish between different types of schedules.

4.2.4.3 Calendar Drawer

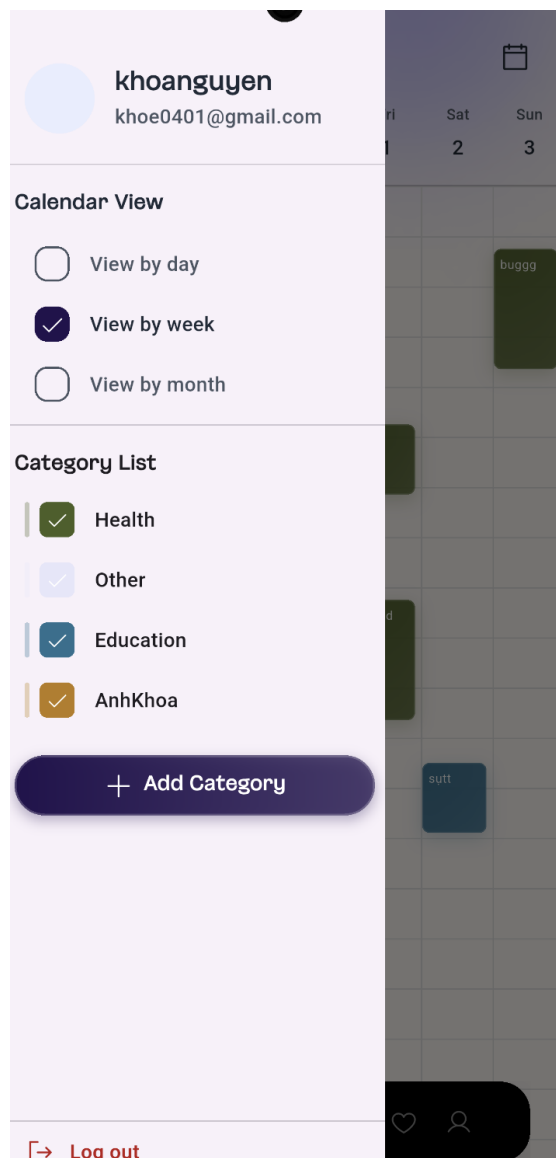


Figure 4.6: Calendar drawer for view configuration and category filtering

The navigation drawer provides additional configuration options for the calendar view, as illustrated in **Figure 4.6**. It allows users to switch between day, week, and month views, supporting flexible schedule visualization based on user preferences.

The drawer also includes a category list that enables users to filter events dynamically. Each category is associated with a specific color and can be toggled on or off. When a category is disabled, its corresponding events are temporarily hidden from the calendar view. This filtering mechanism helps reduce visual clutter and allows users to focus on specific types of activities.

In addition, the drawer supports category management through an add action, allowing users to create new categories and customize their scheduling system. This improves the flexibility and personalization of the calendar module.

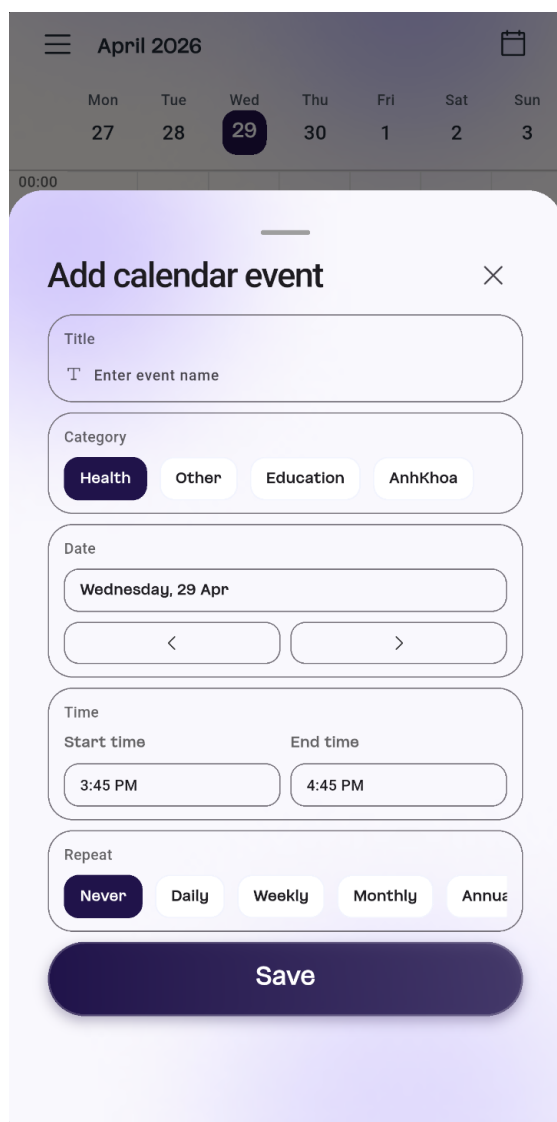


Figure 4.7: Event creation interface

The event creation interface, shown in **Figure 4.7**, enables users to add new events to the calendar in a structured and user-friendly manner. It is presented as a modal bottom sheet, allowing users to enter event details without leaving the main calendar screen.

The form is organized into clear input sections. Users can enter the event title, select a category,



choose a date, and define the start and end time. Each category is represented with a distinct color, maintaining consistency with the color-coding system used throughout the calendar view.

The date selection component allows users to specify the event date and quickly navigate between days. The time selection fields support precise scheduling by separating the start time and end time of the event.

In addition, the interface supports recurring event configuration through repeat options such as daily, weekly, monthly, and yearly. This allows users to create repeated schedules without manually adding each occurrence.

After completing the required information, users can confirm the creation process using the primary Save button. The newly created event is then reflected in the calendar view, helping maintain a smooth connection between event input and schedule visualization.

4.2.5 Task Module

4.2.5.1 Task Home Page

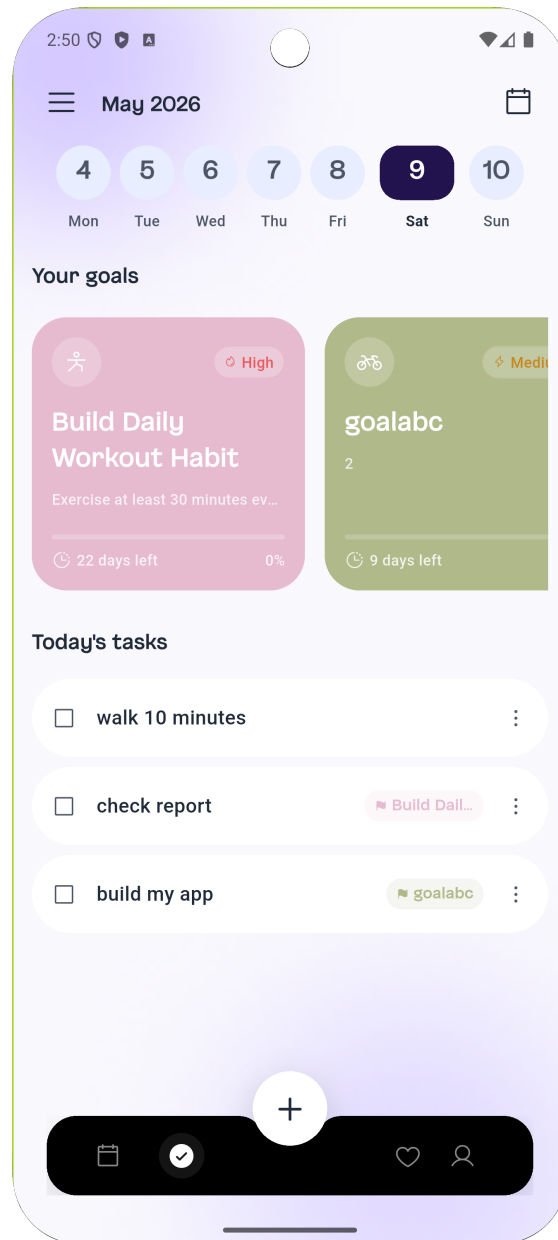


Figure 4.8: Task management interface with daily task list

The task management screen provides users with a daily overview of their goals and tasks, as illustrated in **Figure 4.8**. At the top of the screen, a horizontal date selector allows users to navigate between days and view the tasks associated with each selected date. The current or



selected date is highlighted to provide clear visual feedback.

The Your Goals section displays active goals as visual cards. Each goal card presents key information such as the goal title, priority level, remaining days, and progress percentage. The use of soft colors and small icons helps distinguish goals visually while maintaining a clean and modern interface.

Below the goal section, the Today's Tasks area lists tasks scheduled for the selected day. Each task item includes a checkbox for completion, the task title, and an optional goal label when the task is linked to a specific goal. This structure allows users to quickly understand what needs to be completed and how each task contributes to their larger goals.

The screen also supports task interaction features such as completing, editing, and deleting tasks. When a task is deleted, an undo action is provided, allowing users to restore the task immediately if the deletion was accidental. This improves usability by reducing the risk of losing important task information.

A floating action button at the bottom center allows users to create new tasks or goals efficiently. The bottom navigation bar provides quick access to other main modules of the application, ensuring smooth movement between calendar, task, goal, and profile-related features.

4.2.5.2 Task Detail Page

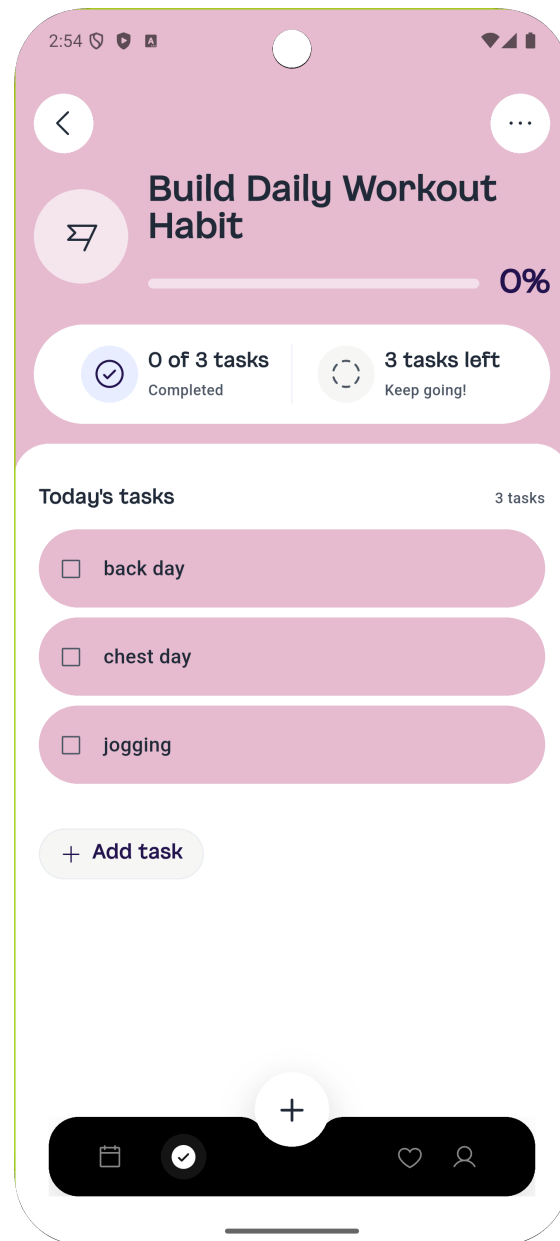


Figure 4.9: Goal overview and task grouping interface

The goal detail screen, illustrated in **Figure 4.9**, provides a focused view of a specific goal and its associated tasks. At the top of the screen, the goal title is displayed together with a progress indicator, allowing users to quickly understand their current completion status.

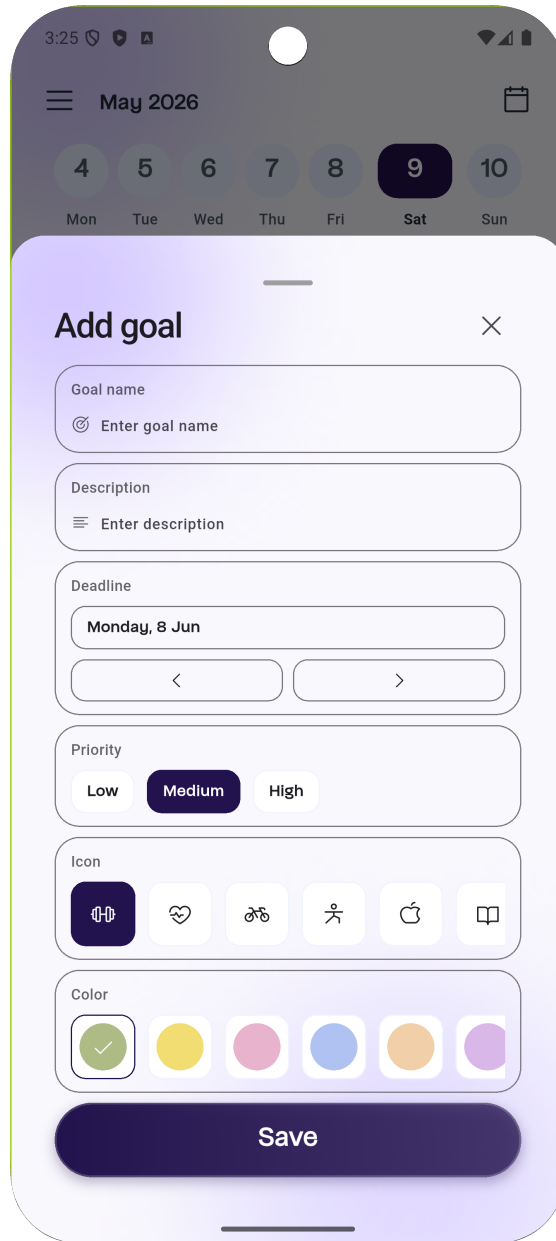
The progress section summarizes the number of completed tasks and remaining tasks. This in-



formation is dynamically updated based on the completion status of the tasks linked to the goal, giving users immediate feedback on their advancement.

The Today's tasks section lists the tasks associated with the selected goal. Each task is displayed as a list item with a checkbox, allowing users to mark it as completed or incomplete. This interaction helps users track daily progress in a simple and direct manner.

Users can add new tasks to the goal through the Add task action. This supports goal-oriented task management by allowing users to expand a goal into smaller actionable steps without leaving the detail screen.



The image shows a mobile application interface for creating or editing a goal. At the top, there is a status bar with the time 3:25 and various icons. Below it is a calendar header for May 2026, with days 4 through 10 visible. The main content is a modal form titled "Add goal" with a close button (X) in the top right corner. The form contains several input fields and options: a "Goal name" field with a placeholder "Enter goal name", a "Description" field with a placeholder "Enter description", a "Deadline" field showing "Monday, 8 Jun" with left and right navigation arrows, a "Priority" section with buttons for "Low", "Medium" (selected), and "High", an "Icon" section with six icons (a selected one with a plus sign, a heart, a bicycle, a person, an apple, and a book), and a "Color" section with six colored circles (a selected green one with a checkmark, yellow, pink, blue, orange, and purple). At the bottom of the form is a large "Save" button.

Figure 4.10: Goal creation and editing interface

The goal creation and editing interface is presented as a modal form, similar to the event creation interface in the calendar module, as shown in **Figure 4.10**. This design ensures consistency across different parts of the application and provides users with a familiar interaction pattern when creating or updating data.

The form allows users to define the main information of a goal, including the goal name and description. These fields help users clearly describe the objective and provide additional context



for what they want to achieve.

The deadline section enables users to specify the target completion date of the goal. Navigation controls are provided to move between dates quickly, making the date selection process simple and efficient.

The priority section allows users to classify the importance of the goal using predefined levels such as low, medium, and high. This helps users visually distinguish between goals based on urgency or importance.

In addition, users can customize the goal by selecting an icon and color. These visual elements make each goal easier to recognize in the goal list and improve the overall personalization of the task management experience.

Chapter 5

Deployment and System Evaluation

In Chapter 5, we delve deeply into the meticulous process of testing the system to guarantee its intended functionality. We employ diverse testing methodologies, including unit tests tailored for blackbox testing, to scrutinize each facet of the system comprehensively. By devising specific test sets, we gain nuanced insights into the system's performance and its alignment with user expectations. Additionally, this chapter expounds on deployment strategies, enabling informed decisions regarding system enhancement and implementation procedures.

5.1 Deployment

5.1.1 Mobile Application Deployment

This section describes the deployment process of the mobile application for testing on physical devices. The application is developed using Flutter and supports both Android and iOS platforms.

5.1.1.1 Android Deployment

For Android testing, the application is packaged into an APK (Android Package Kit), which allows direct installation on physical devices. The release APK is generated through the Flutter build process, and the output file is located at:

```
build/app/outputs/flutter-apk/app-release.apk
```

Release builds must be signed using a valid keystore. Unlike debug builds, which use a default debug key, release APKs require a custom signing configuration to ensure authenticity and compatibility with physical devices. In addition, the application must be built in *release mode* to achieve optimized performance and production-like behavior, as debug builds include additional logging and are not suitable for real device testing.

The generated APK can be installed on Android devices via USB connection. This process requires enabling developer options and USB debugging on the target device. If a previous version of the application exists with a different signature, it must be uninstalled prior to installation to prevent conflicts.



Before deployment, it is also necessary to ensure that the application is properly configured to communicate with a valid backend environment. Emulator-specific configurations, such as local host addresses, must be replaced with accessible endpoints.

The primary testing artifact for Android is:

`app-release.apk`

This file is suitable for internal distribution and manual installation on physical devices.

5.1.1.2 iOS Deployment

The iOS deployment process requires a macOS-based development environment, including Xcode and a valid Apple Developer account. Unlike Android, iOS applications must be properly signed before they can be installed on physical devices.

Applications are signed using Apple-issued certificates and provisioning profiles, ensuring that the application is trusted by the operating system and can be deployed to registered devices. Similar to Android, the application must be built in *release mode* to ensure stable performance and behavior consistent with production environments.

The iOS application is compiled using the Flutter build system and then archived using Xcode. This process produces an IPA (iOS App Store Package) file, which serves as the distributable format for iOS applications. The resulting testing artifact is:

`Runner.ipa`

Due to platform restrictions, iOS applications cannot be installed directly in the same manner as Android APK files. Instead, distribution must be performed through official channels such as TestFlight, Ad Hoc provisioning, or third-party services such as Firebase App Distribution. Among these options, TestFlight is commonly used for standard testing scenarios.

5.1.1.3 Additional Considerations

Testing builds should be validated across multiple devices, screen sizes, and operating system versions to ensure consistent behavior and user experience.

Applications deployed on physical devices must correctly handle runtime permissions, including access to the camera, storage, and notifications. Failure to manage these permissions properly may result in incomplete or non-functional features.

Although applications may run successfully on emulators or simulators, discrepancies can occur on physical devices due to differences in hardware capabilities, performance constraints, and system-level behavior.

The deployment process follows a structured workflow:

Source Code → Build Release → Generate APK/IPA → Install on Device

Applications cannot be directly exported from an emulator to a physical device. A proper release build must always be generated to ensure compatibility and reliability during testing.

5.1.2 BackEnd Deployment

5.1.2.1 Alignment with Cloud Native Computing Foundation (CNCF) Standards

Cloud-native applications are built fundamentally to leverage the flexible and distributed characteristics of cloud environments. In contrast, monolithic applications operate as a singular, unified entity, frequently relying on custom-built operating systems, middleware, and language stacks. Consequently, this traditional architecture fosters tight internal dependencies, complicating the processes of modifying, testing, deploying, and maintaining the system as it scales over time.

Conversely, cloud-native architectures deliver enhanced speed, agility, scalability, dependability, and overall efficiency. These applications are decoupled into numerous independent services through the application of DevOps methodologies and technologies. This modular structure empowers development teams to deploy and scale individual components in isolation, facilitating seamless updates, bug fixes, and the introduction of new features without disrupting the broader system. [28]

To ensure scalability, resilience, and vendor neutrality, the deployment architecture of the Alice Backend is designed in strict adherence to the principles outlined by the Cloud Native Computing Foundation (CNCF).

The architecture is established by first decomposing the backend modules into distinct services. These services are subsequently containerized into deployable images. A container orchestration platform is then employed to manage these instances, enabling the independent scaling and automated self-healing of each service. Finally, external access is consolidated by exposing a single LoadBalancer, which acts as the primary gateway bridging the backend with other system components.

5.1.2.2 Containerization Strategy

Containerization represents a modern paradigm for packaging and executing software. Rather than installing applications and their requisite dependencies directly onto a host server, all necessary components—including the source code, libraries, and system tools—are encapsulated within a single, isolated container. [29]



Figure 5.1: Docker Containerization



Docker is a widely adopted containerization platform, depicted in **Figure 5.1**, which provides a comprehensive suite of tools for container lifecycle management [30]. Within the Alice Backend architecture, every service utilizes a dedicated **Dockerfile** to construct its container image. The lightweight `python:3.12-slim` was selected as the base image for these services. During the build process, essential system utilities such as `curl` and `dnsutils` are installed. Subsequently, the service's source directory is copied into the container, and the Python dependencies specified within the `requirements.txt` file are installed. Finally, each service is configured to expose port 8000 and execute a predefined startup command.

Once the **Dockerfile** configurations are established, the services are compiled into deployable images using the Docker build tools. These generated images are then uploaded to **Docker Hub**, a centralized cloud registry for Docker containers.

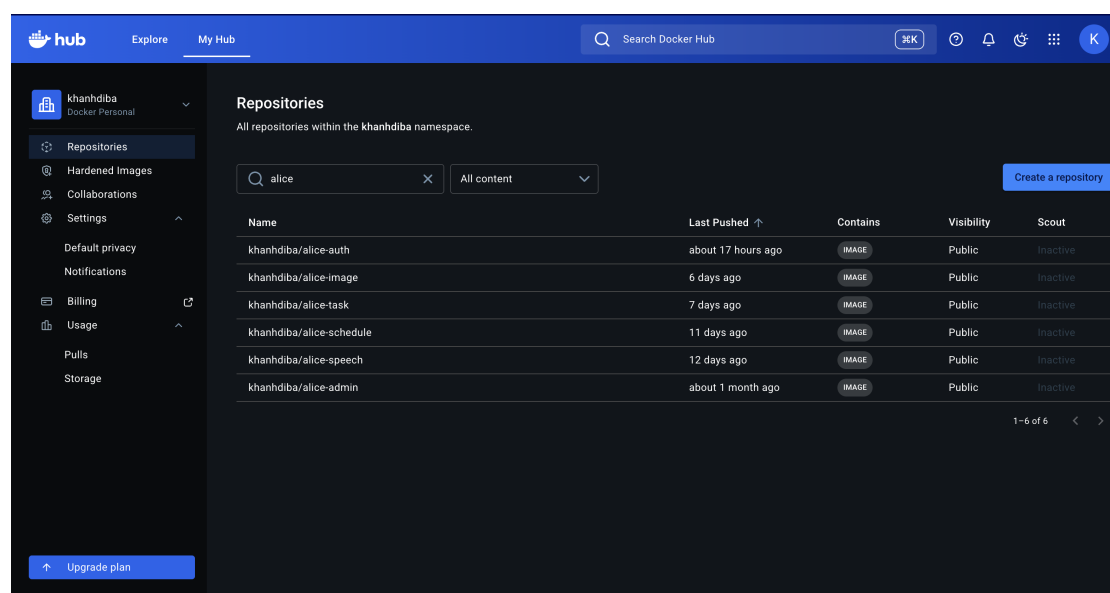


Figure 5.2: Docker Hub

As illustrated in **Figure 5.2**, Docker Hub serves as the repository where these built images are stored, ensuring they are readily accessible for subsequent deployment phases. age, where we store the images for later use in next phase.

5.1.2.3 Cloud Native Infrastructure

Kubernetes (K8s), illustrated in **Figure 5.3**, is an open-source platform designed to automate the management of containerized workloads and services, supported by a rapidly expanding ecosystem [31]. In the Alice Backend architecture, Kubernetes serves as the primary container orchestration engine. To interact with the cluster, we utilize `kubectl`, a dedicated command-line interface.



Figure 5.3: Kubernetes

As demonstrated in **Figure 5.4**, `kubectl` is executed via the terminal to provision and administer various cluster resources, including pods, services, and deployments.

```
mac — zsh — 120x30

[khanhdiba ~ % kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
admin-service-64d4cd8578-fsfwf      1/1     Running   0           33d
admin-service-64d4cd8578-sz6ls      1/1     Running   0           33d
auth-service-69747696dc-kxkmr       1/1     Running   0           17h
auth-service-69747696dc-mrvz9       1/1     Running   0           17h
chromadb-0                           1/1     Running   0           11d
image-service-d849bb588-7qjrj       1/1     Running   0           5d14h
image-service-d849bb588-wtcgg       1/1     Running   0           5d14h
mongodb-auth-0                       1/1     Running   0           40d
mongodb-image-0                     1/1     Running   0           10d
mongodb-schedule-0                  1/1     Running   0           40d
mongodb-speech-0                    1/1     Running   0           16d
mongodb-task-0                      1/1     Running   0           40d
schedule-service-56d4496d76-d297d   1/1     Running   0           11d
schedule-service-56d4496d76-tlcd5    1/1     Running   0           11d
speech-service-95bf884cd-6pvjq       1/1     Running   1 (11d ago)  11d
speech-service-95bf884cd-n6bdt       1/1     Running   1 (11d ago)  11d
task-service-7d95bb7df-n4tcj        1/1     Running   0           7d
task-service-7d95bb7df-xx8cm        1/1     Running   0           7d
web-frontend-54c88f9f7-s6qj4        1/1     Running   0           10d
khanhdiba ~ %
```

Figure 5.4: How kubectl is used in system

To host the orchestration environment, a cloud provider with native Kubernetes support is required. DigitalOcean, an American multinational cloud infrastructure provider depicted in **Figure 5.5**, was selected to host the backend cluster. The primary motivation for adopting DigitalOcean is the availability of free promotional credits provided through the GitHub Student Developer Pack [32].

Managing a Kubernetes environment necessitates robust node and infrastructure management tools. For local development and orchestration, `minikube` (**Figure 5.6**) is employed to simulate



Figure 5.5: Digital Ocean

a localized cluster. Conversely, `doctl`, the official command-line interface for DigitalOcean, is utilized for remote cluster administration.



Figure 5.6: Minikube

The dashboard in **Figure 5.7** overviews the Alice system's production environment on DigitalOcean's Managed Kubernetes Service (DOKS). The cluster utilizes a High Availability (HA) control plane to ensure fault tolerance. Workloads are hosted on the `alice-pool` node pool, which operates a single worker node equipped with 2 vCPUs, 4 GiB of RAM, and 80 GiB of storage. The network is secured within the Singapore-based `default-sgp1` VPC, using explicitly defined CIDR blocks for internal routing. The estimated total cost is \$64 per month, covering both compute resources and the HA premium.

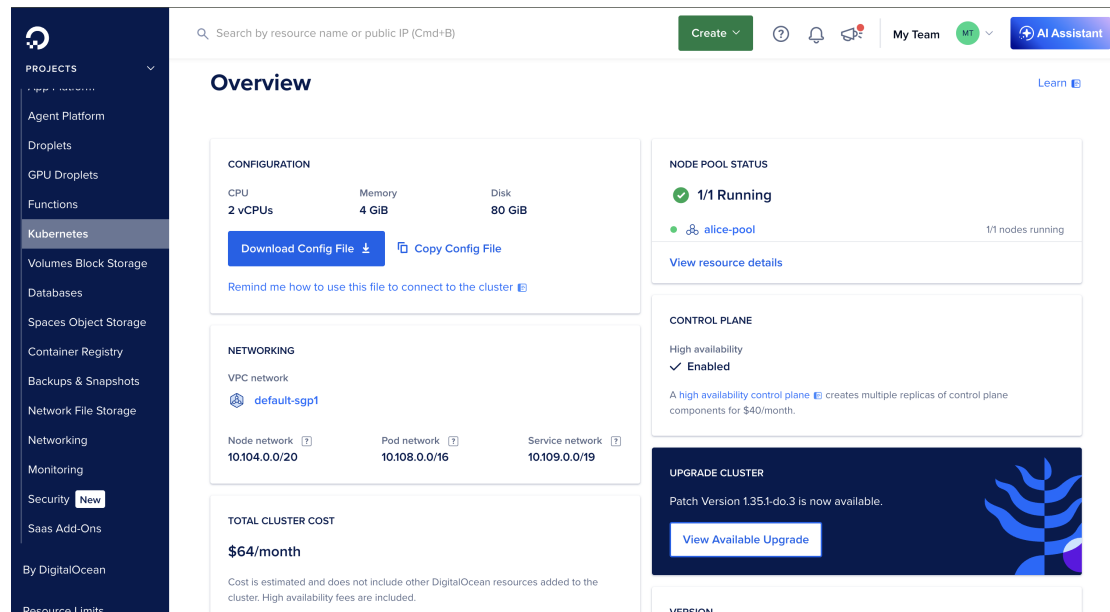


Figure 5.7: DigitalOcean Dashboard

5.1.2.4 Stateless Workload Management

The Alice Backend consists of independent, domain-specific microservices. The five primary services—**auth-service**, **schedule-service**, **task-service**, **speech-service**, and **image-service**—are orchestrated utilizing **Deployment** workloads. Each deployment object is configured with 2 **replicas** to ensure High Availability (HA) and robust load distribution. This redundancy guarantees that in the event of a pod failure, the secondary pod seamlessly maintains operations. Furthermore, Kubernetes' self-healing capabilities are leveraged alongside a **RollingUpdate** strategy for each deployment to facilitate zero-downtime updates. **Figure 5.8** depicts the services and deployments within the backend; while the system comprises various services, only the core modules are managed via Deployments. Additionally, a dedicated **home-page-service** is provisioned specifically to host the API documentation.

```

● kxanhdiba Alice_Backend % kubectl get svc
NAME                                TYPE             CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
admin-service                      ClusterIP         10.109.18.87    <none>           8000/TCP         40d
auth-service                       ClusterIP         10.109.13.114   <none>           8000/TCP         40d
chromadb-service                   ClusterIP         None            <none>           8000/TCP         11d
home-page-service                  ClusterIP         10.109.10.107   <none>           80/TCP           38d
image-service                      ClusterIP         10.109.20.138   <none>           8000/TCP         10d
kubernetes                         ClusterIP         10.109.0.1      <none>           443/TCP          41d
mongodb-auth-service              ClusterIP         None            <none>           27017/TCP        40d
mongodb-image-service             ClusterIP         None            <none>           27017/TCP        10d
mongodb-schedule-service          ClusterIP         None            <none>           27017/TCP        40d
mongodb-speech-service            ClusterIP         None            <none>           27017/TCP        16d
mongodb-task-service              ClusterIP         None            <none>           27017/TCP        40d
schedule-service                  ClusterIP         10.109.20.55    <none>           8000/TCP         40d
speech-service                    ClusterIP         10.109.11.204   <none>           8000/TCP         26d
task-service                      ClusterIP         10.109.11.49    <none>           8000/TCP         40d
● kxanhdiba Alice_Backend % kubectl get deploy
NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
admin-service                      2/2      2              2            40d
auth-service                       2/2      2              2            40d
image-service                      2/2      2              2            10d
schedule-service                   2/2      2              2            40d
speech-service                     2/2      2              2            26d
task-service                      2/2      2              2            40d
web-frontend                      1/1      1              1            38d
○ kxanhdiba Alice_Backend %

```

Figure 5.8: Services and Deployments in Alice

To facilitate internal routing, each component is configured as a **ClusterIP** service, ensuring that inter-service communication relies strictly on internal DNS resolution. With the exception of the **home-page-service** (exposed on port 80), all deployments are exposed internally on port 8000. By utilizing **ClusterIP**, these microservices are intentionally isolated without public IP addresses, remaining fully encapsulated within the DigitalOcean cluster network. Consequently, all external traffic must be routed through an Ingress Controller, as illustrated in **Figure 5.9**, before reaching the backend services.

```

● kxanhdiba Alice_Backend % kubectl get svc -n ingress-nginx
NAME                                TYPE             CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
ingress-nginx-controller            LoadBalancer     10.109.20.14    <none>           80/TCP           40d
ingress-nginx-controller-admission ClusterIP         10.109.6.74     <none>           443/TCP          40d
○ kxanhdiba Alice_Backend %

```

Figure 5.9: NGINX Ingress Controller

To maintain environmental security and adhere to best practices for stateless workloads, configuration data is strictly decoupled from the deployment objects. Environment variables are never hard-coded into the container images. Instead, Kubernetes **ConfigMap** and **Secret** resources are

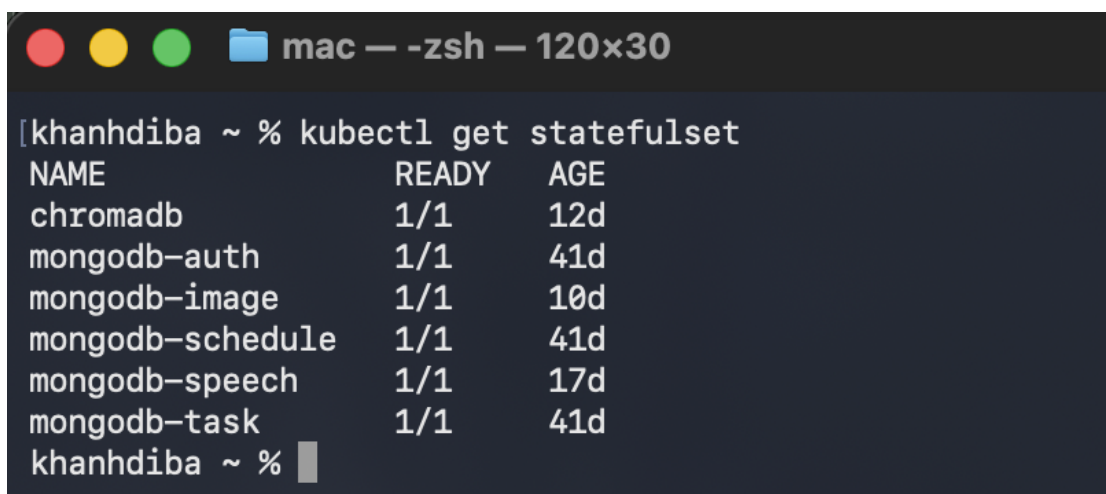
utilized to manage these parameters dynamically, as shown in **Figure 5.10**. ConfigMaps are employed to store non-confidential operational variables, such as database names and access token expiration times. Conversely, **Secrets** are strictly designated for highly sensitive credentials, including API keys and database passwords.

```
● khanhdiba Alice_Backend % kubectl get configmap
NAME          DATA  AGE
admin-config  3      40d
auth-config   3      40d
home-page-html 1      38d
kube-root-ca.crt 1      41d
schedule-config 1      40d
task-config    1      40d
● khanhdiba Alice_Backend % kubectl get secret
NAME          TYPE          DATA  AGE
alice-tls     kubernetes.io/tls  2      38d
auth-secret   Opaque        3      40d
image-secret  Opaque        7      10d
schedule-secret Opaque        1      40d
speech-secret Opaque       15      26d
task-secret   Opaque        1      40d
○ khanhdiba Alice_Backend %
```

Figure 5.10: ConfigMaps and Secrets

5.1.2.5 Stateful Workload and Data Persistence

In conjunction with the stateless components, stateful workloads are implemented to provision dedicated databases for the microservices. To ensure robust data persistence across pod lifecycles, **StatefulSet** objects are utilized in place of standard deployments, as illustrated in **Figure 5.11**. Specifically, five **StatefulSets** are provisioned to manage the MongoDB database instances, while one additional **StatefulSet** is dedicated to hosting ChromaDB. To facilitate reliable internal communication and stable network identity resolution between these databases and their corresponding services, each database workload is exposed via a **Headless Service**.



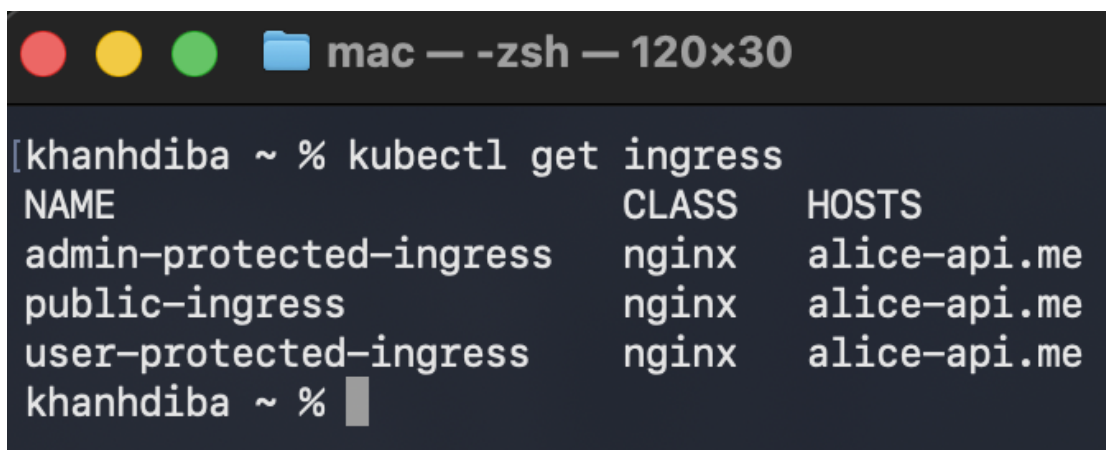
```
mac — -zsh — 120x30

[khanhdiba ~ % kubectl get statefulset
NAME                READY   AGE
chromadb            1/1     12d
mongodb-auth        1/1     41d
mongodb-image       1/1     10d
mongodb-schedule    1/1     41d
mongodb-speech      1/1     17d
mongodb-task        1/1     41d
khanhdiba ~ %
```

Figure 5.11: Persistent Volume Claims

5.1.2.6 Networking, Security and External Traffic

As previously established, backend services communicate internally via DNS. However, exposing these services to external clients requires a dedicated routing mechanism. In the Alice Backend, this is achieved using an NGINX Ingress Controller, as depicted in **Figure 5.9**. This controller manages the **Ingress** resources within the cluster and routes external traffic through a centralized **LoadBalancer**. As illustrated in **Figure 5.12**, the routing architecture comprises three distinct **Ingress** objects: **public-ingress** routes traffic to the API documentation, **admin-protected-ingress** handles administrative endpoints, and **user-protected-ingress** directs traffic to the core user-facing services.



```
mac — -zsh — 120x30

[khanhdiba ~ % kubectl get ingress
NAME                                CLASS    HOSTS
admin-protected-ingress            nginx    alice-api.me
public-ingress                     nginx    alice-api.me
user-protected-ingress             nginx    alice-api.me
khanhdiba ~ %
```

Figure 5.12: Ingress Configuration

To secure the protected routes, incoming requests are authenticated against two dedicated endpoints: `/api/auth/verify-user` and `/api/auth/verify-admin`. These endpoints evaluate the validity of the provided access tokens; if a token is deemed invalid, the system immediately rejects the request with an HTTP 401 Unauthorized status code. Furthermore, because the application is still in development and relies on external services with strict usage limits, public-facing endpoints (such as `/api/auth/signup`) also require temporary restriction. To prevent unauthorized account creation, a custom `X-API-KEY` header requirement was implemented, restricting access exclusively to the development team.

With the `LoadBalancer` bridge established, a custom domain name, `alice-api.me`, was registered through `Namecheap` to serve as the public entry point for the application. While registering the domain establishes accessibility, securing the communication channel is imperative. To transition from standard HTTP to secure HTTPS, Transport Layer Security (TLS) encryption was implemented utilizing `cert-manager` and Let's Encrypt. Within this ecosystem, `cert-manager` functions as a Kubernetes add-on that automates the provisioning and lifecycle management of TLS certificates, while Let's Encrypt acts as the trusted Certificate Authority (CA) issuing the certificates. **Figure 5.13** illustrates the integration of these technologies.



Figure 5.13: Cert Manager, Let's Encrypt, and Namecheap

5.2 Testing and Results

5.2.1 Test Data

The test data is systematically generated to ensure comprehensive coverage of the system's functional requirements. By applying Black Box testing methodologies, the dataset is categorized into three primary groups:

- **Valid Test Data:** This includes inputs that fall within the expected range and format, such as standard strings for event names, correctly formatted email addresses, and clear audio/image files for the AI modules (ASR and OCR).
- **Invalid Test Data:** This includes inputs designed to trigger error handling, such as null values, special characters in restricted fields, incorrect file extensions, and logical inconsistencies (e.g., an end date occurring before a start date).



- **Boundary Test Data:** Based on the Boundary Value Analysis (BVA) technique, this data focuses on the extreme edges of input limits, such as passwords at the minimum required length or text descriptions at the maximum character capacity.

5.2.2 Test Plan

The test plan focuses on Unit Testing to verify the correctness of individual components and modules within the system. The testing strategy strictly follows **Black Box** techniques, ensuring that the system is evaluated based on its specifications rather than its internal code structure. The following techniques are applied to generate the test cases:

- **Equivalence Class Partitioning (ECP):** This technique is used to divide the input data into several partitions of valid and invalid data. Each partition is treated as a single class, and a representative value from each class is selected for testing to reduce the total number of test cases while maintaining effectiveness.
- **Boundary Value Analysis (BVA):** Since errors frequently occur at the edges of input ranges, BVA is employed to test values at the minimum, maximum, and just outside these boundaries. This is particularly crucial for time-based scheduling and character-limited fields.

The execution of this plan ensures that all features—ranging from CRUD operations in personal management to complex AI-driven data extraction—meet the predefined functional standards and provide robust error feedback to the user.

Each test set table consists of six columns:

- **TC ID:** The unique identifier for the test case.
- **Title:** A brief description of the test case.
- **Input / Sample:** The specific input data, user actions, or preconditions applied during the test.
- **Expected Result:** The anticipated system behavior or output.
- **Status (A/I):** The actual execution outcome. The first value indicates the result tested on Android (A), and the second value indicates iOS (I).
- **Notes:** Additional observations, encountered defects, or relevant context regarding the test execution.

Test Set 1: Management Personal Event This test set evaluates the CRUD (Create, Read, Update, Delete) operations for personal events, specifically applying Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) to validate time logic and input constraints.

TC ID	Title	Input / Sample	Expected Result	Status (A/I)	Notes
EVT-01	Create Valid Event	Title: "Lab Meeting", Cat: Work, Start: 4:30, End: 5:30	Success; Event created successfully.	Pass / Pass	Positive Case
EVT-02	Empty Title	Title: "", Start: 4:30, End: 5:30	Exception: "Please enter title"	Pass / Pass	BVA: Min-1



TC ID	Title	Input / Sample	Expected Result	Status (A/I)	Notes
EVT-03	Invalid Time Range	Start: 10:00, End: 08:00	Error: End time cannot be before start time.	Pass / Pass	BVA: Logical
EVT-04	Zero Duration	Start: 4:30, End: 4:30	Exception: "Duration must be at least 1 min"	Pass / Pass	BVA: Lower Limit
EVT-05	Max Title Length	Title: [String of 100 characters]	Success; Title saved correctly.	Pass / Pass	BVA: Max Limit
EVT-06	Special Characters	Title: "Meeting @ BK #1!"	Success; Special chars rendered.	Pass / Pass	ECP: Char sets
EVT-07	Recurring Event	Repeat: "Daily", End Repeat: "Never"	Series of events generated in DB.	Pass / Pass	ECP: Frequency
EVT-08	Day View	Action: Click "Day View"	Correct timeline visualization.	Pass / Pass	UI Check
EVT-09	Week View	Action: Click "Week View"	Correct 7-day column rendering.	Pass / Pass	UI Check
EVT-10	Update Category	Change from "Work" to "Personal"	Icon and Color label update correctly.	Pass / Pass	CRUD Update
EVT-11	Reschedule Date	Date: "21 Apr" → "22 Apr"	Database updates; event moves on UI.	Pass / Pass	CRUD Update
EVT-12	Delete Event	Action: Click "Delete" → Confirm	Event removed from UI and DB.	Pass / Pass	CRUD Delete
EVT-13	Delete Instance	Action: Delete one instance of a series	One day removed; rest of series remains.	Pass / Pass	CRUD Delete

Test Set 2: Management Personal Task This test set verifies the management of personal tasks, focusing on input constraints, hierarchical structures (subtasks), and UI/UX interactions such as filtering and gestures.

TC ID	Title	Input / Sample	Expected Result	Status (A/I)	Notes
TSK-01	Basic Task Creation	Title: "Lab DSA"	Task pinned to the list successfully.	Pass / Pass	Positive Case
TSK-02	Long Name Handling	Title: "North Korea" (100+ chars)	Text truncated with "..." but stored correctly.	Pass / Pass	BVA: Max Limit
TSK-03	Empty Title	Title: "" (Empty string)	Nothing create.	Pass / Pass	BVA: Min-1
TSK-04	Special Characters	Title: "Finish Assignment #1! @Room 609"	Symbols rendered correctly in UI.	Pass / Pass	ECP: Char set
TSK-05	Priority Assignment	Title: "Submit Project", Priority: High	Task marked with High Priority icon.	Pass / Pass	ECP: Priority
TSK-06	Create Subtask	Parent: "Lab DSA", Subtask: "Coding part"	Subtask linked and indented under parent.	Pass / Pass	Logic: Hierarchy
TSK-07	View Task List	Action: Open Task Module	All personal tasks displayed correctly.	Pass / Pass	UI Check
TSK-08	Mark Completed	Action: Click checkbox on "Lab DSA"	Task name has a strike-through line.	Pass / Pass	CRUD Update
TSK-09	Revert Completion	Action: Uncheck completed task	Task returns to Pending (no strike-through).	Pass / Pass	CRUD Update



TC ID	Title	Input / Sample	Expected Result	Status (A/I)	Notes
TSK-10	Edit Task Name	"Lab DSA" → "Final Lab DSA"	Name updated successfully in Database.	Pass / Pass	CRUD Update
TSK-11	Remove Task from Goal	Action: Click the three-dot menu on task "Lab DSA" → "No Goal".	Task is detached from the goal.	Pass / Pass	Goal Management
TSK-12	Swipe to Delete	Action: Drag task to the left	Task is removed via gesture.	Pass / Pass	UX: Gesture

Test Set 3: Management Personal Goal This test set focuses on the management of long-term goals, specifically validating the progress tracking logic, task-to-goal relationships, and data visualization of goal analytics.

TC ID	Title	Input / Sample	Expected Result	Status (A/I)	Notes
GOL-01	Create Valid Goal	Title: "IELTS 7.5", Deadline: 31 May, Color: Purple	Goal framework created successfully.	Pass / Pass	Positive Case
GOL-02	Empty Title	Title: "", Deadline: 31 May	Exception: "Goal name is required".	Pass / Pass	BVA: Min-1
GOL-03	Auto-Progress	Complete 3 out of 10 subtasks	Progress bar updates to 30%.	Pass / Pass	Core Logic
GOL-04	Completion State	Mark the final subtask as done	Goal status changes to "Completed".	Pass / Pass	Logic Transition
GOL-05	Link Task to Goal	Action: Drag Task → Link to Goal	Task is now part of Goal hierarchy.	Pass / Pass	ECP: Relationship
GOL-06	Unlink Task	Action: Drag Task to the left	Task returns to independent list.	Pass / Pass	UX/Logic
GOL-07	Analytics View	Action: Open "Insights" page	Progress charts rendered correctly.	Pass / Pass	UI: Data Viz
GOL-08	Zero Tasks Progress	Goal has 0 tasks linked	Progress bar displays 0%.	Pass / Pass	ECP: Boundary
GOL-09	Delete Goal (Keep)	Option: "Delete Goal only"	Goal removed; tasks become independent.	Pass / Pass	CRUD: Logic
GOL-10	Cascade Delete	Option: "Delete Everything"	Goal and all subtasks wiped from DB.	Pass / Pass	Integrity
GOL-11	Identity Color	Choose color: "Blue"	UI elements for Goal update to blue.	Pass / Pass	UI/UX
GOL-12	Overdue Status	Current Date > End Date (by 7 days)	Status shows "7 days overdue".	Pass / Pass	BVA: Time

Test Set 4: Manual Login This test set evaluates the authentication module, specifically focusing on input validation, security constraints (SQL injection and brute force), and environment handling (network status).



TC ID	Title	Input / Sample	Expected Result	Status (A/I)	Notes
LOG-01	Valid Login	Email: test@gmail.com, Pass: Abc@1234	Success: Redirect to Home.	Pass / Pass	Positive Case
LOG-02	Empty Email	Email: "", Pass: Abc@1234	Error: "Email is required".	Pass / Pass	BVA: Min-1
LOG-03	Empty Password	Email: test@gmail.com, Pass: ""	Error: "Password is required".	Pass / Pass	BVA: Min-1
LOG-04	Unregistered Email	Email: newuser@gmail.com, Pass: Abc@1234	Error: "User not found".	Pass / Pass	ECP: Invalid Email
LOG-05	Invalid Format	Email: test.gmail.com (No @)	Error: "Invalid email format".	Pass / Pass	ECP: Format
LOG-06	Wrong Password	Email: test@gmail.com, Pass: WrongPass123	Error: "Incorrect password".	Pass / Pass	ECP: Invalid Pass
LOG-07	Case Sensitivity	Pass: abc@1234 (all lower)	Error: "Incorrect password".	Pass / Pass	ECP: Sensitivity
LOG-08	Max Length Email	Email: [64chars]@gmail.com	Success: System accepts length.	Pass / Pass	BVA: Max Limit
LOG-10	Password Min-Limit	Pass: 12345678 (8 chars)	Success (If password matches).	Pass / Pass	BVA: Min Limit
LOG-11	Password Below Min	Pass: 1234567 (7 chars)	Error: "Password too short".	Pass / Pass	BVA: Boundary -1
LOG-12	Auto-trim Spaces	Email: " test@gmail.com "	Success: System trims spaces.	Pass / Pass	ECP: Auto-trim
LOG-13	Brute Force	Fails 5 times consecutively	Error: "Too many attempts".	N/A / N/A	Security ECP
LOG-14	SQL Injection	Email: ' OR 1=1 -, Pass: any	Error: "Invalid credentials".	Pass / Pass	ECP: Security
LOG-15	Special Chars	Email: test+extra@gmail.com	Success: Validates alias email.	Pass / Pass	ECP: Valid chars
LOG-16	Network Offline	App open, No internet	Error: "No internet connection".	Pass / Pass	ECP: Environment

Test Set 5: Automatic Speech Recognition (ASR) This test set evaluates the system's ability to convert voice commands into text using the Whisper model. It focuses on audio boundaries, environmental robustness (noise/echo), and linguistic variations such as accents and code-switching.

TC ID	Title	Input / Sample	Expected Result	Status (A/I)	Notes
ASR-01	Min Duration	0.5s audio (silence/click)	System ignores or prompts: "No voice detected".	Pass / Pass	BVA: Lower limit
ASR-02	Max Duration	60s continuous speech	System processes or warns: "Audio too long".	Pass / Pass	BVA: Upper limit
ASR-03	Noise Resilience	"Remind me to study" + 50dB street noise	Whisper correctly extracts the text.	Pass / Pass	Robustness test
ASR-04	Low Volume	Speaking at 10-15dB (whispering)	Whisper amplifies and transcribes correctly.	Pass / Pass	BVA: Low amp



TC ID	Title	Input / Sample	Expected Result	Status (A/I)	Notes
ASR-05	Clipping Audio	Shouting into mic (Signal > 0dB)	System recovers text despite distortion.	Pass / Pass	BVA: High amp
ASR-06	Mid-speech Silence	"Add meeting... (5s pause) ...at 9 AM"	System waits and joins segments into one string.	Pass / Pass	ECP: Pause logic
ASR-07	Non-speech Artifacts	"Họp nhóm" + Loud coughing/typing	Whisper filters artifacts and extracts text.	Pass / Pass	ECP: Filtering
ASR-08	Code-switching	"Set schedule lúc 10 giờ PM cho tôi"	Correct transcription of mixed Vi-En words.	Pass / Pass	ECP: Bilingual
ASR-09	Regional Accents	Strong Central/Southern accent	System maps phonetic variations to standard text.	Pass / Pass	ECP: Accents
ASR-10	Rapid Speech	Speaking at 200+ words/min	Whisper accurately captures all entities.	Pass / Pass	BVA: Speed
ASR-11	Stuttering Speech	"T...t...thêm... l...lịch... học"	System normalizes stutter and returns clean text.	Pass / Pass	ECP: Fluency
ASR-12	Echo/Reverb	Empty hall environment with high reverb	AI de-reverbs and identifies the intent.	Pass / Pass	ECP: Environment
ASR-13	Technical Jargon	"Họp môn DSA tại phòng B6"	Correct transcription of "DSA" and "B6".	Pass / Pass	ECP: Jargon
ASR-14	Network Dropout	Internet disconnected mid-upload	System saves buffer or shows "Connection Error".	Pass / Pass	ECP: Connection
ASR-15	Overlapping Voice	Two people talking simultaneously	System isolates primary voice or transcribes both.	Pass / Pass	ECP: Overlap

Test Set 6: Optical Character Recognition (OCR) This test set evaluates the system's ability to extract and interpret text from images using Llama V4. It covers a wide range of scenarios, including handwriting recognition, resolution boundaries, orientation adjustments, and the extraction of complex tabular data.

TC ID	Title	Input / Sample	Expected Result	Status (A/I)	Notes
OCR-01	Digital Text Extraction	Screenshot of a Google Calendar event	Llama V4 extracts text with 99%+ accuracy.	Pass / Pass	Positive Case
OCR-02	Handwriting Recognition	Photo of a clear note: "Exam May 20"	System identifies "Exam" and "May 20".	Pass / Pass	ECP: Handwriting
OCR-03	Cursive Handwriting	Photo of cursive text: "Meeting with Prof. An"	AI correctly transcribes proper names.	Pass / Pass	Robustness
OCR-04	Low Resolution	Image size: 200 x 200 pixels	Error: "Image resolution too low".	Pass / Pass	BVA: Lower Limit
OCR-05	High File Size	15MB 8K Resolution Image	Error: "File size exceeds 10MB limit".	Pass / Pass	BVA: Upper Limit
OCR-06	Rotated Image	Schedule photo rotated 90/180 degrees	AI auto-rotates and extracts text correctly.	Pass / Pass	ECP: Orientation
OCR-07	Table Extraction	Photo of a Weekly Timetable (5-10 rows)	All rows extracted as individual event objects.	Pass / Pass	ECP: Complexity
OCR-08	Date Interpretation	Text in image: "Meeting Tomorrow"	AI calculates correct date based on "Today".	Pass / Pass	ECP: AI Logic



TC ID	Title	Input / Sample	Expected Result	Status (A/I)	Notes
OCR-09	Dim Lighting	Photo taken in a dark, underexposed room	AI enhances visibility and reads text.	Pass / Pass	ECP: Lighting
OCR-10	Mixed Language	"Học DSA lúc 2 PM" (Mixed Vi-En)	Correct extraction of bilingual entities.	Pass / Pass	ECP: Language
OCR-11	Background Noise	Flyer on a busy, colorful wall background	AI isolates text from background noise.	Pass / Pass	Robustness
OCR-12	No Text Detected	Photo of a landscape (No text)	System prompts: "No text found in image".	Pass / Pass	ECP: Exception
OCR-13	Acronym Extraction	"Seminar at BK.B6"	System identifies "BK.B6" as location.	Pass / Pass	ECP: Technical
OCR-14	Format Boundary	Attempt to upload .tiff or .bmp	Error: "Unsupported image format".	Pass / Pass	ECP: Formats
OCR-15	AI Timeout	Llama V4 API takes > 30s to respond	UI shows: "Processing timeout, please retry".	Pass / Pass	Reliability

5.3 Non-functional Testing

This section evaluates the system's quality attributes, ensuring that the application remains robust, responsive, and user-friendly under various conditions as defined in the Non-Functional Requirements (NFRs).

5.3.1 Usability Test

The usability testing phase verifies the user interface (UI) and user experience (UX) standards. It ensures the system is accessible, consistent, and easy to learn for new users, directly addressing requirements NFR-Usab-1 through NFR-Usab-6.

ID	Requirement Reference	Test Scenario	Success Criteria
USA-01	Interface Consistency (NFR-Usab-1)	Audit UI components (buttons, icons, nav) across all functional modules.	$\geq 90\%$ of components follow a unified design style.
USA-02	Learnability (NFR-Usab-2)	Measure time taken for 10 first-time users to create a schedule and update water intake.	80% of users complete tasks within 10 minutes without help.
USA-03	Theme Support (NFR-Usab-3)	Toggle between Light and Dark mode during active operations.	Theme applies globally within 1s and persists after restart.
USA-04	Accessibility (NFR-Usab-4)	Validate UI screens using WCAG 2.1 Level AA contrast and screen reader tools.	90% of screens meet readability and contrast guidelines.
USA-05	Visualization (NFR-Usab-5)	Switch between Daily and Weekly schedule views.	Navigation/View transition completes in $< 2s$.
USA-06	Voice Usability (NFR-Usab-6)	10 users issue 5 voice commands each to create schedules.	80% of commands are correctly interpreted on the first attempt.



5.3.2 Performance Test

Performance testing measures the system's stability and responsiveness under load. This includes concurrent user stress testing, AI processing latency, and notification accuracy (NFR-Perf and NFR-Reli requirements).

ID	Requirement	Reference	Test Scenario	Success Criteria
PER-01	Response Time (NFR-Perf-2)		Measure average response time for schedule viewing and task updates.	Average response time $\leq$ 2s.
PER-02	AI Processing (NFR-Perf-3)		Trigger 50 concurrent AI-powered quick scheduling requests.	Schedule generation completed within 5s per request.
PER-03	Data Loading (NFR-Perf-4)		Load 12 months of historical calendar data for a user profile.	Full data rendering completed within 3s.
PER-04	Notification Timing (NFR-Perf-5 / NFR-Reli-4)		Schedule 100 reminders and measure the offset from the target trigger time.	All notifications trigger within ± 1 minute of scheduled time.
PER-05	Fault Recovery (NFR-Reli-3)		Simulate a server failure and measure automatic restoration time.	Core functions restored within 5 minutes; zero data loss.
PER-06	Timezone Accuracy (NFR-Reli-5)		Set reminders for users in UTC-5 and UTC+7 timezones.	Reminders trigger at correct local time with zero offset error.

Chapter 6

Work Evaluation and Future Improvement

In Chapter 6, the authors conclude the specialized project. This chapter consists of two parts: an evaluation of the work completed in the first phase and proposed improvements for the next phase.

6.1 Work Evaluation

Throughout the project implementation process, our team organized the workflow effectively and clearly divided responsibilities among members. By applying the Agile Scrum methodology with iterative cycles of planning, implementation, reporting, and evaluation, and by using the Trello task management platform, the team was able to manage project requirements while balancing workloads from other subjects. Regular internal reviews and reporting sessions with the instructor enabled timely feedback and guided subsequent development iterations.

During the project, the team encountered several challenges, particularly in system design and the development of logical system views derived from requirements. In addition, team members faced constraints due to differing priorities across other major subjects. To address these issues, the team adapted its collaboration approach by increasing the frequency of meetings from one session per week to two or three sessions when necessary. These additional meetings provided opportunities for members to share difficulties, discuss technical problems, and collaboratively identify solutions, which contributed to improved project outcomes.

The project successfully delivered all required system documentation and analysis. Furthermore, the team selected an appropriate technology stack, designed application UI mockups, and implemented the voice-handling feature using an AI-based model. These deliverables demonstrate both the technical progress achieved and the effectiveness of team collaboration.

From an evaluation perspective, the implemented system satisfies the core functional requirements defined during the analysis phase, including task management, schedule management, and voice-based schedule creation. Functional validation through iterative development and instructor feedback confirmed that the primary user workflows operate as intended. While large-scale performance testing was beyond the scope of this academic project, the system exhibited



stable behavior in a controlled testing environment. A key limitation lies in the accuracy of voice interpretation for complex or ambiguous inputs, particularly those involving overlapping temporal expressions, which indicates directions for future refinement of the natural language processing component.

Overall, the project provided valuable experience in software engineering practices and system development. The outcomes achieved form a solid foundation for further enhancement and future development of the proposed work-management application.

6.2 Future Improvements

In the next development phase, the team plans to implement the application based on the completed system documentation and the selected technology stack. Initial development may progress at a moderate pace as efforts are concentrated on studying the chosen technologies, establishing a robust system architecture, and optimizing the overall code structure. Once these foundational components are stabilized, development efficiency is expected to improve significantly.

To enhance system reliability and software quality, a dedicated testing environment will be established. The team plans to adopt continuous integration and continuous deployment (CI/CD) practices to support a more reliable development workflow, reduce integration issues, and shorten development cycles. Knowledge related to DevOps practices, software testing, and mobile application development will be continuously reinforced throughout weekly development iterations. Following the Agile methodology, system features will be regularly reviewed and refined across future sprints, particularly those that were not fully specified during earlier phases.

In addition to technical enhancements, the team intends to further explore concepts related to work-life balance, productivity techniques, and effective personal management. These insights will support the design of features that are both practical and aligned with contemporary user needs.

Finally, while delivering a functional and reliable application remains the primary objective, the development process itself will continue to emphasize both technical and soft skill improvement. Regular weekly meetings will be used to reinforce individual learning, encourage knowledge sharing, and facilitate constructive feedback. Through this iterative and collaborative approach, the team expects to achieve continuous improvement in both the project outcomes and overall development competence.

References

- [1] AWS (n.d.). What is Flutter?. Retrieve from: <https://aws.amazon.com/what-is/flutter/>
- [2] Bonnie Eisenman (2015) What is React Native?. Retrieve from: <https://www.oreilly.com/library/view/learning-react-native>
- [3] Ben Lutkevich (2022) Definition Kotlin?. Retrieve from: <https://www.techtarget.com/whatis/definition/Kotlin/>
- [4] Wikipedia. Django (web framework). Retrieve from: [https://en.wikipedia.org/wiki/Django_\(web_framework\)](https://en.wikipedia.org/wiki/Django_(web_framework))
- [5] Django (n.d.). Official Website. Retrieve from: <https://www.djangoproject.com/>
- [6] W3schools (n.d.). Django Introduction. Retrieve from https://www.w3schools.com/django/django_intro.php
- [7] Wikipedia (n.d.). MongoDB. Retrieve from: <https://en.wikipedia.org/wiki/MongoDB>
- [8] Mark Richards, Neal Ford (2020). Fundamentals of Software Architecture. O'REILLY
- [9] Google Cloud (n.d.). What is Microservices Architecture? Retrieve from: <https://cloud.google.com/learn/what-is-microservices-architecture>
- [10] BMI (n.d.). What is Java Spring Boot?. Retrieve from: <https://www.ibm.com/think/topics/java-spring-boot>
- [11] Wikipedia (n.d.). Node.js. Retrieve from: <https://en.wikipedia.org/wiki/Node.js>
- [12] Node.js (n.d.). About Node.js®. Retrieve from: <https://nodejs.org/en/about>
- [13] FastAPI (n.d.). Official Website. Retrieve from: <https://fastapi.tiangolo.com/>
- [14] Wikipedia (n.d.). FastAPI. Retrieve from: <https://en.wikipedia.org/wiki/FastAPI>
- [15] Wikipedia (n.d.). Document-oriented database Retrieve from: https://en.wikipedia.org/wiki/Document-oriented_database
- [16] Wikipedia (n.d.). PostgreSQL. Retrieve from: <https://en.wikipedia.org/wiki/PostgreSQL>
- [17] PostgreSQL (n.d.). Official Website. Retrieve from: <https://www.postgresql.org/>



- [18] Nearform (2023). Using Google Calendar with Natural Language via LangChain. Retrieve from: <https://nearform.com/insights/using-google-calendar-with-natural-language-via-langchain/>
- [19] Reclaim AI (2025). AI Calendar Assistant for Smart Scheduling and Productivity Optimization. Retrieve from: <https://reclaim.ai/>
- [20] OpenAI (2022). Introducing Whisper. Retrieve from: <https://openai.com/index/whisper/>
- [21] Radford, A., Kim, J.W., Xu, T., Brockman, G., McLeavey, C., Sutskever, I. (2022). Robust Speech Recognition via Large-Scale Weak Supervision. Retrieve from: <https://github.com/openai/whisper>
- [22] Sleep Cycle (n.d.). The Technology That Powers Sleep Cycle's App. Retrieve from: <https://www.sleepcycle.com/features/the-technology-that-powers-sleep-cycle/>
- [23] Notion (2024). Introducing Notion Calendar - An Integrated Calendar for Work and Life. Retrieve from: <https://www.notion.com/blog/introducing-notion-calendar>
- [24] Khadivi, M. et al. (2023). Deep Reinforcement Learning for Machine Scheduling: Methodology, the State-of-the-Art, and Future Directions. Retrieve from: <https://arxiv.org/abs/2310.03195>
- [25] Google (2023). Introducing Gemini: Google's most capable AI model yet. Retrieve from: <https://blog.google/technology/ai/google-gemini-ai/>
- [26] Wikipedia (n.d.). Gemini (language model). Retrieve from: [https://en.wikipedia.org/wiki/Gemini_\(language_model\)](https://en.wikipedia.org/wiki/Gemini_(language_model))
- [27] Facebook (n.d.). Duckling - Language, engine, and tooling for expressing, testing, and evaluating composable language rules on input strings. Retrieve from: <https://github.com/facebook/duckling>
- [28] Google Cloud (n.d.). What is cloud native? Retrieve from: <https://cloud.google.com/learn/what-is-cloud-native>
- [29] Design Guru (n.d.). What is containerization (e.g. using Docker) and how has it changed the way we deploy and scale applications? Retrieve from: [Design Guru article on containerization](#)
- [30] Docker (n.d.). Docker Official Page Retrieve from: <https://www.docker.com/>
- [31] Kubernetes (n.d.). Kubernetes Official Page Retrieve from: <https://www.k8s.io/>
- [32] Digital Ocean (n.d.). Digital Ocean Official Page Retrieve from: <https://www.digitalocean.com/>
- [33] GroqCloud (n.d.). GroqCloud Official Page Retrieve from: <https://console.groq.com/>
- [34] Google AI Studio (n.d.). Google AI Studio Official Page Retrieve from: <https://aistudio.google.com/>
- [35] ElevenLabs (n.d.). ElevenLabs Official Page Retrieve from: <https://elevenlabs.io/>



- [36] Tom Krantz , Jim Holdsworth , Matthew Kosinski from IBM (n.d.). What is a vector database? Retrieve from: <https://www.ibm.com/think/topics/vector-database>
- [37] Tom Krantz , Alexandra Jonker from IBM (n.d.). What are RAG vector database? Retrieve from: <https://www.ibm.com/think/topics/rag-vector-database>
- [38] Chroma (n.d.) Chroma Documentations. Retrieve from: <https://docs.trychroma.com/docs/overview/introduction>
- [39] Artificial Analysis (n.d.). Comparison of Models: Intelligence, Performance & Price Analysis. Retrieve from: <https://artificialanalysis.ai/models>

Appendix A

Appendix: Task Assignment

Nguyen Anh Khoa

- Functional Requirements
- Mobile Application Technology Research
- Use Case Design of Schedule Management Module
- Use Case Scenario of Schedule Management Module
- Activity Diagram Design
- Mobile App Implementation

Tran Chi Tai

- Project Scope Define
- Non-Functional Requirements
- Image-to-Schedule Design and Implementation
- Speech-to-Schedule/Goal-Task Design and Implementation
- Related Work Research
- Use Case Design of Task Management Module
- Use Case Scenario of Task Management Module
- Architecture Diagram Design
- Class Diagram Design
- ERD Design
- Testing



Dinh Ba Khanh

- Domain Context Analysis
- Mobile Application Technology Research
- Support s2s Theory, Design, Implementation and Testing
- Backend Technology Research
- Use Case Design of Authentication and Personal Information
- Use Case Scenario of Authentication and Personal Information
- Sequence Diagram Design
- Component Diagram Design
- Back-end Implementation

Appendix B

Appendix: Timeline

Sprint 1 12/01/2026 - 01/02/2026

- Learn Kubernetes and Docker for Backend
- Implement Mobile UI/UX Schedule
- Replace NLP Module with LLM API for Schedule

Sprint 2 02/02/2026 - 15/02/2026

- First Repository and Kubernetes Cluster Setup
- Implement Mobile UI/UX Authorization
- Implement Speech-to-Schedule/Task-Goal Module

Sprint 3 16/02/2026 - 01/03/2026

- Implement Mobile UI/UX Goal-Task
- Implement Backend with Minikube
- Implement RAG for Speech-to-Schedule/Task-Goal Module

Sprint 4 02/03/2026 - 15/03/2026

- Implement Mobile UI/UX Speech
- Implement Backend and Deploy to Cloud
- Implement Image-to-Schedule Module

Sprint 5 16/03/2026 - 29/03/2026

- Frontend and Backend Integration
- Implement Backend and Deploy to Cloud
- Backend and AI Module Integration



Sprint 6 30/03/2026 - 12/04/2026

- Frontend and Backend Integration
- Implement Backend and Deploy to Cloud
- Backend and AI Module Integration

Sprint 7 13/04/2026 - 26/04/2026

- System Deployment and Performance Measurement
- Bug Fixes and Error Resolution During Testing

Sprint 8 27/04/2026 - 10/05/2026

- Final Review and Comparison with Initial Design
- Report Writing and Thesis Defense Preparation